



MeerK40t User Manual

A hands-on tutorial

Design it, plan it, cut it — from your first square to production-style batches on a CO₂ cutter, a diode engraver, a fibre galvo or a K40.

Written for MeerK40t 0.9.9040

The MeerK40t project and contributors

17 September 2026

This manual is a tutorial, not a reference: it teaches a workflow, in order, with exercises.

How to use this manual

MeerK40t is a laser control program: it holds your artwork, turns that artwork into machine instructions, and talks to the laser. This manual teaches you to do that in the order you will actually do it. Read it front to back once; after that use the contents pages and the appendices.

Who this is for

You have a laser cutter or engraver and you want to use it. You do not need to know Python, and you do not need to know how MeerK40t is built internally. You do need to know the difference between rastering and cutting — Chapter 1 covers the vocabulary.

How the chapters are arranged

Part I — Getting started

What the program is, how to install it, how the window is organised, and how to connect a machine.

Part II — Your first projects

Two complete jobs, from empty scene to finished part: a vector cut and a raster engrave. Then the full operation vocabulary and how to dial settings in with test grids.

Part III — Working with artwork

Files in and out, the element tree, selection, transforms, node editing, text, images and tracing.

Part IV — The machine

Where the job will be cut, in what order, at what cost in time, plus the console, scripting, remote control and troubleshooting.

Appendices

Shortcut tables, a console command reference, per-driver settings, a glossary and where to get help.

Conventions

This	Means this
File ► Open	A menu path. Start at the menu bar, finish on the last item.
Start	A button, checkbox or field caption you can read on screen.
Tree	The name of a dockable window (a <i>pane</i>).
Ctrl+S	A keyboard shortcut.
rect 2cm 2cm 1cm 1cm	Something typed into the MeerK40t console (Chapter 19).
bed_width	The internal name of a setting; useful in the console and in scripts.
tag.svg	A file name, path or file format.

Safety, briefly

A laser cutter is a Class 4 laser product wrapped around a machine tool. It starts fires, blinds people and burns through things that should not burn. A software manual is not a substitute for your machine's own manual, but three rules belong in any manual:

Warning

1. Never run a job with the lid open or the enclosure removed. The interlocks exist because the beam is invisible and instant.
2. Never leave a running job unattended, and keep a suitable extinguisher or a CO₂-safe method of smothering a fire next to the machine.
3. Know which materials you may cut. PVC, polyurethane, vinyl and ABS release chlorine or cyanide; coated and composite materials may contain glues you have not considered. If you cannot identify the material, do not cut it.

Other practical rules — air assist on, focus checked, workpiece secured, settings tested on scrap — appear as **At the machine** boxes next to the tutorials that need them.

Contents

How to use this manual	i
I Getting started	1
1 What MeerK40t does, and what you need	2
1.1 The one-paragraph version	2
1.2 The three things the program does	2
1.3 Laser vocabulary	2
1.4 Which machines are supported	4
1.5 What MeerK40t is not	4
1.6 What you need before Chapter 5	5
2 Installing and launching MeerK40t	6
2.1 Three ways to install	6
2.2 The first launch	7
2.3 Starting the program in different ways	7
2.4 Keeping it up to date	10
3 A tour of the interface	11
3.1 The five regions	11
3.2 The scene	12
3.3 The tree	13
3.4 Laser-Control and the other machine panes	13
3.5 Panes: hiding, showing and resetting	14
3.6 The menus	14
3.7 The ribbon	15
3.8 The status bar	16
3.9 Windows that are not panes	16
3.10 The simplified interface	16
4 Connecting your laser	18
4.1 Which driver is my machine?	18
4.2 The device list	19
4.3 Telling MeerK40t how big the bed is	19
4.4 Connecting each family	20
4.5 USB permissions on Linux	22
4.6 Proving the connection works	22
4.7 Moving the head safely	22
4.8 When it does not connect	23

II	Your first projects	25
5	Your first cut	26
5.1	Before you press Start	26
5.2	Step 1 — Start a project and check the bed	27
5.3	Step 2 — Draw the shape	27
5.4	Step 3 — Create the two operations	27
5.5	Step 4 — Assign the artwork	28
5.6	Step 5 — Set speed and power	28
5.7	Step 6 — Position and check before burning	29
5.8	Step 7 — Run the job	29
5.9	Step 8 — Judge the result, then save	30
6	Your first engrave	32
6.1	Raster: filling vector artwork	32
6.2	Image: engraving a picture	33
6.3	Big images, tiles and keyholes	35
7	Operations in depth	37
7.1	The five burn operations	37
7.2	Creating an operation	38
7.3	Attaching artwork to an operation	38
7.4	Colour classification	39
7.5	The settings of a burn operation	39
7.6	Effects: hatch, wobble and warp	40
7.7	Estimating before you commit	41
8	Dialling in settings	43
8.1	The method	43
8.2	Test grids: the Parameter-Test window	43
8.3	Storing the answer: the Material Library	44
8.4	Kerf: cutting parts that fit	45
8.5	The machine's own calibration	46
8.6	Beyond speed and power	46
III	Working with artwork	48
9	Files, formats and saving	49
9.1	The project file is an SVG	49
9.2	The Recent list and drag and drop	50
9.3	What can be loaded, and what can be saved	50
9.4	Exporting a machine job file	51
9.5	Import behaviour and the preferences that change it	51
9.6	Pasting from other programs	52
9.7	Autosave, notes and startup commands	52
9.8	The console route	53
10	The tree, and selecting things	54
10.1	Three branches, one document	54
10.2	Selecting: scene and tree	54
10.3	The right-click menus, branch by branch	55
10.4	Dragging in the tree	57

10.5	Remarks in practice	57
10.6	Undo, redo and the editing history	57
11	Moving, scaling, rotating, aligning	59
11.1	The transformation handles	59
11.2	Exact numbers: the Position pane	59
11.3	Nudging with the keyboard	60
11.4	Alignment, distribution and arrangement	60
11.5	Reference objects	61
11.6	Snapping and magnets	61
12	Paths and nodes	64
12.1	What an element actually is	64
12.2	Drawing	64
12.3	Editing nodes	65
12.4	Reshaping paths	66
12.5	Generating shapes you cannot draw	66
13	Text and fonts	68
13.1	Two kinds of text	68
13.2	Choosing a font	68
13.3	Working with text elements	69
13.4	Text that changes between burns: wordlists	69
13.5	Turning text into geometry	70
14	Images: editing, splitting and tracing	72
14.1	Preparing the picture first	72
14.2	Image manipulations in the tree	72
14.3	The Image Splitter: pictures bigger than the bed	73
14.4	The image command tree	74
14.5	Tracing: from bitmap to cuttable vector	74
IV	The machine	77
15	Where the job runs	78
15.1	The scene is the bed	78
15.2	Positioning the workpiece instead of the artwork	78
15.3	Placements: job start positions	79
15.4	Registration marks	79
15.5	Reading positions off the machine: Place Template	79
15.6	The camera	80
16	Order, optimisation and simulation	83
16.1	The plan	83
16.2	The optimisation settings	84
16.3	The Simulation window	85
16.4	The spooler and running many jobs	85
17	Rotary, coolant and remote control	87
17.1	Rotary: engraving on cylinders	87
17.2	Coolant, air assist and sleep	88
17.3	Remote control over the network	89

17.4 Connecting to other software	90
18 The console, and scripting	92
18.1 Opening the console and reading it	92
18.2 Asking the console for help	92
18.3 The syntax rules that matter	93
18.4 The job pipeline in one line	93
18.5 A complete example, typed	93
18.6 Settings from the console	94
18.7 Selecting and acting from the console	94
18.8 Aliases: your own commands	94
18.9 Scripts: files, batches and the CLI	94
18.10 Batch production from a spreadsheet	95
18.11 Driving the interface from the console	95
19 Preferences, housekeeping and troubleshooting	97
19.1 The Preferences window	97
19.2 Appearance, language and updates	98
19.3 Where your data actually lives	99
19.4 Troubleshooting by symptom	99
A Keyboard shortcuts and mouse controls	102
A.1 Movement and nudging	102
A.2 Selection, editing and document keys	104
A.3 Machine, windows and panels	105
A.4 Test, rotary, wizard and reset bindings	106
A.5 Keys with no default binding	106
A.6 Menu accelerators that are not in the keymap	107
A.7 Mouse and trackpad	108
A.8 Where these defaults come from	109
B Console command reference	111
C Machine settings by driver	126
D Glossary, help and reporting problems	138

Part I

Getting started

Chapter 1

What MeerK40t does, and what you need

Before you touch a menu, it is worth being clear about what this program is for and what the words mean. This chapter is the map; the rest of the manual is the walk.

1.1 The one-paragraph version

MeerK40t is a free, open-source program that drives laser cutters and engravers. You give it artwork — a drawing, a photograph, a shape you drew in the program itself — tell it how that artwork should be burned, and it converts both into the precise stream of movements and laser pulses your machine understands. It runs on Windows, macOS, Linux and the Raspberry Pi, supports several popular controller families, and is extensible through plugins. It is free software under the MIT licence.

Behind the name and the spelling

“MeerK40t” is a pun on the K40, the inexpensive Chinese CO₂ laser cutter that the project originally targeted, and on “meerkat”. “K40” with “t” appended gives “K40t”, read as the sound in “meerkat”. You will see the name written both *MeerK40t* (the program) and *Meerk40t* (variant capitalisation in older text, screenshots and file names). It is the same thing.

1.2 The three things the program does

Everything in MeerK40t belongs to one of three activities, and every chapter of this manual is about one of them.

1. **Hold artwork** — shapes, paths, text and images live in a document called the *element tree* (Chapter 10 covered later). This is where you import, arrange, edit and position.
2. **Describe the burn** — each group of artwork is attached to an *operation*: cut it, engrave it, raster it, dot it. Operations carry the machine settings: speed, power, passes, resolution and so on.
3. **Run the job** — MeerK40t turns operations into *cutcode*, optimises the order of the moves, wraps it in a *job*, hands it to a spooler and streams it to the controller.

1.3 Laser vocabulary

You can use MeerK40t without this section, but you will misread the screens without it. Learn these ten terms.

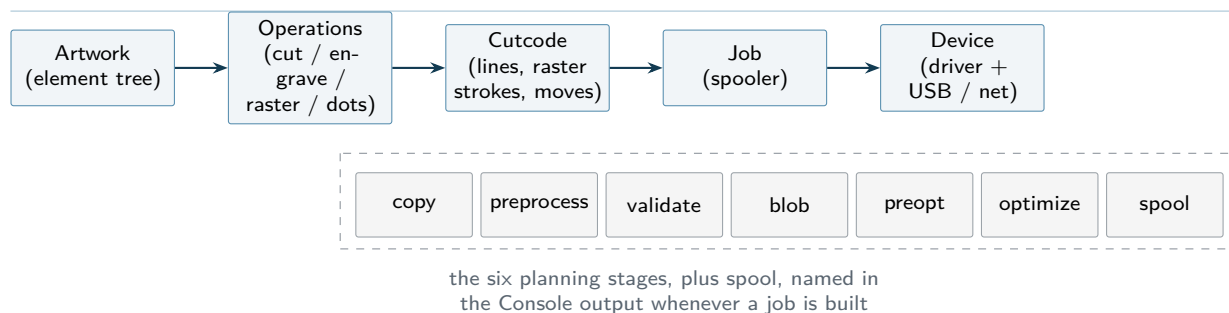


Figure 1.1: The path from artwork to laser. The upper row is the journey you will follow all manual long; the lower row is the planning pipeline described in Chapter 16 and printed in the console whenever a job is built.

Vector

A shape described mathematically: a line, an arc, a bezier curve. Vectors scale without pixellating. SVG and DXF files are vector files.

Raster

A grid of pixels, as in a photograph or a PNG. Engraving a raster means burning it line by line, switching the beam on and off (or varying its power) as the head sweeps across.

Cut / Score

A vector burned at high power and low speed, through the material. A *score* or *mark* is the same path burned lightly so it is visible but not severed — a fold line, for example. In MeerK40t both are done with a **Cut** or **Engrave** operation, differing only in settings.

Engrave

A vector burned at engraving settings: the beam traces the path, removing a thin line of material. Dark, thin, precise — but slow over a large area. Filling an area by engraving a dense set of lines is called *hatching*.

Speed and power

The two dials that decide everything. Speed is how fast the head moves (millimetres per second, or per minute on some machines); power is how hard the beam burns. Thicker material wants slower and/or more power.

Passes

How many times the beam repeats the same path. Two passes at moderate power cut cleaner than one at maximum, especially on plywood and acrylic.

PPI / frequency

Pulses Per Inch — how many laser pulses per inch of travel. Older controllers expose pulse rate; modern ones expose frequency in kHz. It changes how a cut edge looks more than whether it cuts.

Kerf

The width of material the beam destroys. A 0.15 mm kerf means a part designed 20 mm wide comes out 19.85 mm wide unless you compensate. MeerK40t can offset paths to compensate.

DPI / raster step

Dots per inch — how many scan lines per inch in a raster engrave. More DPI is finer and much slower. MeerK40t usually works in *raster step* internally (the distance between lines) and shows DPI beside it.

Origin / home / jog

Home is a fixed corner the machine can find itself (usually by limit switches). *Origin* is the zero point the job is measured from. *Jogging* is moving the head by hand without cutting.

Focus

Focus — the height of the lens above the material — is not controlled by MeerK40t on most hobby machines: you set it with a gauge or a motorised bed. Everything in this program assumes the focus is correct. Blurry cuts with otherwise sensible settings almost always mean focus, not software.

1.4 Which machines are supported

MeerK40t drives machines through dedicated *drivers*. Find your controller in the table below; if you do not know which one you have, Chapter 4 helps you identify it and the manufacturer’s model name is usually enough.

Driver	Typical machines	Notes
Lihuiyu	K40 and other M2-Nano / M3-Nano boards	USB, no controller-side settings, raster and vector both supported
GRBL	Diode engravers: Ortur, Atomstack, Longer, Creality, Sculptfun, generic G-code machines	The most predictable driver; serial or USB, also network (Chapter 17)
Ruida	Black-and-yellow CO ₂ machines with an RDC6442/RDC6445 panel	Network-first, and also able to <i>emulate</i> Ruida for other software
Moshi	Older Moshiboard machines	USB
Newly	NewlyDraw System 8.1 machines	USB, PWM frequency exposed
Balor (galvo)	Fibre and UV galvo markers with Ezcad2-compatible JCZ controllers	Field-based marking, not gantry motion

Driver	Use status	Development	Known issues
Lihuiyu	usable	under revision	reported raster Y-registration issues
GRBL	good	stable	none known
Moshi	good	stable	none known
Newly	good	stable	none known
Ruida	good	under revision	fit for testing rather than production
Balor	good	stable	none known

That second table is worth reading carefully if your work matters: this manual describes features that are complete and supported, but a driver marked “under revision” is exactly what it says. Test any new machine profile on scrap before running something expensive.

1.5 What MeerK40t is not

Setting expectations saves frustration:

- It is **not a CAD or illustration program**. It can draw shapes, edit paths and lay out text, and many people do whole projects in it — but serious artwork is usually made in Inkscape, Illustrator, Affinity Designer or Fusion 360 and imported (Chapter 9).
- It is **not a photo editor**. It rasterises photographs beautifully, but cropping, retouching and levels belong in an image editor first.
- It does **not replace your machine’s manual**. Homing behaviour, limit switches, interlocks, water chillers and lens maintenance are machine matters.

1.6 What you need before Chapter 5

- A supported laser, powered, with its interlocks intact.
- The cable your machine uses (usually USB; Ruida machines are usually network).
- MeerK40t installed and starting (Chapter 2).
- A few offcuts of the material you intend to use. Your first job should burn scrap, not your final workpiece.
- A way to secure the workpiece: tape, magnets, or the machine's own clamps. Nothing moves a laser cut like a workpiece that shifted mid-job.
- Air assist, if your machine has it, and a focused lens.

Try it 1.1 — Inventory your machine before you start

Write down, in one place, the answers to these questions. You will need every one of them when you configure the device in Chapter 4:

1. What is the working area of the bed, in millimetres? (Measure it; the specification is often optimistic.)
2. Where is the machine's origin — front-left, front-right, or centre?
3. What controller is inside it — Lihuiyu, GRBL, Ruida, Moshi, Newly, or a galvo head?
4. What is the maximum power of the tube or diode, in watts?
5. Does it home itself on power-up, and in which corner?
6. Does it have air assist, a rotary attachment, or a camera?

Chapter 2

Installing and launching MeerK40t

This chapter gets the program onto your computer and onto the screen. Nothing in it touches the laser.

2.1 Three ways to install

A. Download a ready-made build (recommended for most people)

Go to the MeerK40t releases page on GitHub and download the file for your platform. The builds are self-contained: there is nothing else to install, no Python, no libraries. On Windows it is an installer or a portable executable; on macOS it is an application bundle; on Linux there are AppImage-style and archive builds for x86-64 and for the Raspberry Pi.

Stable release or weekly build?

The project publishes two kinds of build. **Releases** (for example 0.9.9000) are the versions to use when you want the machine to work. **Weekly builds** are automated snapshots of the development branch: they contain new features and new bugs, and they are explicitly not for production work. If you want the newest feature, expect to test it on scrap.

B. Install with pip

If you already have Python 3.6 or newer, one command installs the program and the console entry point:

```
pip install meerk40t[all]
meerk40t
```

The `[all]` extra pulls in the GUI (wxPython), image support (Pillow), camera and tracing support (OpenCV) and DXF import (ezdxf). Without it you get the headless core: USB, serial and numpy. The individual extras are `gui`, `cam`, `camhead` and `dxf`, so `pip install meerk40t[gui,dxf]` is a reasonable minimal GUI install.

C. Run from source (for developers and the curious)

```
git clone https://github.com/meerk40t/meerk40t.git
cd meerk40t
python -m pip install -r requirements.txt
python meerk40t.py
```

On Linux, three optional system packages add speed and polish. They are worth installing if your distribution packages them:

```
# faster bitmap vectorisation (pypotrace)
sudo apt install libagg-dev libpotrace-dev && pip install pypotrace
# faster and better path offsetting, used by kerf compensation (pyclipr)
sudo apt install cmake libeigen3-dev && pip install pyclipr
# only needed if you must build wxPython yourself
sudo apt install libgtk-3-dev libpython3-dev
```

The repository also ships helper scripts: `install-run.sh` (Linux/macOS) and `install_run.bat` (Windows) install the requirements, ask about the optional packages, and start the program.

What you should see afterwards

Whichever route you chose, you now have either an application to launch from your desktop, or a `meerk40t` command in your shell. Running `meerk40t --version` prints the name and version, for example `MeerK40t 0.9.9040`. The version string has a suffix that tells you how the program was obtained: `pkg` for a packaged install, `git` followed by the branch name when running from a git checkout, and `src` when running from an unpacked source tree.

2.2 The first launch

The very first time you start MeerK40t it creates a settings file, shows its main window, and — because it does not yet know what hardware you own — asks you what kind of machine you have.

Notice the shape of the list. The first level is a family — *K-Series CO₂-Laser*, *Ortur Diode-Laser*, *Longer Diode-Laser*, *Newly CO₂-Laser*, *Generic UV-Laser*, *Generic CO₂-Laser*, *Generic Fibre-Laser* (with JCZ variants), *Generic Diode-Laser*, and a plain *Generic* entry. Picking one creates a device that is pre-filled with sensible defaults for that family; the model name only decides defaults, and every default can be changed afterwards (Chapter 4).

What got written to disk

MeerK40t keeps everything it remembers — device list, window layout, operation defaults, recent files, your theme — in a single text configuration file. It is written when the program shuts down cleanly, and the previous five versions are kept as backups beside it.

Platform	Location
Linux	<code>~/.config/MeerK40t/MeerK40t.cfg</code>
macOS	<code>~/Library/Application Support/MeerK40t/MeerK40t.cfg</code>
Windows	<code>%LOCALAPPDATA%\textbackslash MeerK40t\textbackslash MeerK40t.cfg</code>

Backups are `MeerK40t.bak`, `MeerK40t.ba1` ... `MeerK40t.ba5`, newest first. When something about the program has gone strange — a pane is missing, a device will not connect — the cure is more often than not to shut the program down, rename this file aside, and start again. You will then be asked what device you own, exactly as on a first run.

2.3 Starting the program in different ways

The graphical interface is the default. The same binary is also a scriptable console program, which is what makes unattended and headless use possible. These are the command line options:

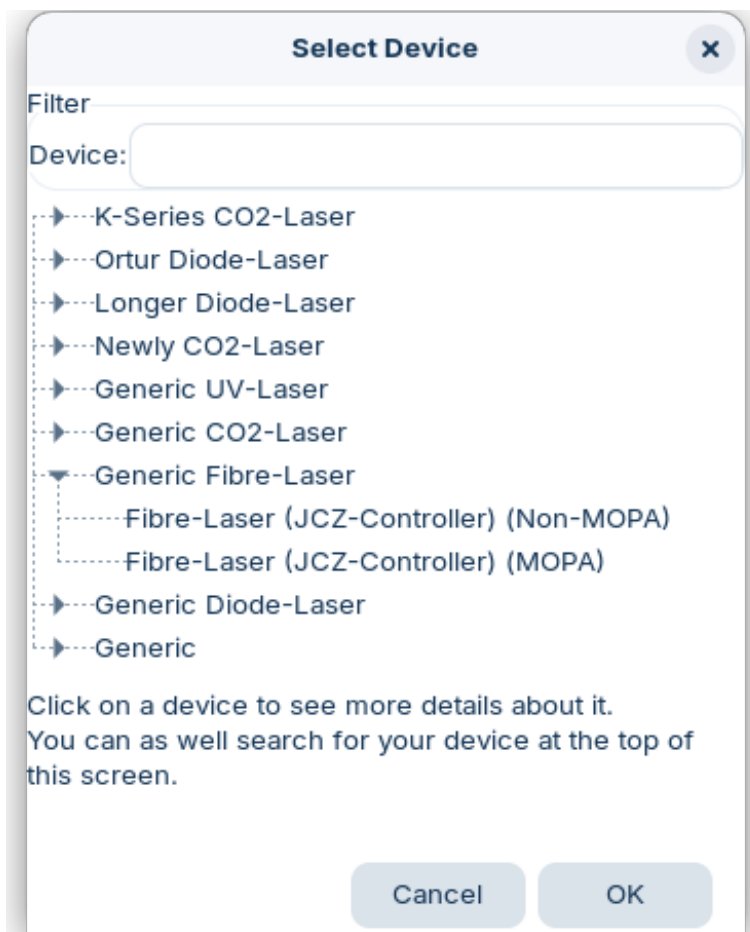


Figure 2.1: The device chooser on a first run. Type a few letters into the **Device:** filter box to narrow the tree; the list groups generic profiles (CO₂, diode, fibre, UV) underneath the drivers that serve them. Click a device to read its description, then **OK**.

Option	Effect
input	An input file to open at startup (a positional argument: put the file name last, or first, but not after a flag that takes a value).
-o, --output FILE	An output file name.
-V, --version	Print MeerK40t <version> and exit.
-z, --no-gui	Run without the graphical interface.
-Z, --gui-suppress	Suppress the GUI completely — never even try to create a window. Useful on a headless server or over a serial-only link.
-w, --simpleui	Start in the simplified interface (Chapter 3).
-c, --console	Start as a console session: the command prompt is where you work.
-e, --execute "CMD"	Execute a console command, then continue (or exit, in console mode). May be given more than once.
-b, --batch FILE	Run a file of console commands, one per line.
-a, --auto	Start running the laser as soon as the program is up.
-q, --quit	Quit when the spooler has finished — the batch combination.
-d, --daemon	Keep MeerK40t running in the background after the job.
-s, --set key=value	Set a device or kernel setting at startup, e.g. <code>-s bed_width=400mm</code> . May be repeated.
-p, --no-plugins	Do not load external plugins (a plugin troubleshooting switch).
-X, --nuke-settings	Ignore the configuration file at startup — effectively a factory-fresh run, without deleting anything.
-L, --language xx	Force the interface language: <code>en</code> , <code>de</code> , <code>es</code> , <code>fr</code> , <code>hu</code> , <code>it</code> , <code>ja</code> , <code>nl</code> , <code>pt_BR</code> , <code>pt_PT</code> , <code>zh</code> , <code>ru</code> .
-u, --lock-device-config	Lock the device configuration so it cannot be edited (a kiosk / shared-machine switch).
-U, --lock-general-config	Lock the general configuration.
-m, --minimized	Start with the window minimised.
-M, --maximized	Start with the window maximised.
-A, --disable-ansi	Disable ANSI colour output in the console.
-v, --verbose	Print verbose debugging output.
-f, --profiler FILE	Record a Python profile of the session into a file (for bug reports about speed).
-P, --profile N	Accepted but currently unused: the settings file name is fixed.

A few combinations are worth knowing by heart:

```
# ordinary use
meerk40t

# open a file straight away
meerk40t ~/designs/tag.svg

# headless: load a file, build the job, run it, quit when done
meerk40t -Z -a -q -e "load /home/me/tag.svg" -e "plan copy-selected preprocess validate blob
preopt optimize spool"

# a fresh, factory settings run without touching your real configuration
meerk40t -X
```

```
# start the simplified interface
meerk40t -w
```

Wayland

On Linux, MeerK40t forces the X11 backend (`GDK\_BACKEND=x11`) because Wayland does not let an application reposition its own top-level windows, which the dockable pane system relies on. If you start the program from a terminal you may therefore see it appear through XWayland. This is intentional.

2.4 Keeping it up to date

Packaged installs and source checkouts are updated differently: for a source checkout, `git pull` and restart; for a packaged build, download the new release. The program also contains an updater component that can check the project's release feed and tell you when a newer version exists (Chapter 19).

Try it 2.1 — Get to the main window, with something recognisable on screen

1. Launch MeerK40t.
2. If the device chooser appears, pick *Generic CO₂-Laser* or the family that matches your machine and press **OK**. You can change this later; nothing is permanent.
3. Look at the title bar. It reads something like `MeerK40t v0.9.9040 pkg - lihuyu-device - <file>`: application version, active device, and the currently open file.
4. Find the **Laser-Control** pane and confirm it is not complaining about a missing connection. The exact message depends on your driver (Chapter 4).
5. In the **Console** pane, type `help` and press **Enter**. You should get a list of commands. This pane is your instrument panel for the rest of the manual.
6. Close the program. Reopen it. The window should come back exactly as you left it. That is your settings file working.

If step 6 fails — if the window comes back scrambled, or a pane is missing — you have found the one failure mode worth knowing early. Quit the program, move `MeerK40t.cfg` aside, and start again. You lose your layout, not your work.

Chapter 3

A tour of the interface

Start MeerK40t and you are looking at a frame full of dockable panels around one large canvas. Nothing about the arrangement is fixed: every panel can be moved, floated, hidden and brought back, and the arrangement you leave behind is the one you get next time. This chapter is the floor plan.

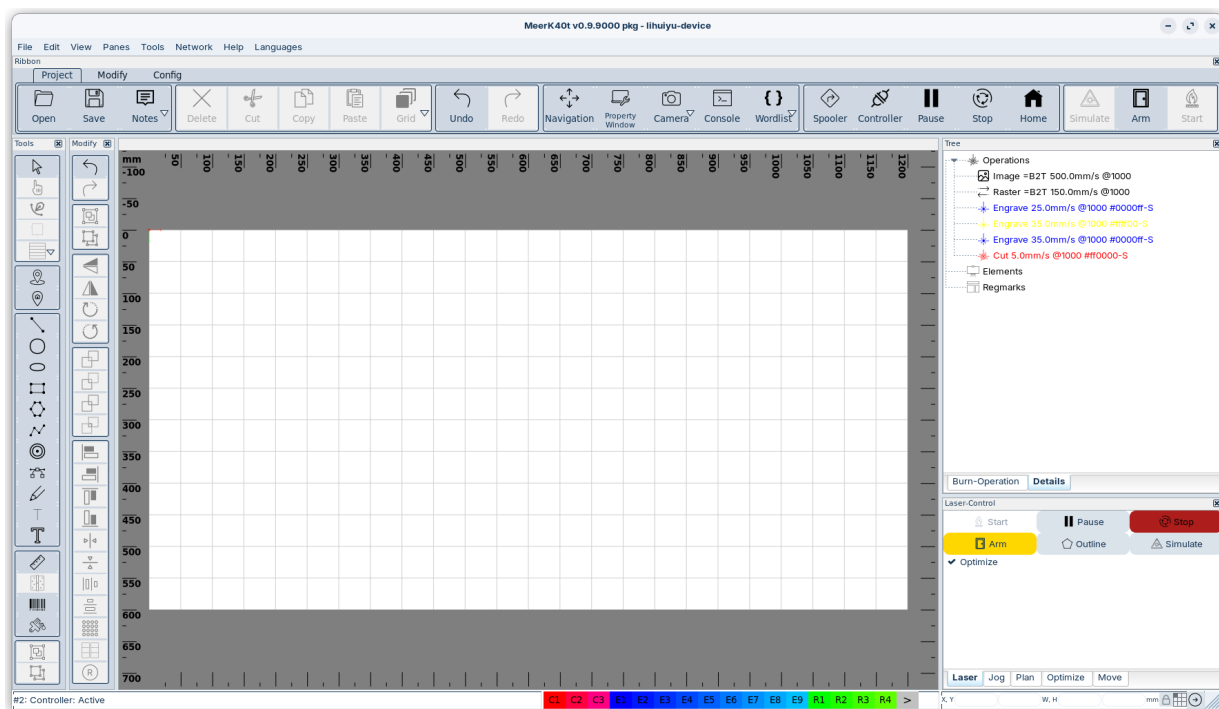


Figure 3.1: A fresh MeerK40t window with the ribbon interface. Top: the ribbon (**Project** page) with the **Tools** and **Modify** icon bars docked at the left. Centre: the scene, with rulers, the bed outline, the work origin and a laser-path preview. Right: the **Tree** pane above the **Laser-Control** pane. Bottom: the status bar with its operation squares, coordinate readouts and snap toggles.

3.1 The five regions

The menu bar

Nine menu titles at most — File, Edit, View, Panes, Tools, Network, Help and (when translations are installed) Languages. Each one is covered in §3.6.

The ribbon

Tool buttons arranged on pages. Three ribbon bars exist: the main one across the top, and two compact icon bars (**Tools** and **Modify**) usually docked down the left edge.

The scene

The canvas. This is your artwork, drawn to scale in millimetres, with the machine’s bed as a white rectangle and the machine origin marked inside it.

The tree

The document outline: *Operations*, *Elements* and *Remarks*.

The status bar

Along the bottom. It holds quick-assign squares, the current selection’s position and size, and the snap toggles.

3.2 The scene

Everything you draw or import appears in the scene. Three cues tell you where you are:

- **The bed rectangle** is the physical working area of your machine, drawn from the device’s bed size setting. If your artwork is outside it, the job will not fit — better to discover that here than at the machine.
- **The origin indicator** marks the zero point the job is measured from. **View ► Scene Appearance ► Hide Origin-Indicator** turns it off.
- **The rulers** run along the top and left edges in the current unit (millimetres by default, changeable in **Edit ► Settings ► Preferences**, **General**, units panel).

The grey area outside the bed is legal to draw on but will not be cut unless you move the artwork or the job origin (Chapter 15).

Moving around

Input	Effect
Mouse wheel	Zoom in/out around the pointer
Shift +wheel, or wheel left/right	Pan horizontally
Wheel + Ctrl	Toggle the other of zoom/pan (so Ctrl +wheel pans vertically by default)
Right-drag + Alt	Drag-zoom
Middle-drag + Ctrl	Drag-zoom
Right-drag + Ctrl	Drag-pan
Left-drag on empty space	Rubber-band select: drag right to enclose, drag left to touch. Shift adds to the selection, Ctrl inverts it
Left-drag on the selection	Move the selected elements
Left click	Select the element under the pointer
Right click	Scene menu: grids, background, “Move laser head here”, zoom commands

Two preferences change the wheel behaviour: `mouse_wheel_pan` (*MouseWheel Pan*) makes the wheel pan instead of zoom, and `mouse_pan_invert` / `mouse_zoom_invert` flip the directions. They live in **Edit ► Settings ► Preferences**, **GUI**.

Tip

A three-button mouse makes layout work much nicer: **Ctrl** +right-drag pans, **Alt** +right-drag zooms, and the left button stays free for selecting and dragging. Practise for a minute before you start drawing.

3.3 The tree

The **Tree** pane is the document. It has exactly three branches, always in this order:

Branch	What lives there
<i>Operations</i>	The recipes: what to cut, engrave, raster or dot, and with which settings. Created by you; they hold references to elements, not the elements themselves.
<i>Elements</i>	The artwork: paths, rectangles, ellipses, text, images, and groups or layers holding them.
<i>Regmarks</i>	Registration marks: shapes used to align a job with something already on the material, or with a camera view (Chapter 15).

The tree pane is itself a notebook with two tabs: **Burn-Operation** and **Details**. Right-clicking anything in the tree gives a context menu specific to what you clicked — those menus are how operations get created and how elements get assigned, and they are the subject of Chapters 7 and 10. Treat the tree as the control room: most mistakes are visible here before they reach the machine.

3.4 Laser-Control and the other machine panes

Laser-Control is the pane you will press when the job is ready. It is a notebook with five tabs:

Tab	Contents
Laser	Start , Pause / Resume , Stop , Arm / Disarm , Outline , Simulate , the Optimize checkbox, the Override checkbox with power and speed sliders, and the job progress readout.
Jog	Manual movement of the head.
Plan	The current plan: its stages, status and contents; buttons Update , Export , Spool , checkbox Hold .
Optimize	Recalculation controls and the optimisation settings (Chapter 16).
Move	Position readout and move-to commands.

Other panes worth knowing on day one:

Pane	Purpose
Console	The command interface (Chapter 18). Open it with the Console button in the ribbon (or Panes ► Console) and keep it open; it reports what the program is actually doing.
Position	Numeric position and size of the selection.
Snap-Options	Snap to Grid , Snap to Element and related toggles.
Magnet-Options	Constraint helpers for dragging.
Operations	Coloured squares, one per operation, for assigning the selection with one click (Chapter 7).
Jog, Move, Drag, Pulse	Movement and positioning controls for the machine.
Spooler	The job queue and the history of completed jobs (Chapter 16).
Devices	The list of configured machines.
Notes	Free-text notes attached to the current file.
Wordlist	Text lists used for serial-number style production work.
Tasks	Background tasks and threads, for troubleshooting.

3.5 Panes: hiding, showing and resetting

Every pane appears as a check item in the **Panes** menu, sorted into submenus — *Laser*, *Navigation*, *Editing*, *Tools*, *Help* and *Debug*. Click an item to show or hide that pane. Three more commands sit at the bottom of the same menu:

- **Lock Panes** — freeze the layout so a stray drag cannot rearrange anything.
- **Reset Panes** — restore the default layout.
- *Long UI descriptions (ribbon + context menus)* — whether hovering a button or menu item shows the long explanation or just the short one.

View ► GUI Appearance ► Show/Hide UI-Panels (or **Ctrl+U**) hides every visible pane at once, leaving only the scene — convenient on a small screen. Press it again to get everything back.

Tip

The console can do all of this too, which is handy for scripting a familiar layout: `pane show console`, `pane hide tree`, `pane float jog`, `pane reset`, `pane lock`, `pane save mysetup` and `pane load mysetup`.

3.6 The menus

The menu bar is stable; the ribbon changes. Both reach the same commands. The important entries:

Menu	What you will use it for
File	New (Ctrl+N) clears operations, elements and notes. Open Project (Ctrl+O) loads a project. Import File merges artwork into the current project. Save / Save As write the project. Holding Shift while loading asks where the imported elements should be placed. A <i>Recent</i> submenu lists recent files, with Clear project on load and Clear Recent .
Edit	Undo / Redo (with a submenu of undo steps), Cut / Copy / Paste / Paste from system , Select all (Ctrl+A), Select None , Invert selection (Ctrl+I), Select all non-assigned , Delete , Properties . Further down: Device-Manager , Device-Configuration , and the <i>Settings</i> submenu with Wordlist-Editor , CSV Batch Run , Camera Calibration , Font-Manager , Key-Bindings and Preferences (Ctrl+,).
View	Zoom commands (Ctrl+- , Ctrl++ , Ctrl+B zoom to bed, Ctrl+Shift+B zoom to selection), Show Variables , and three submenus: <i>GUI Appearance</i> , <i>Scene Appearance</i> (hide grid, background, guides, remarks, laser path, reticle; invert; flip XY) and <i>Render-Options</i> (ignore shapes, text or images when drawing and when burning).
Panes	Pane visibility, locking and resetting — see §3.5.
Tools	Every separate window: Properties , Console , Preferences , About , Keymap , Wordlist , CSV Batch Run , Material Library , Navigation , Notes , Auto-Exec , Spooler , Tasks , Simulation , Execute Job , Device Manager , Camera Calibration , Font-Manager , Image Splitter , Operation Info , Place Template , Parameter-Test and the per-device windows (Controller , Configuration , Rotary , Cylinder , and so on), plus Reset Windows .
Network	Start and stop the Web, Telnet, Ruida and GRBL servers — remote control (Chapter 17).
Help	Help (F1) and Online Reference (F11) open the wiki page for whatever is under the mouse; Beginners' Help , Github , Releases , Facebook , Discord , Makers Forum , Feature request/feedback , Check for Updates , Tips && Tricks and About MeerK40t .
Languages	Switch interface language. Takes effect after a restart.

Context-sensitive help is worth knowing

F1 does not open one fixed page. MeerK40t looks at the widget under your mouse, reads its help tag, and opens the wiki page for that feature at <https://github.com/meerk40t/meerk40t/wiki/>. Hover over the **Kerf** field, press **F1** , and you get the kerf page. It is the fastest way to answer “what is this box for?” without leaving the program.

3.7 The ribbon

The ribbon is the toolbar, and it comes in three pieces:

- the main **Ribbon** across the top, with three pages — **Project** , **Modify** and **Config** ;
- two vertical icon bars on the left, **Tools** (drawing and selection tools) and **Modify** (transform and alignment tools).

Ribbon buttons from your device plugins also appear here — **Pause** , **Stop** and **Home** for GRBL, **Red Dot** for diode machines, **Galvo Light** and **Center** for galvo, **Focus Z** for Ruida, and

so on. Depending on what is configured, some pages will look busier than Figure 3.1 suggests.

Tip

Do not like the defaults? **Edit ► Settings ► Preferences**, tab **Ribbon**, lets you add, remove and reorder pages, panels and buttons; **Reset to Default** puts everything back. You can also switch pages from the console with **page project**, **page modify** or **page config**.

3.8 The status bar

Left to right, the status bar gives you: a message area (it also shows the description of whatever menu item you are hovering), then a row of *operation squares*, then the selection readout and snap toggles.

The operation squares are the fastest way to classify artwork. Each square is one operation, coloured with that operation’s colour. Click a square with elements selected and they are assigned to that operation. In Figure 3.1 you can see them labelled *C1*, *C2*, *C3*, *E1–E9* and *R1–R4*: cuts, engravings and rasters. Chapter 7 explains the colour rules in full.

3.9 Windows that are not panes

Some features open as separate windows rather than docks: **Preferences**, **Keymap**, **Wordlist**, **Material Library**, **Parameter-Test**, **Simulation**, **Execute Job**, **Device Manager**, **Camera Calibration**, **Image Splitter**, **Operation Info**, **Place Template** and **About**. They are all listed in the Tools menu, and **Reset Windows** forgets their stored positions if one ends up off-screen.

3.10 The simplified interface

If the full window feels like an aircraft cockpit, launch with **-w** (or **--simpleui**) for a stripped-down version: a window with four pages — **Project** (load a project and see whether it is ready to burn), **Laser**, **Jog+Move** and **Console**. The project page summarises the current file: how many elements, how many burn operations, and either “Ready to burn.”, “Unassigned elements!” or “Unburnable elements!”. That single line is a good habit even in the full interface: it tells you the job is complete before the machine does.

Try it 3.1 — Make the window yours

1. Open the **Console** pane (its button is in the ribbon under **Panes**, and you can bind **F12** to **window open Console** in the Keymap window if you want it) and the **Position** pane (**Panes ► Editing ► Position**).
2. Drag the **Console** pane by its caption to the bottom edge until it docks there. Notice the docking hints.
3. Press **Ctrl+U** twice: everything disappears, then comes back.
4. Right-click the **Tree** pane caption and float it, then dock it back on the right.
5. Tick **Lock Panes** in the **Panes** menu, try to drag a pane, and confirm it will not move. Untick it again.
6. In the console, type **pane reset**. The layout returns to the default.
7. Close the program and reopen it. Whatever layout you had at closing is restored.

Keyboard shortcuts and mouse behaviour are collected in Appendix A; you do not need to memorise them now. The four worth learning today are **Ctrl+U** (show and hide the panes),

Ctrl+B (zoom to bed), **Ctrl+S** (save) and **Home** (send the head home). None of them is more than a menu click away, and all of them can be changed in the Keymap window.

Chapter 4

Connecting your laser

Connecting a laser is three separate jobs, and it is worth keeping them separate in your head: tell MeerK40t *what* machine you own (a *device*), tell it *how* to reach that machine (the *interface*), and then prove that the two agree by moving the head. This chapter walks all three. It ends with a troubleshooting table you will come back to.

4.1 Which driver is my machine?

MeerK40t talks to a controller, not to a brand. Find your machine in the catalogue below — the entries are exactly what you see in the **Select Device** dialog of Figure 2.1 — and note the driver name in the right-hand column. The driver is what later chapters mean when they say “your device”.

Family	Entry in the list	Driver
K-Series CO ₂	<i>K40-CO2-Laser (m2nano-Board)</i> <i>(Green/Blue)</i>	Lihuiyu
	<i>K40-CO2-Laser (m3nano-Board)</i> <i>(Purple/Blue)</i>	Lihuiyu
	<i>K40 CO2 (GRBL-Controller)</i>	GRBL
	<i>K40 CO2 — MKS DLC32 (405×285 mm)</i>	GRBL
	<i>K50/K60-CO2-Laser (Ruida-Controller)</i>	Ruida
Diode	<i>Ortur Laser Master 2 (GRBL), Longer</i> <i>Ray5 (GRBL), Generic</i> <i>(GRBL-Controller), Diode-Laser</i> <i>(GRBL-Controller)</i>	GRBL
Other GRBL	<i>GRBL-FluidNC (FluidNC-Controller)</i>	GRBL
Moshi	<i>CO2-Laser (Moshi-Board)</i>	Moshi
Newly	<i>RayLaser/U-SET, Beijing SZTaiming,</i> <i>Gama, Artsign JSM-40U/3040U/3060U,</i> <i>Rabbit40B, Jinan Suke, Lion laser and a</i> <i>dozen more</i>	Newly
Galvo	<i>Fibre-Laser (JCZ-Controller)</i> <i>(Non-MOPA) or (MOPA), CO2</i> <i>(JCZ-Controller), UV (JCZ-Controller)</i>	Balor

If you cannot tell: a K40 with the original green or blue control board is Lihuiyu; a diode engraver that speaks G-code is GRBL; a big black-and-yellow CO₂ machine with a panel full of buttons is Ruida; a fibre or UV marker with a scan-head and no gantry is Balor. When in doubt, add a generic entry for the right family — the defaults are only a starting point.

4.2 The device list

Open **Edit ▶ Device-Manager**, or the **Devices** pane (Panels), or **Tools ▶ Devices**. You get a table with one row per configured machine:

Column	Meaning
<i>Device</i>	The label you gave it. Double-click the cell to rename in place.
<i>Driver</i>	The internal driver class, for example <code>GRBLDevice</code> or <code>LihuiyuDevice</code> . This is how you confirm which family a row belongs to.
<i>Type</i>	The material source family: <code>Co2</code> , <code>Diode</code> and so on. It decides which default operation settings are used.
<i>Status</i>	The driver's own report — <code>Idle</code> , <code>Running</code> or <code>paused</code> .
<i>Interface</i>	How the device is reached, for example <code>/dev/ttyUSB0:115200</code> , <code>192.168.1.5:23</code> , <code>usb</code> or <code>mock</code> . If this looks wrong, nothing else will work.

The active device is drawn in red. Only one device is active at a time; that is the machine MeerK40t will drive. The buttons along the bottom are **Create New Device**, **Duplicate**, **Rename**, **Remove**, **Activate** and — right-aligned, opening the settings — **Config**. Grouping the rows is the whole point of having several: a shop with a K40 and a diode machine keeps a device for each, duplicates one as a base for a second identical machine, and switches with **Activate** or by double-clicking the row. The active device cannot be removed, and **Remove** asks for confirmation because it cannot be undone.

The same thing from the console

```
device                # list the registered device entries
device add grbl-generic -l "My diode"
device activate "My diode"
device duplicate "My diode" "Spare diode"
device delete "Spare diode"
devinfo               # current logical and native position
```

These commands are what the dialog is doing behind the scenes, and they are the quickest way to script a machine setup.

4.3 Telling MeerK40t how big the bed is

Press **Config** on the active row. Every driver has a Configuration window whose first tab contains the geometry settings. Get these right before you draw anything, because the bed size defines the scene's bed rectangle and the origin — and therefore where your job will be cut.

Label	Typical default	What to do
Width , Height	235 mm (GRBL), 310×210 mm (Lihuiyu, Newly), 330×210 mm (Moshi), 24×16 in (Ruida), 110 mm lens (Balor)	Measure your machine's real travel and enter it. The specification is often optimistic.
Flip X , Flip Y	off, except GRBL Y and Balor X/Y	These make the on-screen axes match the machine. GRBL devices usually want <i>Flip Y</i> on so that Y grows into the bed rather than away from it.
Swap X and Y	off (on for Balor)	For machines whose axes are mounted rotated. Applied before the flips.
Force Declared Home	auto	Which corner is the machine's zero: <i>auto</i> , <i>top-left</i> , <i>top-right</i> , <i>bottom-left</i> , <i>bottom-right</i> or <i>center</i> .
X-Axis / Y-Axis scale	1.000	Corrections for machines that move the wrong distance — calibrate with a measured test burn, not by guesswork.
Laserspot	0.3 mm	Your focal spot size. Used by the raster planner and by “consider laserdot”.

Two settings you will see but should ignore

X-Margin and *Y-Margin* exist in several drivers' settings, with a default of 0, but they are hidden from the GUI and marked in the program's own source as not working yet. Do not build a workflow on them. Position jobs on the scene instead (Chapter 15).

4.4 Connecting each family

Lihuiyu / K40 over USB

The K40's controller board is reached through a CH341 USB-to-serial chip. In the **Interface** tab, choose the radio button **USB** (or **Networked**, see below, or **Mock** for a pretend machine). The USB settings let you pin down which board you mean when several are attached: **Restrict Multiple Lasers** offers a CH341 version, a device index, and a serial-number match. The **Write Buffer** group limits how much is queued at once (default 1500) — relevant on the oldest boards.

Two board-level settings on the first tab matter: **Board** (M2 is the classic green board, M3 the newer one) and **Maximum Laser strength**, which caps output as a percentage of the tube's rating. The default cap is deliberately low and the program warns when you raise it: setting it too high can damage a tube that is smaller than the software assumes.

GRBL

Pick the interface in the **Interface** tab with **Interface Type** :

Interface	Settings	Notes
<i>Serial</i>	Serial Port + Baud Rate (115200 default)	The normal USB cable case. The port list is built from the ports the system reports; on Linux expect <code>/dev/ttyUSB0</code> or <code>/dev/ttyACM0</code> .
<i>TCP-Network</i>	Address + Port (23)	For a networked or Wi-Fi-equipped controller; also used by the K40 MKS DLC32 profile, which defaults to <code>192.168.10.90:8080</code> .
<i>WebSocket-Network</i>	Address + Port (81)	For FluidNC-style firmware.
<i>Mock</i>	—	A fake machine that accepts everything. Excellent for practising the workflow without hardware.

Four other GRBL settings save a lot of time:

- **Require Validator** with **Welcome Validator** (`Grbl`) — refuses to send anything until the controller has introduced itself. Turn it on once your connection is reliable; it turns “nothing happens” into an explicit error.
- **Reset on connect** — sends a soft reset when the port opens. Useful when a controller wakes up confused.
- **Reset on ‘Clear Alarm’** (`extended_alarm_clear`) — also resets the controller when you clear an alarm, for machines that need it.
- **Device has endstops** — enables real homing. With endstops on a top-left-homing board, also enable **Sequential homing**, which homes Y before X; without it such boards stop short by around 50 mm.

The ribbon for a GRBL device adds **Clear Alarm**, **Home** (right-click for physical home), **Red Dot On** / **Red Dot Off** for machines with a pilot laser, and — if ESP3D upload is enabled — the ESP3D upload and control buttons.

Ruida

Ruida machines are configured as either **USB** (a serial interface field accepting `/dev/...` or `COM\emph {n}`) or **Networked**, which is UDP with an address setting. Two facts to know before you rely on it: the driver is marked in the project’s own documentation as under revision, and users of at least one tested controller report that *Flip X* and a raster *Overscan* of zero are required. Test on scrap.

MeerK40t can also *pretend* to be a Ruida controller — useful when you want to drive a K40 from software that only speaks Ruida (`ruidacontrol`, Chapter 17).

Moshi and Newly

Both use USB, both are older CO₂ machines, and both are reported as stable. Moshi boards talk through the same CH341 chip as the K40, so the USB-restriction settings look familiar. Newly adds a machine-specific concept: the controller stores the job in one of ten numbered *files* (**Output File**, default 0). File 0 executes immediately; the others can be sent and then replayed from the machine’s own panel, which is what the ribbon buttons **File 0 ... File 9**, **Send Only**, **Draw Frame**, **Move Frame** and **Replay** are for.

Balor / JCZ galvo

Galvo machines connect over USB only — there is no host and port to set. What matters in the configuration is the optics and the machine index: **Lens Size** (default 110 mm) describes your

field, and enabling a correction file (**Enable** + **File** , a `.cor` file from the lens supplier) makes the marked geometry match the design. **Machine index to select** picks the controller when more than one is attached; **Check serial no** lets you match a specific unit. The **Timings** tab holds the laser on/off and jump delays — change these only with a documented reason, since they control whether corners are burned through or left open.

4.5 USB permissions on Linux

If the machine is not found on Linux, or only works when you run the program as root, the cause is almost always permission on the USB device. Create `/etc/udev/rules.d/99-meerk40t.rules` with the vendor rules for the chips MeerK40t uses:

```
# CH341 USB-to-serial chips (used by Lihuiyu laser controllers)
SUBSYSTEM=="usb", ATTR{idVendor}=="1a86", ATTR{idProduct}=="*", MODE="0666"
# FTDI USB-to-serial chips (used by Ruida laser controllers)
SUBSYSTEM=="usb", ATTR{idVendor}=="0403", ATTR{idProduct}=="*", MODE="0666"
# BJJCZ galvo controllers
SUBSYSTEM=="usb", ATTR{idVendor}=="9588", ATTR{idProduct}=="*", MODE="0666"
# GSI Group galvo controllers
SUBSYSTEM=="usb", ATTR{idVendor}=="28e9", ATTR{idProduct}=="*", MODE="0666"
# Philips chips (used by Newly controllers)
SUBSYSTEM=="usb", ATTR{idVendor}=="0471", ATTR{idProduct}=="*", MODE="0666"
```

then apply them and add yourself to the serial group:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
sudo usermod -a -G dialout $USER
```

Log out and back in for the group change. Do not run MeerK40t with `sudo`: it is not recommended, and it produces configuration files your normal user cannot then read.

4.6 Proving the connection works

Before you cut anything, answer one question: can MeerK40t talk to the machine? Three windows tell you, all of them under **Tools**:

- **Controller** (per device) — the driver's own window, with connection state, packet counters, a **Reset statistics** button, **Clear Status-Log** and the raw traffic. If the connection is alive, the counters climb.
- **Buffer** (**BufferView**) — shows the command flow queued to the device. Useful when a job appears to hang: is the queue empty because the job finished, or because it never started?
- **UsbConnect** — tails the raw USB conversation and lets you type into the pipe by hand. This is the deepest view available.

In the console, `devinfo` prints the current logical and native position, and `device` lists your devices. If the interface column in the device list shows a plausible port and the controller window shows traffic, the link is up.

4.7 Moving the head safely

The **Jog** pane is where you learn to trust the machine. It has nine direction buttons plus **Home** , **Unlock rail** , **Confine** and **Lock rail** .

At the machine 4.1 — Do your first movements with the beam off

First moves are the ones where the head tries to leave the machine. Keep the enclosure closed, keep your hand near the emergency stop, and do not have material — or your fingers — under the head. If a direction button sends the head the wrong way, do not chase it with software: stop, and fix *Flip X*, *Flip Y* or *Swap X and Y* in the device configuration.

Three distinctions worth learning on day one:

Home versus physical home

Left-clicking **Home** sends the head to the *defined* home — the origin the driver and your settings describe. Right-clicking it runs the *physical* home, which seeks the controller's endstops. On GRBL the physical home sends `\$H` (or `\$HY` then `\$HX` when sequential homing is on). Machines without endstops simply move to 0,0.

Locked versus unlocked rail

Unlock rail lets the head be moved by hand; **Lock rail** re-engages it. On some drivers the rail locks itself automatically after a job (**autolock**).

Confined movement

Confine limits jogging to the bed size. Turning confinement off lets you drive the head outside the declared area — which on GRBL means soft-limit alarms, and on anything else means crashing into the frame. The program warns you when you switch it off.

Jog distance is set in the **Distances** pane (**Jog Distance:**); the default is 10 mm per press, and holding a button repeats the move. **Move Laser to:** moves to typed X/Y coordinates, and the nine buttons beside it are saved positions — left-click to go there, right-click to store the current position. They persist between sessions.

4.8 When it does not connect

Symptom	Check, in this order
No device entry matches your machine	Add a generic entry for the family (Chapter 4); the defaults are editable.
Interface column says UNCONFIGURED or is empty	Choose the serial port (or type the network address) in the Interface tab; replug the cable and reopen the list.
Port exists but nothing moves	Baud rate (115200 on almost all GRBL boards), <i>require validator</i> and the welcome string, and <i>reset on connect</i> .
Works only with <code>sudo</code> (Linux)	Install the udev rules above and add yourself to <code>dialout</code> .
GRBL answers with an alarm	Clear Alarm ; if the alarm returns immediately, the machine is hitting soft limits — check bed size, flips and homing corner.
Head moves the wrong way	<i>Flip X/Flip Y/Swap XY</i> . Change one at a time and re-test.
Head moves the wrong distance	Axis scale factors; calibrate with a measured move rather than arithmetic.
Job stops halfway with the queue empty	Check the controller window for an alarm or an error, and the console for the last message received.
Two K40s, only one responds	<i>Restrict Multiple Lasers</i> : set the device index or serial number for each.

Try it 4.1 — Commission a machine in ten minutes

Work through this once, on any machine, and keep the results in your notes.

1. Add or activate the correct device. Confirm the *Driver* and *Type* columns look right.
2. Enter the real bed size, measured with a tape, and set *Force Declared Home* to the corner your machine actually uses.
3. Open the **Controller** window and confirm traffic. Note the interface string.
4. In the **Jog** pane, set the jog distance to 10 mm and jog in all four directions, watching the head.
5. Home the machine (left-click), then right-click to run the physical home if the machine has endstops.
6. Jog to a known point, then use **Move Laser to:** to send the head to X 0, Y 0 and confirm it lands where the machine thinks zero is.
7. Lock the rail, then unlock it and nudge the head by hand and back.
8. Save the console transcript or take a screenshot of the device configuration. When something misbehaves in three months, this is your reference.

Full per-driver settings tables, including every default mentioned in this chapter, are collected in Appendix C.

Part II

Your first projects

Chapter 5

Your first cut

This is the chapter to do with your hands on the machine. We will cut and engrave a small plywood tag: one cut operation for the outline and the hanging hole, one engrave operation for a line of lettering. Nothing here is specific to a particular machine except the numbers, and the numbers are the part you will adjust.

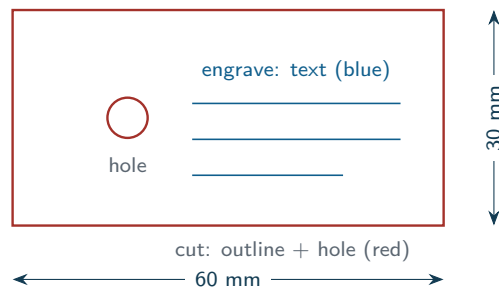


Figure 5.1: The finished job: a 60×30 mm tag. Two operations share one drawing — a red **Cut** operation for the outline and the 4 mm hole, and a blue **Engrave** operation for the text. Colour is how you keep track of which is which.

5.1 Before you press Start

At the machine 5.1 — The five-minute setup

1. **Focus.** Set the lens height for your material with the machine's own gauge or jig. Software cannot fix focus.
2. **Air assist on** (if fitted). It clears smoke from the beam path and reduces flare.
3. **Secure the material.** Tape it, magnet it, or clamp it. A workpiece that shifts mid-job produces scrap and possibly a fire.
4. **Focus test on scrap.** If you have never cut this material, treat your first job as the test. Keep a fire extinguisher within reach and never leave the machine running unattended.
5. **Know your escape.** Locate the pause and stop controls — on the machine, and in the **Laser-Control** pane (**Pause** , **Stop**).

5.2 Step 1 — Start a project and check the bed

Walkthrough 5.1 — A new project

1. Start MeerK40t and make sure your machine is the active device (the red row in the **Devices** pane, Chapter 4).
2. File ► New (**Ctrl+N**) clears operations, elements and notes. You now have an empty document.
3. Press **Ctrl+B** (zoom to bed). The white rectangle you see is your machine's working area, drawn from the device's *Width* and *Height* settings. If the proportions look wrong, stop and fix the device configuration now — everything downstream depends on it.

5.3 Step 2 — Draw the shape

You can draw the tag here or import it; both are normal. Drawing it makes the later steps easier to see.

Walkthrough 5.2 — Draw a rectangle, a hole and some text

1. Click **Rectangle** in the **Tools** icon bar. Click once near the top-left of the bed for the first corner, and again for the opposite corner. A rectangle appears and is selected.
2. Open the **Position** pane (Panels ► Editing ► Position). Set **Width** to 60mm and **Height** to 30mm, pressing **Enter** in each field. Set **X** and **Y** to place it, for example 20mm and 20mm.
3. Click **Circle** in the **Tools** bar and click near the left end of the rectangle to place a circle. In the **Position** pane set both **Width** and **Height** to 4mm (a circle with equal width and height), then set **X** to 26mm and **Y** to 33mm so it sits 8 mm from the end and centred vertically.
4. Click **Text** in the **Tools** bar and click to the right of the hole. With the new text element selected, open Tools ► Properties and type your lettering, for example the word WORKSHOP. Set the height so it fits in the remaining width.

If you prefer to type rather than click, the console does the same work. `rect 20mm 20mm 60mm 30mm` creates the rectangle, `circle 26mm 33mm 4mm` the hole, and `text WORKSHOP` a text element. Console-created elements are selected automatically, so `scale`, `position` and friends apply to them immediately — see Chapter 18.

Get the size right before the burn, not after

Importing from Inkscape or Illustrator usually brings a scale with it: the drawing arrives at whatever size the file says, which is rarely the size you want. Press **Ctrl+B**, look at the bed, and use the **Position** pane to set the real width. The dimension lines drawn on the scene update as you type.

5.4 Step 3 — Create the two operations

Operations are recipes, and this job needs two: one to cut, one to engrave.

Walkthrough 5.3 — Cut and engrave

1. Right-click the *Operations* branch in the **Tree** and choose **Append operation ► Append Cut**. A red operation appears. Rename it if you like — a name such as *Cut 3mm ply* is worth more than *Cut* when you have six operations.
2. Right-click *Operations* again and choose **Append operation ► Append Engrave**. A blue operation appears below it.
3. The order of the list is the burning order. Cutting last is usually what you want: engrave the text first, while the part is still held by the surrounding material. Drag one operation above the other if you need to reorder; dragging in the tree changes burn order.

5.5 Step 4 — Assign the artwork**Walkthrough 5.4 — Put each shape in the right operation**

1. **Ctrl+A** selects everything on the scene.
2. Open the **Operations** pane (Panels ► Tools ► Operations). It shows one coloured square per operation. Click the **blue** square to assign everything to the **Engrave** operation.
3. Click the rectangle on the scene to select just it, then hold **Shift** and click the circle. Both are now selected.
4. Click the **red** square. The rectangle and the circle move to the **Cut** operation; the text stays engraved.
5. Look at the tree. Each operation now lists the elements it contains. If something is in the wrong place, drag it from one operation to the other, or right-click it and use **Assign Operation**.

Text elements belong to **Engrave** or **Raster**, not **Cut**: a cut operation traces outlines, and letters have no useful outline to trace into a hole. Everything you need to know about the five operation types is in Chapter 7.

Watch for orphans

If a shape is in the elements branch but referenced by no operation, it will not burn and MeerK40t will warn you before starting (“Some shapes are not assigned to any operation. . .”). Always take that warning seriously — usually the fix is one click on an operation square.

5.6 Step 5 — Set speed and power

This is the part of the job that only your machine can answer, but here is a sane order of operations. Select the **Cut** operation and open **Tools ► Properties**.

Setting	Typical 40 W CO ₂ on 3 mm plywood	Typical diode on 3 mm plywood	Why
Speed	5–10 mm/s	3–6 mm/s	Slower means more energy per millimetre; below a few mm/s the material chars.
Power	60–80%	80–100%	Enough to cut through in one or two passes.
Passes	1, then 2 if needed	2–4	Two moderate passes cut cleaner than one aggressive pass.
Kerf compensation	0 until you measure it	0	Compensate only when you know your kerf (Chapter 8).

Then select the **Engrave** operation and give it its own numbers: engraving is a lighter burn at a higher speed, commonly something like 25–35 mm/s at 20–40% power for a visible, shallow mark on wood. “Power” may be displayed as a 0–1000 scale or as a percentage depending on your device; the tooltip on the field tells you which.

Write the numbers down

Keep a small table on the wall next to the machine: material, thickness, speed, power, passes, and the date. Two of those lines save you an entire afternoon three months from now. Chapter 8 shows how to build the same information systematically with a test grid and the Material Library.

5.7 Step 6 — Position and check before burning

Walkthrough 5.5 — Frame the job, then estimate it

1. In the **Position** pane, make sure the whole tag lies inside the bed rectangle on the scene. Anything outside the bed will not be cut, and on some machines it will not even be attempted — or will trigger a soft-limit alarm.
2. On the **Laser-Control** pane, **Laser** tab, press **Outline**. If your machine has a pilot laser or low-power mode, the head traces the job’s bounding outline so you can see where it will land. If it does not, the outline is drawn on screen instead. Either way, this is the step that catches the job positioned 40 mm to the left of your material.
3. Press **Simulate** to open the **Simulation** window, then press **Recalculate**. You get an animated preview and time estimates: *Laser Time*, *Travel Time*, *Total Time* and distances. Watching the animation is the fastest way to see an obviously wrong order — the head leaping back and forth across the bed between two cuts that are next to each other.
4. Close the simulation window when you are satisfied.

5.8 Step 7 — Run the job

Walkthrough 5.6 — Cut the tag

1. Check the **Optimize** checkbox on the Laser tab. With it on, the job is planned through the full pipeline; with it off, the plan skips optimisation passes. Beginners should leave it on.

2. Press **Start** . Deliberately: the button is guarded so that a stray keystroke cannot begin a burn, which is why the mouse click matters.
3. Watch the progress header on the Laser tab. It reports the job state and the percentage; the detail line shows steps completed and time remaining.
4. If the material begins to flare, press **Pause** and be ready with **Stop** . **Pause** finishes the current move and holds; **Resume** continues. **Stop** aborts the job.
5. When the job ends, wait for the head to stop and the smoke to clear before opening the lid.

Behind that one click, the program ran the familiar chain of stages: copy the operations, preprocess the coordinates, validate them, convert to cutcode (*blob*), add optimisation commands, optimise, and spool. You can watch the same stages run line by line in the **Console** pane, and run them yourself in the **Plan** tab or the **Execute Job** window (Chapter 16). If you ever want the manual version, those are the places to look.

Job progress, and what “Optimize” really changed

With **Optimize** ticked, the plan is built and optimised before the spooler receives it. With it unticked, the same plan is built without the optimisation stages. The spooler then feeds the driver, and the driver feeds the machine. Each stage reports into the console channel, which is why the console is the single best diagnostic tool in the program.

5.9 Step 8 — Judge the result, then save

Lift the part out and look at the cut edge. Three outcomes:

- **It fell out cleanly.** Save the project (**Ctrl+S**) and write your settings down. You have a working recipe.
- **The outline cut, the text is faint.** The **Engrave** operation needs more power or less speed.
- **The edges are charred and the material is sticky.** Too much energy: raise the speed or lower the power. On wood, charring often means the beam dwelled too long — one faster pass beats two slow ones.
- **It did not cut through.** Add a pass, then reduce speed, then raise power — in that order, and re-test on scrap each time.

Saving is worth a word, because it is not what most people expect. MeerK40t’s project file is an **.svg** file — the same format you import — with extra attributes that preserve operations, settings and notes (**Ctrl+S** , or **File ► Save As** for a new name). It is not a proprietary container, and it is not a device job file; if you want the **.egv**, **.gcode**, **.rd** or **.lmc** that the machine itself consumes, use **Plan ► Export** on the Laser panel (Chapter 9).

Try it 5.1 — Repeat the exercise on your own material

Now do it again without reading the steps, with one change: cut a material you actually care about.

1. Draw or import a part with one cut contour and one hole.
2. Create **Cut** and **Engrave** operations and assign the shapes by colour.
3. Take a guess at the settings, write the guess down, and cut it.

4. Adjust exactly one setting and cut again.
5. Record both attempts. That page of notes is your first material profile, and Chapter 8 turns it into something repeatable.

Chapter 6

Your first engrave

Cutting is one-dimensional: the beam follows a line. Engraving removes material from an *area*, and that makes it a different craft with different dials — resolution, direction, dot patterns. This chapter covers both flavours of it:

- **raster engraving**, where MeerK40t converts vector artwork into scan lines and burns it, and
- **image engraving**, where the artwork is a picture and the pixel values decide how hard the beam burns.

Both end up as scan lines at the machine. The difference is where the decisions are made.

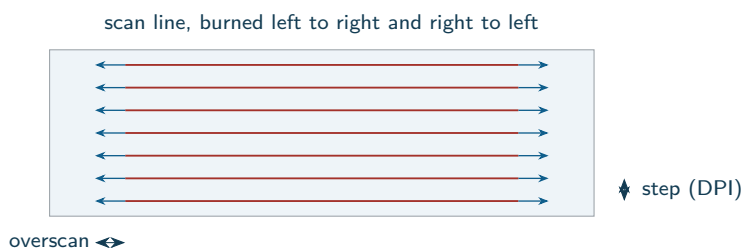


Figure 6.1: How a raster is burned. The head sweeps across the shape, reverses, and steps down one line at a time. *Overscan* is the extra run-up at each end that lets the head reach full speed before the beam starts; without it, the edges of the raster are darker than the middle. *Directional Raster* decides whether the return pass also burns. The line spacing is set by DPI.

6.1 Raster: filling vector artwork

Use this when the thing you want is a *shape* — a filled logo, a block of text, a data plate — and you want it engraved rather than cut. MeerK40t renders the assigned elements to an image internally and scans it out.

Walkthrough 6.1 — Engrave a filled logo or text block

1. Draw or import the artwork. Filled shapes work best; a thin outline engraves as a thin outline.
2. Right-click *Operations* in the tree and choose **Append operation ► Append Raster**. A black operation appears.
3. Select the artwork and assign it — **Ctrl+A** then click the black square in the **Operations** pane, or right-click the elements and use **Assign Operation**.
4. Open **Tools ► Properties** on the raster operation and set the two dials that matter first: **DPI** and **Speed**.

5. Add **Passes** if one pass is too pale; a second pass doubles the time and usually darkens more evenly than raising power does.

Choosing DPI

DPI and the machine's spot size are the same conversation. If your spot is about 0.1 mm, then your step is 0.1 mm, and there is no point scanning at more than about 250 dpi, because the next line lands where the last one burned. In practice:

DPI	Step	Typical use	Notes
100	0.254 mm	quick marking, deep text	visible lines; fast
250	0.102 mm	general CO ₂ engraving	matches a normal focus spot; a good default
500	0.051 mm	fine detail, photographs	the shipped default; twice the time of 250 for often the same practical darkness
1000	0.025 mm	diode machines with tiny spots	slow; check that the result is actually finer before keeping it

Time scales with the *area* you fill and inversely with the line step and the speed: doubling DPI doubles the number of lines, halving the area of the same shape cuts the time in half. The **Simulation** window's time estimate will tell you the number exactly; the arithmetic above tells you which way to push.

Direction, overscan and the rest

- **Direction** (top-to-bottom, bottom-to-top, left/right, crosshatch, greedy, crossover, spiral, diagonal) changes the grain of the fill. For wood, a crosshatch looks woven; a single direction looks brushed.
- **Directional Raster** on means the return pass also burns, roughly halving the time. Turn it off if your machine shows backlash at line ends — the classic symptom is a faint ghosting to one side of each line.
- **Overscan** (default 0.5 mm) is the run-up described in Figure 6.1. If the first and last few millimetres of your engraving are noticeably darker, increase it. Ruida machines are reported to need it at zero, so check the device notes in Appendix C.
- **Use grayscale instead** and **Consider laserdot** are refinement switches: the first rasterises the artwork's antialiasing, the second skips pixels the beam already covered.
- **Start Preference** decides which corner the scan begins in. On machines that are slow to reach speed, starting from the corner nearest the previous operation saves real seconds.

6.2 Image: engraving a picture

Photographs and logos supplied as .png, .jpg or another bitmap format become image elements, and image elements are burned by an **Image** operation. The image's own properties carry the dithering and tone controls, so the workflow is: import, adjust the picture, then engrave.

Importing

Walkthrough 6.2 — Bring in a photograph

1. File ► Import File, choose your picture. It lands in the elements branch, selected.
2. Check its size. Preferences (Input/Output) control what happens on import: Image DPI Scaling decides whether the picture's stored DPI is trusted for physical size (unset: treat it as 96 dpi); Center image on import places it in the middle of the bed instead of the top-left corner; Scale oversized images shrinks anything bigger than the bed; Create a file-node for imported image wraps it in a file node for tidier trees.
3. Resize deliberately. Set the real width in the Position pane rather than dragging — you usually know how wide the engraved picture should be, and DPI is derived from it.
4. Right-click Operations, Append operation ► Append Image. Assign the image to it.
5. Open Tools ► Properties on the image element (or right-click it in the tree and choose Image properties) to reach the panels that matter: DPI , Dither , Contrast , Halftone , Tone , Sharpen , Gamma , Edge Enhance , Auto Contrast , and the Modification script list.

Dither: how a photograph becomes laser pulses

A laser is either on or off. A photograph has thousands of shades. Dithering is the trick that bridges them: shades become patterns of dots, dense where the picture is dark and sparse where it is light. The pattern you choose changes the character of the result as much as its darkness.

Dither algorithm	Character
<i>Floyd-Steinberg</i> (default)	Error-diffusion classic. Smooth gradients, fine grain. The safe default for photographs.
<i>Legacy-Floyd-Steinberg</i>	The older implementation; use it to reproduce work made before 0.9.
<i>Atkinson</i>	Softer, more contrast, less error carried — the old Macintosh look. Good for portraits on wood.
<i>Jarvis-Judice-Ninke, Stucki, Burkes, Sierra3, Sierra2, Sierra-2-4a, Shiau-Fan, Shiau-Fan-2</i>	Variants of error diffusion with progressively sharper or smoother character. Try two or three on a test piece rather than over-thinking the names.
<i>Bayer, Bayer-Blue</i>	Ordered dithering: a regular grid of dots. Looks intentional and graphic — excellent for graphics, logos and gradients you want to look mechanical, poor for photographs.

Turn Dither off if the image is already pure black and white (a scanned line drawing, for instance) — there is nothing to dither, and the result is cleaner without it.

Tone controls

The panels under the image properties do what the equivalent commands in an image editor do, and they apply to the laser output rather than to a screen:

Panel	What it does
Contrast	<i>Contrast Amount</i> (−127...127) and <i>Brightness Amount</i> . The first thing to reach for on a flat-looking photo.
Halftone	Converts to dot-screen style output with <i>Sample</i> , <i>Angle</i> and <i>Oversample</i> sliders and a <i>Use black rather than white dots</i> switch. This is how you get a newspaper-print look.
Tone	A full 256-point tone curve. Precise and slow to use; the right tool when a specific band of the picture must burn darker.
Gamma	Shifts midtones with a single factor. The most useful control for matching a photo to a material's burn response.
Sharpen , Edge Enhance	Emphasise edges. Useful on wood, where subtle detail is lost in the grain.
Auto Contrast	A one-click stretch of the tonal range.
Modification	Applies a named <i>Raster-Wizard</i> script — <i>Gold</i> , <i>Stipo</i> , <i>Gravy</i> , <i>Xin</i> , <i>Newsy</i> or <i>Simple</i> — as an ordered list of image operations you can edit.
Crop	Crop by pixel, with four sliders.

If the picture needs more than this — retouching, colour removal, serious sharpening — do it in an image editor first. MeerK40t's job is to burn the picture well, not to be Photoshop.

Grayscale and depth maps

For materials that respond to dwell rather than to dots — and for diode lasers where power modulation is smooth — the image can be burned as *grayscale* instead of dithered: the pixel's brightness sets the power. The **Depthmap** checkbox goes further: it treats every grey level as a number of times that pixel will be burned, with a selectable resolution from *256 - 8bit* down to *4 - 2bit*. Depth maps are ideal for engraving height into wood, and they are slow, because repeated burns mean repeated passes over the same area. The **Grayscale** box also offers *Invert* and *Red/Green/Blue/Lightness* sliders — the last is the quickest way to lift a dark photograph before engraving.

6.3 Big images, tiles and keyholes

Two tools exist for the practical problems of large or awkward artwork, both under **Image ops** (Tools ► Editing ► Image Splitting):

- **Create split images** (**Render + Split**) renders the selection to an image at a chosen **DPI** and cuts it into an *X-Axis* × *Y-Axis* grid (up to 25 by 25). Each tile becomes its own element, which is what you want when a pattern is wider than the bed: engrave the tiles in separate runs, moving the material or using the machine's own repeat feature.
- **Keyhole operation** (**Keyhole operation**) renders the chosen object as a mask at a given **DPI**, with options to *Invert* the mask and to *Trace* the keyhole outline. This is how you engrave one shape *through* another — a picture inside a frame, or a logo that must avoid a pre-existing hole.

The same two operations are available in the console as `render_split <cols> <rows> <dpi>` and `render_keyhole <dpi>`, which is the faster route once you know your numbers.

At the machine 6.1 — Engraving is where fires start

Raster and image jobs concentrate energy in one small area for minutes at a time, on the same spot. Three habits prevent trouble: keep air assist on, keep the extraction running,

and do not walk away. If a job smells like a bonfire rather than like wood, stop it and lower the power — the smell of a raster job that is too aggressive is unmistakable once you have met it.

Try it 6.1 — Three-pass test on the same picture

The fastest way to learn the tone controls is to burn the same small photograph three times, changing one thing each time.

1. Import a photograph roughly 40 mm wide; convert it to grayscale and raise its contrast in an image editor first.
2. Duplicate it twice (**Edit ▶ Copy**, **Edit ▶ Paste**) and line the three copies up side by side with the **Alignment** window (**Align** tab, *Relative to: Selection*, *Treatment: As Group*).
3. Give all three copies one **Image** operation at 250 dpi, then set different dither algorithms on each element — *Floyd-Steinberg*, *Atkinson* and *Bayer*.
4. Engrave once and compare. Repeat with *DPI* 250 versus 500 on the best-looking setting.
5. Keep the sample in your material notebook and write the settings on the back. The next photo job takes one minute instead of thirty.

Chapter 7

Operations in depth

An *operation* is a recipe: it says *how* to burn the artwork attached to it. Five burn operation types exist, plus three effects that modify them and a family of utility operations that move or pause the machine. This chapter is the reference-tutorial for all of them: what they do, which settings matter, and how to attach artwork to them.

7.1 The five burn operations

Type	Colour	Accepts	Ignores	Use it for
Cut	red	paths, rectangles, ellipses, lines, polylines (and effects)	images	cutting through: high power, slow speed, a few passes
Engrave	blue	the same vector shapes	images	scoring, marking, thin vector engraving; dense hatching
Raster	black	vectors <i>and</i> text, converted to a raster image	images	filling an area quickly, engraving shapes and lettering
Image	(none)	image elements only	everything else	engraving a raster image, with the image's own dither and contrast settings
Dots	(none)	points only	everything else	drilling, perforating, dot marking: the head dwells in place instead of moving

The “Ignores” column matters more than it looks. Assigning a photograph to a **Cut** operation gets you nothing, and by default a **Dots** or **Image** operation *stops* classification, so elements landing there are not also picked up by later operations. When in doubt, watch the operation's icon: unusable children are greyed out or flagged.

Raster versus Image — the difference that confuses everyone

Raster takes *vector* artwork and turns it into a raster at job time, using the operation's own DPI and direction settings. It is how you fill an engraved logo or a rectangle of text. **Image** takes an *existing raster image* and burns it, using the dither, contrast and halftone settings stored on the image element itself. If the thing on your scene is a `.png` or `.jpg`, you want **Image**; if it is a path or a text element, you want **Raster**.

7.2 Creating an operation

Right-click the *Operations* branch in the tree and choose **Append operation**, then one of: **Append Image** , **Append Raster** , **Append Engrave** , **Append Cut** , **Append new Hatch** (which creates an Engrave operation carrying a Hatch effect) or **Append Dots** .

Notice that a new operation arrives with defaults taken from your device — the device can define what “cut” or “engrave” means for that machine, so a GRBL diode machine and a CO₂ cutter do not start from the same numbers. Typical starting points are around 5 mm/s for a cut, 25–35 mm/s for an engrave and 150–500 mm/s for a raster. These are starting points, not answers: Chapter 8 is about finding your own.

Under the same right-click menu, **Append special operation(s)** adds the non-burning steps. These run in the order they appear in the operations list, which lets you build a job with a preamble (home, coolant on, wait for you to press the machine’s button) and a postscript (coolant off, return home).

- Movement and time: **Append Home** , **Append Physical Home** , **Append Return to Origin** , **Append Goto Location** .
- Operator interaction: **Append Beep** , **Append Interrupt** , **Append Wait** , **Append Output** , **Append Input** .
- Ancillaries: **Append Coolant On** , **Append Coolant Off** .
- Combinations that do the above in one step, such as **Append Home/Beep/Interrupt** and **Append Origin/Beep/Interrupt** .

7.3 Attaching artwork to an operation

There are four ways, and you will settle into one of them within an hour.

1. **Drag and drop.** Select elements in the scene, drag them onto the operation in the tree, and drop. The operation gains *references* to those elements.
2. **Right-click assign.** Right-click the selected elements in the scene, choose **Assign Operation**, and pick the operation by name. The same submenu offers **Remove all assignments from operations** , **Exclusive assignment** , **Inherit stroke and classify similar** and **Inherit fill and classify similar** .
3. **Operation squares.** Open the **Operations** pane (Panels ► Tools ► Operations). It shows up to 24 coloured squares, one per operation. Left-click assigns using the element’s *stroke* colour as the classification colour; right-click uses the *fill* colour instead. Options in the pane let you choose what a click does (*Leave* / -> *OP* / -> *Elem*) and whether assignments are *Similar* (colour-matched) or *Exclusive*.
4. **The operation list panel.** The compact operations panel shows one row per operation with an *A* (active) toggle, an *S* (visible) toggle, a coolant indicator, and editable *Power* and *Speed* columns — the fastest way to tune a job by eye. Its *Classification* section chooses whether colours are left alone, the operation inherits the element’s colour, or the element inherits the operation’s colour.

Elements that are not assigned anywhere are invisible to the job

An operation only burns what it references. A shape sitting in the elements branch that is referenced by no operation will simply not be cut — and MeerK40t will warn you: “Some shapes are not assigned to any operation ... Queue anyway?”. Learn to check

for it: **Edit ► Select all non-assigned** highlights the orphans. The **Operation Info** window (**Tools ► Operation Info**) lists every operation, its item count and its runtime, and shows an *Unassigned* row marked with ! for elements that belong to nothing.

7.4 Colour classification

Each operation has a colour, and MeerK40t can use colour to decide where artwork belongs. This is how an imported SVG with a black outline and a red fill becomes “cut the red things, engrave the black things” without you selecting anything.

- A **Cut** operation is red by default. Elements whose stroke or fill matches that red are classified into it.
- An **Engrave** operation is blue by default.
- A **Raster** operation is black by default.

Classification is configured in **Edit ► Settings ► Preferences**, tab **Classification**, which offers two presets: **Automatic** and **Manual**. Automatic classification on load is what people usually want for production work with colour-keyed SVG files; turn it off when you want to place everything by hand.

Tip

If colours confuse you, make them honest: in the element’s own properties set its stroke to the operation colour you intend to use. Then the scene itself shows the plan — red lines will be cut, blue lines engraved — and mistakes are visible at a glance.

7.5 The settings of a burn operation

Select an operation in the tree and its properties appear in the **Properties** window (**Tools ► Properties**, or double-click). The settings below are the ones that actually change results.

Label	Default	Meaning
Speed	10 mm/s (cut), 35 (engrave), 150 (raster/image), 35 (dots)	Head travel speed while burning. Shown as mm/min instead when the device uses that convention.
Power	1000	Power on a 0–1000 scale, or as a percentage when the device displays percentages.
Frequency	20 kHz	Pulse frequency. Only shown for machines that expose it.
Passes	off, 1 pass	Repeat the whole operation. Tick <i>custom</i> to set a count.
Kerf compensation	0 mm	Offset every path by half the kerf so finished parts come out the intended size (Chapter 8).
Enable	on	Include this operation when the job is built.
Stop	off (on for Image/Dots)	Stop classification here: later operations do not also capture these elements.
Coolant	0 = no change	0 no change, 1 turn on, 2 turn off, for machines with coolant or air-assist control.
Color	red / blue / black	The classification colour.
Loops	1	Loop the operation branch (used with effects).

The raster and image operations add their own settings:

Label	Default	Meaning
DPI (with <i>override</i>)	500 dpi	Scan-line density. Higher = finer and much slower.
Overscan	0.5 mm	Extra travel at each end of a scan line so the head is already at speed when the beam starts.
Direction	Bottom-to-top	Scan direction: top-to-bottom, bottom-to-top, right-to-left, left-to-right, crosshatch, greedy horizontal/vertical, crossover, spiral or diagonal.
Directional Raster	Bidirectional	Burn on the way back as well, doubling throughput; uncheck for a directional look (or when your machine cannot handle it accurately).
Use grayscale instead	on	Rasterise a black-and-white image as grayscale — the raster op's answer to "Override black/white image".
Consider laserdot	off	Skip pixels already covered by the laser's spot; saves time at low DPI.
Start Preference	both on	Which corner of the raster the head starts from.
Dwell Time (ms)	50 ms	(Dots) how long the head rests on each point.

DPI and raster step are two views of the same number: the step is the distance between scan lines, and DPI is how many of them fit in an inch. At 500 dpi the step is 0.0508 mm — about twice the spot size of a typical 40 W CO₂ machine. Pushing DPI far beyond the spot size buys darkness, not detail, and costs time linearly.

7.6 Effects: hatch, wobble and warp

Effects are children of a Cut or Engrave operation. They modify how that operation's paths are burned.

Hatch

Add effect ► Add hatch effect fills a closed shape with parallel lines instead of tracing its outline — the classic way to engrave a solid area with a *vector* (not raster) operation.

Label	Default	Meaning
Hatch Distance	1 mm	Spacing between hatch lines: smaller is darker and slower.
Angle	0 deg	Direction of the lines.
Angle Delta	0 deg	Rotate the pattern on each loop: 90 deg with 2 loops gives a cross-hatch.
Loops	1	Repeat the fill, rotating per angle delta.
Unidirectional	off	Burn every line in the same direction instead of back-and-forth.
Include Outline	off	Also trace the shape's outline.
Fill Style	scanline	Scanline, Eulerian or spiral fill algorithms.
Algorithm	auto	<i>Auto-Select</i> , <i>Scanbeam (Default)</i> or <i>Direct Grid (Fast)</i> . Jumping to a specific algorithm is a speed fix for pathological shapes.

Wobble

Add effect ► Add wobble effect replaces a path with a decorative oscillation — a wave, a row of circles, a gear feel — by moving the head side to side as it advances. Used for leather stitching

effects, cork, and lines that should look scratched or hand-drawn.

Label	Default	Meaning
Wobble Radius	0.5–1.5 mm	Amplitude of the sideways movement.
Wobble Interval	0.1 mm	How far the head travels before the pattern repeats.
Wobble Speed	50	How quickly the pattern revolves along the path.
Fill Style	circle	Circle, left/right circle, sinewave, sawtooth, jigsaw, gear, slowtooth or one of three meanders.

Warp

Add effect ► Add warp effect distorts the operation’s artwork to fit a containing shape you edit with the finger tool — useful for engraving onto curved or irregular objects where a straight projection would not follow the surface.

Effects hide their children

A wobble effect, in particular, is drawn as the wobbled result rather than as the original path. If a shape looked right before you added the effect and strange after, remove the effect and compare; **View ► Scene Appearance** and the operation’s *S* (visible) toggle control what you see without changing what burns.

7.7 Estimating before you commit

Two windows tell you what a job will cost in time:

- **Operation Info** (**Tools ► Operation Info**) lists each operation with its type, name, item count and runtime; press **Get Time Estimates** to fill in the last column. Unassigned elements appear as their own row, marked !.
- **Simulation** (**Simulate** on the Laser tab, or **Tools Simulation**) replays the job on screen and reports *Laser Time*, *Travel Time*, *Total Time*, *Extra Time*, and the corresponding distances. Chapter 16 uses this window properly.

At the machine 7.1 — Burn scrap first, always

For any new material or any setting you have not personally verified on this machine, run the job on scrap with the lid closed and watch it. The estimate is arithmetic done on your settings; the burn is physics. They agree most of the time.

Try it 7.1 — Build a two-operation test plaque

1. In the console, make two shapes: **rect** 10mm 10mm 40mm 40mm then **circle** 80mm 30mm 15mm.
2. Right-click *Operations* in the tree, **Append operation ► Append Cut**. Note it appears red.
3. Right-click *Operations* again, **Append operation ► Append Engrave**. Note it appears blue. Now two operations exist and the first is “exclusive” by default — only one of them will own the shapes.
4. Select both shapes in the scene (**Ctrl+A**), open the **Operations** pane, and click the blue *Engrave* square.
5. Select only the circle, and click the red *Cut* square.
6. Open **Tools ► Operation Info** and press **Get Time Estimates** . You should see both opera-

tions with one item each.

7. Select the rectangle's operation and change its *Speed* and *Power*. Watch the estimate change.

Chapter 8

Dialling in settings

Every laser, every tube, every diode and every sheet of material is slightly different. The recipes you find on the internet are starting points for someone else's machine. This chapter is about replacing them with numbers that belong to yours — systematically, and in an afternoon rather than over a year of frustration.

8.1 The method

Three rules make all the difference:

1. **Change one thing at a time.** A grid with speed on one axis and power on the other, all burned in a single job on a single piece of material, tells you more than fifty separate experiments.
2. **Write down what you burned.** The grid is worthless without a record of the settings and the result. Engrave the labels into the test piece, or photograph it with the numbers beside it.
3. **Test the material you will actually use.** Thickness, moisture, glue content and the colour of the sheet all change the answer. Plywood from two suppliers is two materials.

8.2 Test grids: the Parameter-Test window

Tools ► Laser-Tools ► Parameter-Test builds the grid for you. It has two tabs: **Generator**, where the grid is designed, and **Templates**, where you save and reload designs.

Walkthrough 8.1 — Burn a speed/power grid on 3 mm plywood

1. Open Tools ► Laser-Tools ► Parameter-Test.
2. In *Operation to test*, choose **Cut**, **Engrave**, **Raster**, **Image**, **Hatch** or **Wobble**. The parameter rows below change with the choice: Cut and Engrave give you *Speed*, *Power* and *Passes*; Raster and Image add *DPI* and *Overscan*; Hatch adds *Hatch Distance* and *Hatch Angle*; Wobble adds its four wobble parameters.
3. Fill in *First parameter (X-Axis)* and *Second parameter (Y-Axis)*: which parameter, how many steps (**Count**), the **Minimum** and **Maximum**, the cell **Width / Height**, the **Delta** between steps and the colour. A sensible first grid for cutting 3 mm plywood is speed 5–20 mm/s in five steps on X, and power 40–100% in five steps on Y.
4. Decide about labels. **Labels** prints a descriptive label down the sides of the grid; **Values** prints the numbers; **Simple** uses vector shapes for the labels. Leave both on for your first grid.
5. Optionally tick **Create boundary shape** to repeat the shape around the effect, which makes the grid easier to cut free afterwards.

6. Press **Create Pattern** . *This deletes all existing operations and elements* — the dialog says so, and asks whether you want to empty the project or keep it. On a fresh project, empty it.
7. Inspect the result on screen: it is a real, editable document, so you can move it onto your material's position, or change one row before burning.
8. Press **Queue to Spooler** . It plans the job in the background and puts it in the **Spooler**. It does *not* start it; press **Start** in the spooler when the material is in place.

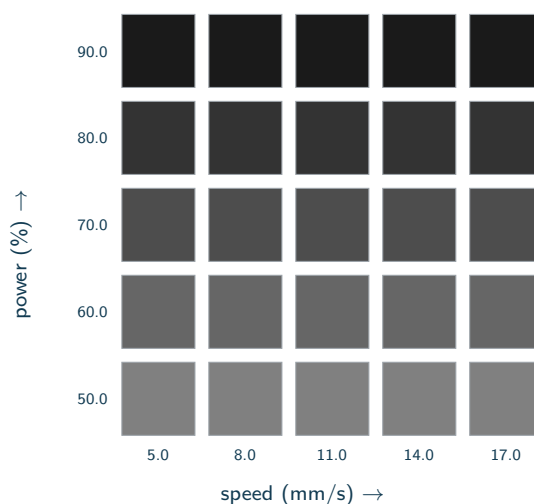


Figure 8.1: A completed 5×5 cut grid. Read it once: the squares that fell out define your cutting window; the top-right corner is burned through or charred; the bottom-left does not cut at all. The darkest squares that still cut cleanly are the ones worth writing down.

Reading the grid

Look for three boundaries rather than for a single best square:

- the **does-not-cut** boundary — everything below and right of it wastes material;
- the **clean-cut** band — squares that separate and have brown, not black, edges;
- the **charring** boundary — where the edge is black, sticky, or the material has a wide heat-affected zone around it.

Pick the setting in the clean-cut band that is nearest to the does-not-cut boundary: it is the fastest one that works, and it is the least likely to cause a fire. For engraving, the equivalent goal is the lightest burn that is consistently dark.

Save the grid as a template

In the **Templates** tab, type a name and press **Save** . You then have a one-click grid for the next new material, sized the way you like it. Reloading a template and pressing **Create Pattern** takes ten seconds.

8.3 Storing the answer: the Material Library

A grid gives you numbers; the **Material Library** keeps them. Open it with the **Material Library** button or **Tools ► Material Library**. An entry is a named collection of operation presets, keyed by *Material*, *Thickness* and *Laser type*, with an optional note.

Walkthrough 8.2 — Turn your grid into a library entry

1. Build the operations you want to keep — for example a **Cut** at your measured speed, power and passes, and an **Engrave** at its own numbers. If you have a saved project that already contains them, load it.
2. Open the Material Library and press **Get current** to capture the operations from the tree as a new entry. (Alternatively, **Add new** creates an empty entry and you add operations by name from the right-click menu: *Add Cut*, *Add Engrave*, *Add Raster*, *Add Image*, *Add diagonal Hatch*, *Add Wobble*, and so on.)
3. Fill in the *Information* box: a good *Title*, the *Material* and its *Thickness*, the *Laser* type, the laser's *Power* in watts and — for galvo machines — the *Lens-Size*. Add anything you would otherwise forget in the note field: supplier, tape or no tape, focus setting, whether it needed two passes.
4. Use the filter boxes at the top (*Material*, *Thickness*, *Laser*, **Reset Filter**) to check your entry lands where you expect, and press **Use for statusbar** if this is the entry you want the quick-assign squares to offer.
5. Later: select the entry and press **Load into Tree** to put its operations straight into your job.

The library imports other people's data too: **Import** reads EZcad, LightBurn and MeerK40t library files (*.lib, *.ini, *.clb, *.cfg), with options to compensate for a different lens size or a different laser power. That is worth knowing when you inherit a machine: the settings are probably already in a file somewhere, in units and geometry you do not share, which is exactly what the compensation fields are for.

Keep it honest

A library is only as good as its entries. If you change the tube, clean the lens, or switch suppliers, either re-test or mark the affected entries as stale. A wrong entry is more dangerous than no entry, because it saves time — right up until the job fails.

8.4 Kerf: cutting parts that fit

The beam removes material. That removed width is the *kerf*, and it is why a 10 mm tab does not fit a 10 mm slot.

Measuring your kerf

1. Cut a simple shape of known size — a 20 mm square in 3 mm plywood is ideal.
2. Measure the finished part. If it is 19.85 mm wide, the beam took 0.15 mm: 0.075 mm from each side.
3. Measure it in both directions and on two different materials if you cut both. Kerf varies with material, focus, speed and power — and with how well the machine tracks.

Then enter the total width in the operation's **Kerf compensation** field, for example 0.15mm. MeerK40t offsets each closed path by half that amount so the finished part comes out at the designed size. For installation instructions on the faster offsetting library, see Chapter 2: without `pyclipr` installed, offsets are computed by a slower fallback.

- For *fits that must be tight or loose*, do not rely on kerf compensation alone: design the clearance into the artwork. A “10 mm” slot intended for a 10 mm tab is always wrong; make it 10.1 mm or 10.2 mm and cut a test coupon.

- For one-off offsets, the element context menu is quicker: **Outline element(s)** produces an outline path at a given distance, and the **Offset shapes** submenu offsets a shape to the outside or the inside — useful for making a lid that matches a box, or a mask that surrounds a part.
- Compensation applies per operation, so cut and engrave can differ.

8.5 The machine's own calibration

Some wrong numbers are not material settings at all, but the machine's idea of distance. Calibrate these once, correctly, and they stay correct.

What	Symptom	How to fix
Axis scale (<code>scale_x</code> , <code>scale_y</code>)	A 100 mm move measures 103 mm	Jog exactly 100 mm, measure, and correct the scale factor in the device configuration by the ratio measured $\div$ commanded.
Backlash	Squares are not square, or lines repeat shifted in one direction	Newly exposes <i>Horizontal/Vertical backlash</i> in millimetres; other drivers need mechanical attention (belt tension, eccentric nuts).
Speed chart (Newly)	Corners rounded or overshooting at speed	The device's <i>Raster Chart</i> holds <i>Speed</i> $\leq$, <i>Acceleration Length</i> , <i>Backlash</i> and <i>Corner Speed</i> rows.
Acceleration (K40/Lihuiyu)	Raster lines ragged or the machine skipping at high speed	The Acceleration-Chart window (Tools ► Acceleration-Chart) has separate breakpoints for <i>Vector</i> , <i>Vertical Raster</i> and <i>Horizontal Raster</i> .
Rotary steps (GRBL)	Wrapped artwork stretched or squashed along the circumference	<code>rotarycal <measured_mm></code> recalculates steps/mm from a test burn; <code>rotarysuggest <diameter></code> estimates it from your mechanics.
Home corner	The head homes to the corner you do not expect, and jobs land mirrored	<i>Force Declared Home</i> in the device configuration.

8.6 Beyond speed and power

Four settings change results in ways that surprise people.

PPI / frequency

Pulses per inch, or kHz on newer machines. Lower PPI leaves a visible serration on acrylic edges and gives wood a rougher, more charred cut; higher PPI gives smoother edges but can overheat the material. If your machine exposes **Frequency**, run one row of your test grid across the values rather than guessing.

Focus and material height

Focus is depth of field, and half of “my settings stopped working” is a job run a millimetre out of focus. Focus on the surface of the material you cut, not on a fixed table height, and re-check it between pieces of different thickness.

Masking and surface protection

Masking tape or transfer film reduces scorch marks around a raster and matters more than most settings for the final appearance on light-coloured wood and on painted surfaces. Remember that tape changes what the beam sees first.

Air assist

Clearing the smoke changes the burn as much as 10% of power: too little and the cut re-deposits soot and catches fire; too much and a delicate engraving is blown pale.

At the machine 8.1 — Keep a record that ages well

The most valuable page in a workshop is not a settings table; it is a *sourced* settings table. For each material write down: supplier and product, thickness measured with callipers, date, machine, focus setting, air assist on or off, tape or no tape, speed, power, passes, PPI and the verdict in one word. Six months later those columns are the difference between a repeatable process and superstition.

Try it 8.1 — Build your own starting kit

1. Pick the material you use most. Burn a 5×5 cut grid and a 3×3 engrave grid on one piece.
2. Photograph both with a written card in shot, and keep the photographs with the physical samples in a labelled bag.
3. Measure the kerf from the cut grid and record it.
4. Enter the winning settings into the Material Library as two entries: one for cutting, one for engraving.
5. Repeat for your second material. Then stop: two well-characterised materials beat ten guesses.
6. From now on, when you have to guess on a new material, guess by interpolation from these two — and test the guess on scrap before it touches a real job.

Part III

Working with artwork

Chapter 9

Files, formats and saving

MeerK40t speaks more formats than it writes, and the one it writes is not the one most people expect. This chapter is about getting artwork in, keeping a project you can come back to, and producing the file your machine actually consumes.

9.1 The project file is an SVG

MeerK40t's own file format is **SVG** — the same Scalable Vector Graphics that Inkscape and Illustrator produce — extended with extra attributes in a `meerK40t:` namespace. Everything is in there: the elements, the operations with their settings, the remarks, and the notes you typed. Saving a project is therefore saving an SVG, and opening one is opening an SVG.

File ► New (`Ctrl+N`)

Clears operations, elements and notes. Your unsaved work is gone; there is no “are you sure” beyond the obvious.

File ► Open Project (`Ctrl+O`)

Clears the current project and loads a file. Hold `Shift` to be asked where the loaded content should be placed.

File ► Import File

Loads a file *into* the current project without clearing anything. This is how you combine artwork from several sources into one job. `Shift` again offers the placement target.

File ► Save (`Ctrl+S`)

Writes the project to the file it came from. If the project has more than one file attached, or the extension is not the plain SVG one, Save falls back to Save As.

File ► Save As (`Ctrl+Shift+S`)

Writes to a new file. You get three choices: *Scalable Vector Graphics* (the default, with MeerK40t's attributes), *SVG-Plain* (no extensions — a normal SVG that other programs will read cleanly, but which loses your operations) and *SVG-Compressed* (`.svgz`).

“SVG-Plain” discards the recipe

Saving plain is the right move when you want to hand the geometry to another program. If you open that file again in MeerK40t, you will find the shapes, but not the operations, the speeds or the notes. Keep the full-fat version as your working file and export a plain copy when you need to share.

One project per job, versions when it matters

The cheapest workflow that scales is: one project file per physical job, named after the job, saved with a copy for each successful material test (`tag-3mm-ply-v2.svg`). Use **File ► Import File** to pull in parts from your library of reusable drawings rather than rebuilding them.

9.2 The Recent list and drag and drop

File ► Recent keeps up to twenty files. Each entry loads with the same behaviour as **File ► Open** (clear first), and holding **Shift** while choosing asks for a placement target. Two extra items live in the submenu: **Clear project on load**, which decides whether a recent file replaces the current project or merges into it, and **Clear Recent**, which empties the list.

You can also drop files onto the main window: drag one or several files from your file manager and MeerK40t loads them all. Anything it does not recognise produces an error dialog listing the files it rejected — worth reading rather than dismissing, because the usual cause is a format that needs an optional dependency.

9.3 What can be loaded, and what can be saved

Filter in the Open dialog	Extensions	Notes
<i>All valid types</i>	—	The default; everything below.
<i>Scalable Vector Graphics</i>	svg, svgz	The native format.
<i>SVG (elements only)</i>	svg, svgz	Reads geometry, ignores MeerK40t's attributes.
<i>Portable Network Graphics, Bitmap Graphics, EPS Format, TIFF Format, GIF Format, Icon Format, JPEG Format, Webp Format</i>	png, bmp, eps, tiff, gif, ico, jpg/jpeg, webp	Raster images. EPS is read as a bitmap here.
<i>Inkscape supported files</i>	pdf, ai, cdr, cmx, wmf, vsd and more	Converted through an installed Inkscape executable. If Inkscape is missing, these formats are not offered.
<i>Drawing Exchange Format</i>	dxf	Requires the ezdxf package. Centring and 3D-polyline reading are preferences.
<i>EZCad2 Files</i>	ezd	Galvo job files, including their laser settings.
<i>LightBurn Files</i>	lbrn, lbrn2	LightBurn projects.
<i>XTool Files</i>	xcs	Two variants: with and without the stored laser settings.
<i>Gcode File</i>	gcode, nc, gc	Reread a GRBL job.
<i>Engrave Files</i>	egv	Lihuiyu/K40 job records.
<i>RDWorks File</i>	rd	Ruida job records.

On the way out there is exactly **one** saver: SVG, in the three flavours of the Save As dialog. Everything else you might want to produce — G-code for a diode machine, an **.egv** for a K40, an **.rd** for Ruida, an **.lmc** for a galvo — is a *device job file*, and it is produced by the job pipeline, not by Save As.

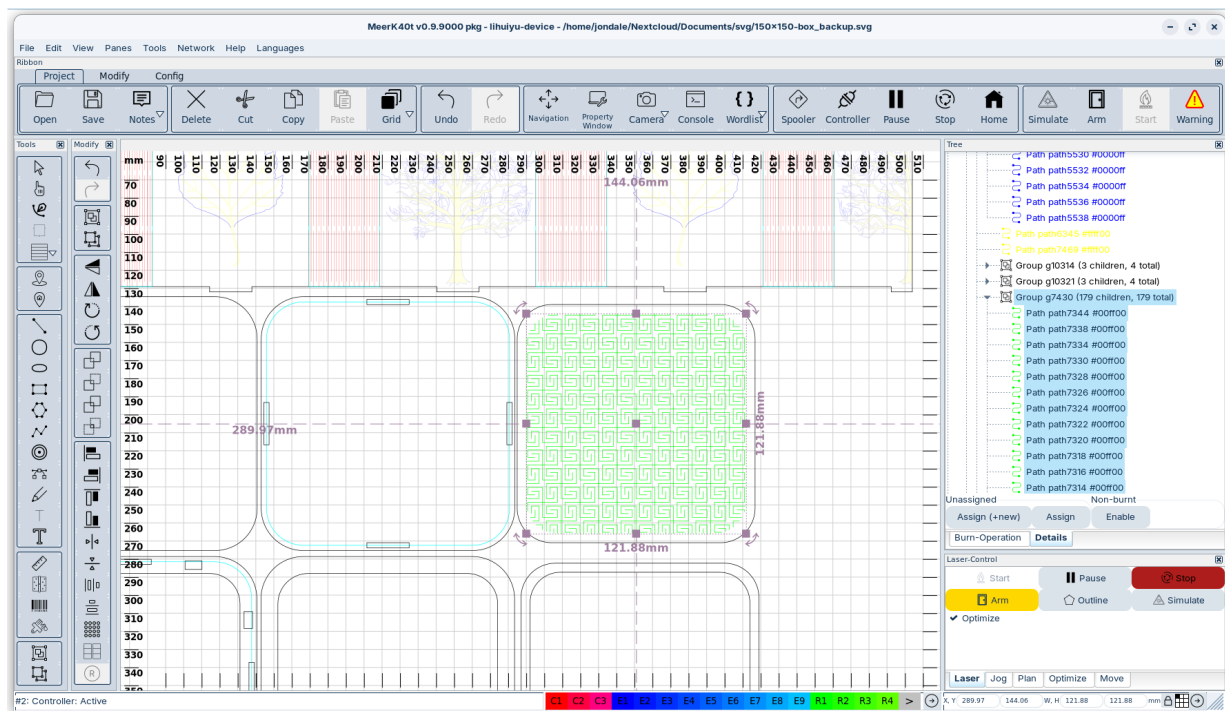


Figure 9.1: An SVG open in MeerK40t. The tree shows what the file actually contained: a group with 179 path children, each carrying its stroke colour, and the scene shows the dimension overlays drawn with the measuring tool. Nothing has been re-arranged by the import — this is a faithful reading of the file, which is exactly why importing good artwork is worth the effort.

9.4 Exporting a machine job file

Walkthrough 9.1 — Produce the file your machine reads

1. Build and optimise the plan as usual (Chapter 16).
2. On the **Laser-Control** pane, open the **Plan** tab.
3. Press **Export**. The dialog asks for a file name and offers the extension belonging to your active device: `.lmc` for a galvo, `.gcode` for GRBL, `.egv` for Lihuiyu, `.mos` for Moshi, `.hpgl` for Newly and `.rd` for Ruida.
4. The export runs in the background; watch the **Console** for progress. The plan commands behind the button are `save_job "<path>"` on the current plan.

This is how you use MeerK40t as a *design and planning* tool while running the job from an SD card, a USB stick or the vendor's own software. It is also the mechanism behind the ESP3D workflow on GRBL machines with Wi-Fi, where the exported file is uploaded to the controller's SD card and started from its own panel.

Lihuiyu users have a bonus pair of commands, `egv_import <file>` and `egv_export <file>`, for reading and writing that format directly from the console.

9.5 Import behaviour and the preferences that change it

Four preferences change what an import does. They live in **Edit ► Settings ► Preferences**, tab **Input/Output**:

Label	Default	Effect
SVG Viewport is Bed	on	The SVG's viewport dimensions are used to scale the rest of the elements. Turn it off if a file arrives at a bizarre size.
Load hidden objects to regmarks	on	Hidden objects in an SVG are put in the Regmarks branch rather than loaded as invisible elements. Vector editors use hidden layers for construction geometry, so this is usually what you want.
Inkscape	empty	Path to the Inkscape executable, enabling the Inkscape-supported formats.
Unsupported elements	<i>Always load into meerk40t</i>	What to do with SVG constructs MeerK40t cannot represent: load them anyway, convert the file with Inkscape first, or ask at load time.
DXF Center and Fit	on	Centre a DXF on the bed instead of leaving it at its own origin.
DXF: Try to read 3D-polylines	on	Read polylines that carry a Z coordinate, projected flat.

Image imports have their own preferences — *Image DPI Scaling*, *Center image on import*, *Scale oversized images* and *Create a file-node for imported image* — described in Chapter 6.

9.6 Pasting from other programs

Shift+Ctrl+V (Edit ► Paste from system) is more useful than it looks. It inspects the clipboard and does the right thing per type:

- a bitmap becomes an image element;
- text becomes a text element;
- a list of files is loaded as if you had dragged them in;
- a URL is fetched and placed on the bed.

So “copy an image from a browser, paste into MeerK40t” works, and so does “copy a path from Inkscape, paste into MeerK40t”.

9.7 Autosave, notes and startup commands

- **Autosave** writes your workspace to `\_autosave.svg` in the program's working directory every `autosave_interval` seconds (default 300, i.e. five minutes), if `autosave_active` is on. It is a safety net, not a recovery system: MeerK40t does not offer to reopen the autosave file at startup, so if the program crashes, look for that file yourself and open it manually.
- **Notes** attach free text to the project (**Notes** pane, Panes ► Tools ► Notes). They are saved inside the project file and, with `auto_note` on, the notes pane opens automatically when you load a file that has notes. This is the right place for “cut two of these, material is the birch ply from the back shelf”.
- **File startup commands** (Tools ► Auto-Exec) store console commands inside a file and run them when that file is loaded — handy for a project that always needs the same preparation, or for a template you hand to other people. The console side is `file_startup`.

9.8 The console route

Everything above has a text equivalent, which is what makes batch work possible:

```
load /home/me/designs/tag.svg      # load a project or artwork file
save /home/me/designs/tag-v2.svg   # write the project
save /home/me/tag-plain.svg -v plain # write a plain SVG
load_types                        # list everything that can be loaded
save_types                        # list everything that can be saved
plan0 save_job /home/me/tag.egv    # export the device job file
```

Try it 9.1 — Assemble one job from three sources

1. File ► New, then File ► Import File and load a vector drawing.
2. File ► Import File again and load a photograph. Confirm that the first drawing is still there — that is the difference between Open and Import.
3. Paste something from the system clipboard with **Shift+Ctrl+V** .
4. Arrange the three pieces, then File ► Save As **combined.svg**.
5. File ► Save As again, this time choosing *SVG-Plain*, and save as **combined-plain.svg**.
6. File ► Open Project the plain file. The geometry is there; the operations are gone. That is the value of the extended format — and the reason to keep both.

Chapter 10

The tree, and selecting things

The **Tree** is the inventory of your job: what you drew, and what you intend to do with it. Learning where things live in it — and how selection works in the two places you can select, the scene and the tree — turns most confusing moments into obvious ones.

10.1 Three branches, one document

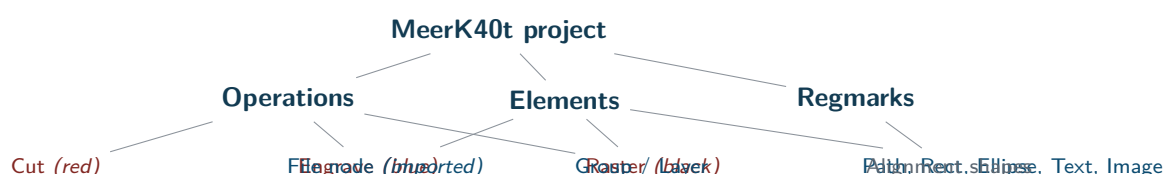


Figure 10.1: The document tree. Elements are the artwork; operations are the recipes; regmarks are alignment aids that never burn by accident. Operations hold *references* to elements rather than copies of them, which is why one shape can be cut and engraved in the same job and still remain a single object you can edit.

Operations

Created by you. Each one is a recipe with settings and a list of the elements it will burn. Right-click this branch to append operations (Chapter 7); right-click an operation for its properties, its burning order, and conversions.

Elements

Your artwork. Imported files arrive as *file* nodes containing groups containing paths, or as loose elements. You can group, ungroup, merge, duplicate and delete here.

Regmarks

Alignment marks, kept deliberately out of the way. Nothing in this branch burns unless you ask it to.

The pane is also a two-tab notebook: **Burn-Operation** and **Details**. On load, both top branches expand automatically.

10.2 Selecting: scene and tree

Selection is the single most important skill in the program, and it has two faces: *emphasis* (the element is highlighted, and operations, handles and the status bar apply to it) and the tree's own highlight.

Action	Result
Click an element	Selects it. With <code>select_smallest</code> on (the default), a click picks the <i>smallest</i> element under the pointer; hold <code>Ctrl</code> to pick the largest instead. The preference is under <code>Edit ► Settings ► Preferences</code> , tab <code>Scene</code> .
<code>Shift</code> + click	Adds the element to the selection.
Left-drag on empty space	Rubber-band select. Drag <i>right</i> to enclose (only fully enclosed elements are selected); drag <i>left</i> to touch (anything the band crosses is selected). <code>Shift</code> adds, <code>Ctrl</code> inverts.
<code>Ctrl+A</code>	Select all elements on the scene.
<code>Ctrl+I</code>	Invert the selection.
<code>Ctrl</code> + click	With the default setting, selects the largest element under the pointer.
Click in the tree	Selects the node, and emphasises the corresponding elements in the scene.
Double-click in the tree	Opens the node's properties.
Right click while drawing	Cancels the tool in progress and returns you to the select tool.

Once a selection exists, the status bar comes alive: the **Position** readout shows X, Y, width and height, and the handle checkboxes (`Move` , `Resize` , `Rotate` , `Skew`) appear. Those checkboxes decide which manipulation handles are drawn on the scene, in case you find them cluttered.

Selecting the unassigned

`Edit ► Select all non-assigned` is the practical cure for the most common mistake in the program: artwork that is sitting in the tree but belongs to no operation, and therefore will not burn. Run it whenever a job seems to be missing a piece.

10.3 The right-click menus, branch by branch

Nearly all tree functionality is in the context menus, and they differ by node type. This is the map.

On the *Operations* branch

- Append operation — `Append Image` , `Append Raster` , `Append Engrave` , `Append Cut` , `Append new Hatch` , `Append Dots` .
- Append special operation(s) — the movement, time and interaction steps listed in Chapter 7, plus `Append Shutdown` and `Append Console` .
- Classification — `Refresh classification for all` , `Classification for unassigned` , `Clear all assignments` , `Select unassigned elements` .
- Housekeeping: `Clear all` , `Clear unused` , `Enable all operations` , `Disable all operations` , `Toggle all operations` .
- Global operation settings , `Material Manager` and the `Reuse existing` toggle, which decides whether loading a file reuses the operations already in the tree instead of creating new ones.
- Placement commands: `Append absolute placement` , `Append relative placement` , `Remove all placements` .

On an operation

- Operation properties ; Enable / Disable ops ; Show / Hide contained elements ; Enable similar , Disable similar .
- Remove all items from operation ; Reset to device default values ; Reverse subitems order .
- Execute operation(s) and Simulate operation(s) — run or preview just this operation.
- Burning sequence — Burn first , Burn earlier , Burn later , Burn last . Dragging rows in the tree does the same job.
- Convert operation — Convert to Image , Convert to Raster , Convert to Engrave , Convert to Cut , Convert to Dots .
- Insert operation and Insert special operation(s) — add a new operation immediately before or after this one.
- Add effect (cut and engrave only) — Add hatch effect , Add wobble effect .
- Quick-sets: speed, power, Passes , DPI and Optimisation ► Reduce travel moves.

On an element or group

- Properties (Element properties , Group properties , Text properties , Image properties) .
- Structure: Group elements , Ungroup elements , Simplify group , Merge items , Break Subpaths , Merge elements , Convert to path , Convert to vector text .
- Visibility and locking: Toggle visibility , Hide elements , Show elements , Lock elements, prevents manipulations , Unlock element, allows manipulation .
- Duplication: Duplicate element(s) with Make 1 copy or Make n copies .
- Transforms as menu items: mirror *Horizontally/Vertically*, a percentage *Scale*, a *Rotate* by angle, Reify user changes , Reset user changes .
- Geometry tools: Outline element(s), Offset shapes (inside and outside), Apply special effect (line fills, wobbles, warp), Add a keyhole .
- Assignment: Assign Operation, Remove all assignments from operations , Exclusive assignment , Inherit stroke and classify similar , Inherit fill and classify similar .
- Tracing and images: Trace bitmap , Trace bitmap via vtracer , Vectorization, Move to remarks .
- Deletion, in two flavours: Delete group ... and all its child-elements fully removes the node from the tree, while Delete ... elements, as selected in scene removes only the selected members. The distinction matters when you have three copies and want to remove one.

On a file node, a remark, and the other branches

File node Convert to normal group (dissolve the file container), Reload , Open containing folder , Open in System , and Remove loaded file ... and all its child-elements fully . Reload is the working pattern: fix the artwork in Inkscape, save, right-click the file node in MeerK40t, reload, and your operations stay attached.

Remarks branch Clear all and Toggle visibility of remarks .

Remark element Move back to elements , Create placement , and Toggle magnet-lines with Around border and At center .

Elements branch Clear all , Remove all assignments from operations and the Classification sub-menu.

Blob `Convert to Elements` , `Execute Blob` , `Convert to Cutcode` — the blob node is what a job looks like after it has been converted to machine instructions.

10.4 Dragging in the tree

Dragging is not decoration; it is a first-class editing operation, and the rules differ by destination:

- Dropping elements onto an **operation** creates references — this is the assign-by-drag described in Chapter 7.
- Reordering **operations** changes the burning order.
- The *Operations* branch accepts only operations and utility operations. The *Elements* branch accepts geometry, images and groups. The *Remarks* branch accepts the same, and removes any operation references on the way in — that is how the program guarantees a remark will not accidentally burn.
- Dragging a remark onto an operation moves it back into the Elements branch, provided the `allow_reg_to_op_dragging` preference is on (it is, by default).

10.5 Remarks in practice

Registration marks exist to answer the question “where exactly is the thing I am engraving?”. Typical uses:

1. Put a shape in the Remarks branch that matches a physical feature of the workpiece — a corner, a hole, a printed square.
2. Position the artwork relative to it.
3. Burn the remarks with a light trace (or use the camera’s bed overlay) to find the alignment, then run the real job.

`Move to remarks` moves selected elements there; `Toggle visibility of remarks` shows them so you can position them; `Move back to elements` returns them to normal life. Hidden objects in an imported SVG also arrive here, when the `Load hidden objects to remarks` preference is on — which is a feature, not an accident: the hidden layer in somebody else’s drawing is often exactly the alignment reference you need.

10.6 Undo, redo and the editing history

Undo is snapshot-based and generous: by default the last twenty states are kept (`undo_levels`, changeable from 3 to 250 in the `Start` page of the preferences). `Edit ► Undo ( Ctrl+Z )` and `Edit ► Redo ( Ctrl+Shift+Z )` do the obvious thing, and the edit menu grows an `Editing History` submenu listing the states by name — clicking one jumps straight to it. The console equivalents are `undo`, `redo`, `undolist` and `save_restore_point`.

What undo does not cover

Undo restores the document tree. It does not take back a job that has already been sent to the machine, and it does not un-cut material. Everything that matters at the machine is protected by testing, not by `Ctrl+Z` .

Try it 10.1 — Learn the tree by breaking it gently

1. Load any SVG with a few shapes in it. Expand the file node and notice the elements; right-click it and try **Convert to normal group**.
2. Select two elements with **Shift** +click and group them (**Group elements**); then **Ungroup elements**.
3. Create a **Cut** operation, then drag one element onto it. Watch the operation grow a child.
4. Right-click that child and remove it from the operation. The element itself is still in the Elements branch — references, not copies.
5. Move an element to the Regmarks branch and confirm it stops appearing in jobs. Move it back.
6. Make a mess, then press **Ctrl+Z** ten times and watch the document walk backwards through your work.

Chapter 11

Moving, scaling, rotating, aligning

Nothing in a laser job is approximate. A part is the size it says it is, the holes are where the drawing says they are, and the two halves of a box either fit or they do not. This chapter is about the tools that make geometry exact: handles, numbers, alignment, distribution and snapping.

11.1 The transformation handles

Select anything and the scene grows handles around it. Each handle does one job:

Handle	What it does, and the modifiers that matter
Border	The selection box itself; drag inside it to move.
Rotation	The round handle above the box. Shift rotates in 5° steps — which is how you get exactly 45°.
Corners	Scale both axes. Alt lets the two axes move independently; Ctrl+Shift scales from the centre.
Sides	Scale one axis.
Skew	Slant the element: useful for italic-looking effects and perspective corrections.
Move (centre)	Move the selection. Shift temporarily ignores magnets and snapping — the fastest way to place something exactly where you clicked.
Rotation centre	Move the pivot the rotation handle turns around.
Reference	Toggle the element's status as the <i>reference object</i> .
Lock	Removes the locked status of an element. Locked elements cannot be dragged by accident.

Which handles are drawn is controlled by the **Move**, **Resize**, **Rotate** and **Skew** checkboxes in the status bar, and the current handle's meaning is spelled out in the status bar as you hover it.

Building a grid from one hole

Make one circle, set its size numerically, then use **Duplicate element(s)** with *Make 1 copy* and the arrow keys to step it across. For anything with more than a handful of repetitions, do the copy-and-step once, then use the **Alignment** window's **Distribute** tab (see below) to space them evenly.

11.2 Exact numbers: the Position pane

The **Position** pane (Panes ► Editing ► Position) is where precision lives. It has four fields — **X**, **Y**, **Width**, **Height** — plus two options:

- **Individ.** — apply the new value to each selected element *individually* rather than treating the selection as one box. This is the difference between “make all four holes 4 mm” and “make their bounding box 4 mm”.
- **Keep ratio** — scale width and height together.

Press **Enter** in a field to apply it. The units combo (mm, cm, inch, mil, \%) decides how your typed numbers are interpreted: with \% selected, a width of 50\% means half of the current width, which is a neat way to halve a shape without arithmetic. The small button beside the fields sets the *reference point* — which corner or edge of the element the X and Y coordinates refer to. If typing X 20mm appears to move a shape to the wrong place, the reference point is why.

Type, do not drag

Dragging is for exploring; typing is for deciding. As soon as you know a dimension — “the plate is 90 mm wide” — put the number in the **Position** pane and stop fiddling.

11.3 Nudging with the keyboard

When you know the direction but not the exact number, the arrow keys are faster than any dialog:

Keys	Movement per press
Arrow keys	1 mm
Shift +arrows	10 mm
Ctrl+Shift +arrows	0.1 mm — the finishing adjustment
Alt +arrows	One screen pixel, scaled to the current zoom
Alt+Shift +arrows	Ten screen pixels
a , d , w , s	Continuous movement left, right, up, down while held (they are bound to timer commands, so they repeat smoothly)
Numpad arrows	The same continuous movement as a / d / w / s

All of these act on the emphasised elements. They are ordinary console commands underneath (**translate 1mm 0** and friends), which means you can rebind them, script them, or use them in a macro by typing them into the console.

11.4 Alignment, distribution and arrangement

Tools ► Navigation, or the **Expert Mode** ribbon button, opens the **Alignment** window. It has three tabs, and it is one of the most under-used parts of the program.

Align

Two independent axes, each with four choices: *Leave*, *Left*, *Center*, *Right* for X, and *Leave*, *Top*, *Center*, *Bottom* for Y.

Control	Meaning
<i>Relative to:</i>	What the selection is aligned against: <i>Selection</i> (the selection’s own bounding box), <i>First Selected</i> , <i>Last Selected</i> , <i>Laserbed</i> (the bed rectangle) or <i>Reference-Object</i> .
<i>Treatment:</i>	<i>Individually</i> aligns each element to the reference; <i>As Group</i> moves the selection as one rigid block. Choosing wrongly here is the most common reason an alignment “does nothing useful”.
Align	Applies it. The info panel below tells you how many elements are selected.

The two instant recipes worth memorising: select everything and align to the *Laserbed* centre to place a job in the middle of the machine; select two elements and align *First Selected* to stack one exactly on the other.

Distribute

Distribution takes a row of objects and spaces them evenly. For each axis you choose what aspect is being distributed — *Leave, Left, Center, Right, Space* for X; *Top, Center, Bottom, Space* for Y — then:

- **Keep first + last inside** keeps the two end elements where they are and distributes everything between them. With it off, the whole row is laid out from the ends inward.
- **Rotate** rotates each element to follow the path it is being placed along — the tool for a row of tick marks around a curve.
- *Work-Sequence* (*Position, First Selected, Last Selected*) decides the order the elements are considered in, which matters when they are not in a clean line.
- *Treatment* (*Position, Shape, Points, Laserbed, Ref-Object*) decides what is measured: element origins, bounding boxes, individual path points, and so on. *Points* is the one to reach for when distributing the nodes of a single path.

Arrange

The **Arrange** tab lays a selection out as a grid: choose the number of columns and rows and the gaps, and press the button. It is the fastest way to build a sheet of small parts for cutting — twelve keyring tags on one piece of plywood, evenly spaced.

11.5 Reference objects

A reference object is a shape that other alignment operations measure against without being moved by them. Two things make it useful:

- It gives you a fixed origin that is not the bed — the corner of a pre-cut panel, for instance.
- It survives multiple alignment passes, because it is never treated as one of the things being aligned.

Right-click a selected element and choose **Reference Object** — **Become reference object**, **Align to reference object**, **Clear reference object**. The reference object is drawn with a distinctive handle so you can see which one it is.

11.6 Snapping and magnets

Snapping pulls the pointer to interesting places while you drag. Magnets are the same idea with permanent, user-defined lines. Together they make manual work accurate enough to trust.

Snap-Options

The **Snap-Options** pane (Panels ► Editing ► Snap-Options) collects the snap preferences:

Label	Default	Meaning
Snap to Grid	on	Snap to the nearest grid intersection.
Snap to Element	off	Snap to the nearest point of another element.
Display-Distance	45 px	How far away snap points light up on screen.
Element-Point-Snap-Threshold	20 px	Attraction radius for element points (1–75).
Grid-Point-Snap-Threshold	15 px	Attraction radius for the grid (1–75).
Show snap line	on	Draw a preview line from the pointer to the snap point.
Instant Snap Display	off	Compute and show snap points continuously while moving. Costs performance on complex drawings.
Clear magnets on File/New	on	A new project starts without leftover magnet lines.

Snap to Grid and **Snap to Element** also appear as checkboxes in the status bar, which is where most people toggle them while working.

Magnet-Options

Magnets are alignment lines you place deliberately. The **Magnet-Options** pane has two tabs.

Actions places and clears them:

- *Set single line*: type a *Position* and press **X** or **Y** to place a magnet line there.
- *Set around selection*: horizontal buttons **Left**, **Center**, **Right**, plus **3**, **4**, **5** to divide the selection into thirds, quarters or fifths; the same set vertically with **Top**, **Center**, **Bottom**.
- **Clear all**, **Clear X**, **Clear Y**.

Options controls how strongly magnets pull and where they act: *Attraction areas...* (**Left/Right Side**, **Top/Bottom Side**, **Center**) and *Attraction strength...* (*Off*, *Weak*, *Normal*, *Strong*, *Very Strong*, *Enormous*), with **Load** / **Save** and a named template combo for storing a configuration.

Regmark elements can carry magnet lines around their border or centre (Toggle magnet-lines on a regmark). That combination — a regmark that represents a physical feature, with magnets along its edges — is how you place artwork precisely onto a pre-existing object.

Try it 11.1 — Build a twelve-tag cutting sheet

1. Draw or import a single small part. Check its real size in the **Position** pane and set it exactly.
2. Select it and **Duplicate element(s) ► Make 1 copy**. Position the copy to the right of the original, using the arrow keys for the last few millimetres.
3. Select both, duplicate again, and use the **Alignment** window's **Distribute** tab (X axis: *Space*) to make the row even.
4. Select the row and duplicate it; align the copy below the first with *Align>Y: Bottom, Relative to: First Selected*.
5. With all four, duplicate twice more and use **Arrange** to lay out three columns and four rows with a 5 mm gap.
6. Check the whole sheet against the bed rectangle with **Ctrl+B**, then assign everything

to your cut operation.

Chapter 12

Paths and nodes

Sooner or later an imported drawing is almost right, or a shape you need does not exist in any library. That is when you work below the level of “rectangle and circle” and edit the actual geometry. This chapter covers the drawing tools, the node editor, and the operations that reshape paths.

12.1 What an element actually is

Elements come in a few kinds, and the kind decides which tools apply:

Kind	Behaviour
Rectangle, ellipse, circle, line, polyline	<i>Parametric</i> shapes: they remember their definition, so width, height, radius and corner behaviour stay editable.
Path	Free geometry: a sequence of points, lines and curves. Imported SVG drawings are almost always paths.
Text	Either bitmap text (a rendered glyph block) or vector text from a font file (Chapter 13).
Image	A bitmap (Chapter 14).
Group, layer, file	Containers. They have no geometry of their own.

Convert to path turns a parametric shape or a text element into a path, after which it can be edited node by node but no longer by its original parameters. Two siblings of that command exist for undoing the transformation in spirit: **Reify user changes** commits the changes you made by dragging (so that later parameter edits no longer fight with them), and **Reset user changes** throws those changes away and restores the parametric shape.

Convert late

Keep shapes parametric while you are still deciding sizes — a rectangle you can still set to 60 mm in the **Position** pane is worth more than a path you have to measure. Convert when you need to move individual corners, not before.

12.2 Drawing

The drawing tools live in the **Tools** icon bar and each has simple click semantics:

Tool	How it works
Line	Click the start, click the end.
Rectangle ,	Click one corner or centre, click the opposite. Set exact sizes afterwards in the Position pane.
Circle ,	
Ellipse	
Polygon ,	Left click adds a point and a segment; double-click completes the shape; right-click cancels. <i>Polygon</i> offers sub-modes: <i>Freehand</i> , <i>Regular</i> , <i>Star 1</i> , <i>Star 2</i> and <i>Grow</i> .
Polyline	
Vector	The bezier tool: click for a corner point, click and hold to pull out a curve handle, double-click to complete, right-click to end an open path. This is the tool to reach for when you are tracing something by hand.
Draw	Freehand drawing: the path follows your pointer.
Point	Places a point element — the input for a Dots operation (drilling, perforation).
Text	Places a text element.
Job Start	Places a job-starting position on the scene (<i>placement</i>).

Two ribbon modes are worth knowing because they are not shapes at all: **Measure** draws dimension lines (the overlays visible in Figure 9.1) without adding elements to the job, and the *nudge* ribbon mode lets you push a drawn line around as if it were physical — used for organic, hand-drawn-looking paths.

Drawing is not design

Everything in this chapter works, and people do build real parts with it. If your artwork is more than a dozen curves, draw it in a vector editor and import it. Chapter 9 covers the round trip, including the trick of right-clicking a file node and choosing **Reload** after editing the source.

12.3 Editing nodes

Select a path and activate **Node Edit** in the ribbon. The tool shows the path's nodes — the points where segments meet and where curves are controlled — and lets you work on them directly:

- **Drag a node** to move it. Snapping applies, so a node can be dropped precisely onto a grid intersection or another element's point.
- **Node Move** is the simpler sibling tool, intended purely for dragging nodes to new positions.
- **Point Move** does the same job for point elements and placements.
- **Parametric Edit** appears for parametric shapes: instead of editing raw nodes you edit the shape's underlying parameters, so a star stays a star while you change its point count or inner radius.
- **Tab Edit** edits the *tabs* of an element — the short uncut segments that hold a part in place during cutting, so it does not shift or fall through. Tabs are what make a cut sheet survive the last few seconds of the job.

Node editing is the one place where the geometry is genuinely fragile: a path that looks closed may have a 0.02 mm gap, and a laser will happily leave a part hanging by that gap. If a shape should be a closed loop, check it afterwards by cutting it on scrap.

12.4 Reshaping paths

These commands live on the element context menu, or in the console:

Command	What it does
Break Subpaths	Splits one path element into its separate subpaths, so each can be moved, assigned or deleted on its own. The cure for a drawing where “one object” is really forty.
Merge elements	Joins selected paths into a single element.
Group elements /	Groups are organisational only: they do not alter geometry and do not merge paths.
Ungroup elements	
Simplify group	Reduces nested group structure without changing geometry.
Remove transparent objects	Deletes elements that would burn nothing (no stroke and no fill). Useful after a heavy import.
Outline element(s)	Creates a new path offset by a given distance — for example 1mm. Not the same as kerf compensation: this makes a new element rather than adjusting the operation.
Offset shapes	<code>...to outside</code> and <code>...to inside</code> by a chosen distance (1–5 mm), used for lids, masks and press fits.
stitch [tolerance]	Connects path ends that are within the tolerance of each other, reducing the number of separate burns and the travel between them. Console only; the cut planner can also stitch automatically (Chapter 16).
clipper	Boolean combination of selected elements: the first argument is <code>union</code> , <code>difference</code> , <code>intersection</code> or <code>xor</code> . Console only — there is no menu entry for it — with options for fill type, interpolation and whether the originals are kept.
offset <distance>, offset2, offsetsimple	The console equivalents of the offset menus, with options for join type, separate output and interpolation.
pocket <offset> <repeats>	Pocketing: repeated inward offsets that hollow an area out, one ring per repeat.

12.5 Generating shapes you cannot draw

Three generators produce geometry that would be tedious to draw by hand.

Living hinges

The **Hinge** window generates a bendable hinge pattern — the classic laser-cut lattice that lets plywood flex. You give it the *X* and *Y* coordinate of the hinge area, its *Width* and *Height*, and choose a *Pattern* and *Algorithm* from the combos, with a *Rotation* and the *Cell-Width / Cell-Height* ratios that control the lattice. **Preview Pattern** shows the result, **Preview Shape** shows the boundary it will occupy, **Apply to scene** puts it into the drawing, and **Default Values** resets the numbers. Living hinges are cut, not engraved, so assign the result to a **Cut** operation and expect a slow job: this is a lot of path.

Tabs and keyholes

Tab Edit places the small uncut bridges described above. The keyhole tool (right-click an element, **Add a keyhole**) does the opposite job: it combines two elements so that one is engraved through the other as a mask — used for nested artwork and for engraving a picture into a shape.

Parametric generators (console)

A family of console commands builds mathematical shapes directly:

```
pgrid 10mm 10mm 4 3      # a grid of copies: cell size, columns, rows
shape 5 0 0 20mm         # star/polygon: corners, x, y, radius
growingshape 2 2 5 6 10mm # a growing spiral-like shape
cycloid 2 2 10mm 4mm 20   # cycloid curve
fractal_tree 2 2 5 6      # a fractal tree
```

Each takes sizes and counts; **shape** and **growingshape** accept options such as `--angle`, `--ratio` and `--density`. They are not in any menu, which is deliberate: they are the escape hatch for the shapes a GUI cannot anticipate. Once created, the result is a normal element that the **Parametric Edit** tool can continue to adjust.

Try it 12.1 — Rescue a messy import

Take an SVG that has too many objects and make it cuttable.

1. Import the file and expand the tree. Note how many elements arrive and whether they are grouped.
2. Use **Measure** to check one known dimension against the real artwork. If it is wrong, fix the size in the **Position** pane before anything else.
3. Find a path that should be several objects, select it, and use **Break Subpaths**. Move the pieces apart to see what they are.
4. Use **Remove transparent objects** to shed anything invisible.
5. Add a **Cut** operation for the outlines and a **Engrave** operation for the interior detail, assigning by colour or by selection.
6. Use `stitch 0.1mm` in the console and compare the plan's travel distance before and after in the **Simulation** window.
7. Save as a new project file. Your rescue is now reusable artwork.

Chapter 13

Text and fonts

Lettering is most of what a laser is used for: names, part numbers, signs, serial numbers, batch codes. MeerK40t handles text in two fundamentally different ways, and choosing the right one decides how the job looks.

13.1 Two kinds of text

Bitmap text

An ordinary text element (`Text` in the **Tools** bar, or `text WORKSHOP` in the console). It behaves like a picture of text: fine for engraving, and it can also be converted into paths.

Vector text

Text drawn as paths from a font file, created with the *linetext* tool. Every stroke is a real vector, so it can be cut, scaled to any size without losing crispness, welded, and given a stroke width for a bolder look.

Which to use:

- **Cutting letters out** (a stencil, a sign, letters with holes): vector text, assigned to a **Cut** operation.
- **Engraving crisp small lettering**: vector text in an **Engrave** operation. Thin, clean, repeatable.
- **Large filled letters** on a plaque: either an **Engrave** operation with a *hatch effect* (Chapter 7) or a **Raster** operation, which fills the letters by scanning them.
- **Text that will be edited in a picture editor**, or where the exact typography matters more than the geometry: bitmap text, burned as a raster.

13.2 Choosing a font

MeerK40t can use three families of font:

Type	Extension	Character
Hershey fonts	.jhf	The classic single-stroke engraving fonts. Thin, even, ideal for small lettering and for cutting.
Autocad shape fonts	.shx	Single-stroke engineering lettering; crisp and utilitarian.
TrueType fonts	.ttf	Ordinary desktop fonts with outlines. Great for display lettering, but thin fonts vanish at small sizes when cut.

Open the `Font-Manager` (from `Edit ► Settings ► Font-Manager`, or the preferences window's `Fonts` button). It shows an information panel, a *Font-Work-Directory* with a picker for your own

fonts, the list of *System font locations* MeerK40t searches, the list of fonts it has found with a preview, and the buttons **Import**, **Delete**, **Refresh**. The *Goto a font-source* combo jumps to the well-known collections — the Hershey font sets and the Autocad SHX fonts. Importing reports how many fonts succeeded and how many failed, which is worth reading: one bad file in a batch does not stop the rest.

Pick fonts that survive the laser

A font with hairlines will burn into a thin scorch mark, and a font with tight counters (the enclosed parts of **e**, **a**, **o**) will fill in when it is cut small. Test your intended font at the size you intend to use — a line of text on scrap costs seconds and settles the argument completely.

13.3 Working with text elements

Select a text element and choose **Text properties** (or **LineText properties** for vector text). The property panel offers:

Control	What it does
<i>Content</i>	The text itself. Multi-line content is supported.
<i>Left / Center / Right</i>	Alignment for multi-line text.
font size buttons	Larger and smaller by one step. Hold Shift or Ctrl for bigger steps; right-click resets to the default.
character-gap buttons	Increase or decrease the kerning between characters.
line-distance buttons	Increase or reduce the space between lines.
Weld	Merges overlapping characters into one outline — essential for script fonts and for anything where two letters touch, since otherwise they cut as two pieces that fall apart.
font list	Double-click a font to apply it there and then; the preview below updates as you browse.
Insert symbol	Right-click the text box for Paste and Insert symbol , which opens a glyph picker listing each character with its Unicode value, ASCII code and preview — the practical way to add °, μ, € or a non-Latin character.

If a font in an imported design is not installed, the panel says *Font missing* and names it. Install the font, or switch the text to a font you have — do not begin a burn with a missing-font warning on screen, because what you are looking at is not what will be cut.

13.4 Text that changes between burns: wordlists

A wordlist turns a text element into a template. Write a placeholder in the text, such as {NAME}, and MeerK40t substitutes the current wordlist entry at burn time. The index then advances by one, so the next burn of the same job carries the next value — which is what makes a single project produce a hundred numbered parts.

The Wordlist Editor

Open **Edit ► Settings ► Wordlist-Editor**, or the **Wordlist** pane with its **Edit** button. Each entry has a *Name*, a type and an *Index*. The editor's controls include:

- *Filter* to find an entry by name;
- *Start Index for CSV-based data* and *Start Index for field* to control where counting begins;

- **Jump** to position the index directly;
- **Add Text** and the list tools (add, edit, delete entries and their values), and a tab containing the how-to text for the feature.

Besides the lists you create, several entries exist automatically and are useful in labels and serial plates: **version**, **date**, **time**, **op_device**, **op_speed**, **op_power**, **op_dpi** and **op_passes**. A date-stamped test piece costs nothing to produce when the text element contains `{date}` and `{time}`.

Counting without a computer in the loop

The **Wordlist** pane's **Next** and **Prev** buttons step the whole list forwards or backwards — left-click for pages and right-click for single entries. Between jobs, one click in that pane is often all the bookkeeping a production run needs. The element context menu has matching **Increase Wordlist-Reference** and **Decrease Wordlist-Reference** entries for adjusting a single reference, and `{NAME##+1}` in the text reaches forward within the list while writing.

CSV data

A list can come from a file: the header row of a CSV becomes the entry names, and the rows become the values, with the *Start Index for CSV-based data* setting deciding where to begin. That is the mechanism for engraving names onto a stack of gifts, or part numbers onto a tray of brackets, from a spreadsheet.

Walkthrough 13.1 — Number ten tags without touching the design

1. Design one tag with a text element reading No. `{SERIAL}`.
2. Open the Wordlist Editor, add a wordlist called **SERIAL** and enter ten values: 001 to 010.
3. Set the index to the first value.
4. Make a cutting sheet of ten tags, all using that same text element.
5. Burn the job. Every tag carries the next value in sequence, and the index is left ready for the next sheet.
6. Check the index in the **Wordlist** pane before re-running anything; if the job is repeated, the numbering continues rather than restarting.

13.5 Turning text into geometry

Three conversions appear on the text element's context menu, and they exist to solve three different problems:

- **Convert to path** — the text becomes paths you can edit node by node. Do this when you must adjust a single letter.
- **Convert to vector text** — a bitmap text element is re-drawn as vector text using the font of the same name. Do this when the design arrived as outlines and you want stroke-based lettering.
- **Trace bitmap** — vectorises the rendered text. Rarely the best option, but it works on text of unknown origin (Chapter 14).

Try it 13.1 — Make a stencil

1. Create vector text with your chosen word or initials, sized so the lettering is at least 20 mm tall.
2. Choose a stencil-friendly font, or use **Weld** on a script font so the letters hold together.
3. Convert it to a path and use **Break Subpaths** to separate the letters.
4. Add small connector bridges with **Tab Edit** or by drawing short lines across the cut, so the counters (the middles of A, O, R) do not fall out.
5. Assign everything to a **Cut** operation, position it, and cut it in card or thin plywood.
6. Adjust the bridges and re-cut. The second attempt is always the one you keep — which is the whole point of testing in card first.

Chapter 14

Images: editing, splitting and tracing

Chapter 6 showed how to burn an image. This chapter is about everything that happens *around* that burn: preparing the picture, cutting it into pieces that fit the bed, masking one shape through another, and turning a bitmap into cuttable vectors.

14.1 Preparing the picture first

The single biggest quality improvement in image engraving happens before MeerK40t is involved. In any image editor:

1. Convert to grayscale. Colour information becomes noise in a laser job.
2. Correct the contrast. A laser has a much smaller effective tonal range than a screen; a low-contrast photo engraves as an even grey smear.
3. Sharpen lightly, and consider a slight blur: engraving resolves detail worse than a screen, so a little smoothing beats a little sharpening for photographs.
4. Crop to the subject and resize so that the picture is roughly 200–300 dpi at the final physical size. More pixels than the burn can resolve only cost time.
5. Save as PNG. Avoid JPEG artefacts, which dither into random speckle.

Then import, and use the panels from Chapter 6 — **Contrast** , **Halftone** , **Tone** , **Gamma** , **Sharpen** , **Auto Contrast** — for the final taste adjustment in laser terms.

14.2 Image manipulations in the tree

Right-clicking an image element gives a large **Image** submenu. The ones that earn their place:

Item	Effect
Dither to 1 bit	Hard conversion to pure black and white. The clean starting point for line art scanned or photographed.
Invert image	Swaps black and white — for white-on-black engravings and for inverting a dark background before burning.
Mirror horizontal , Flip vertical , Rotate 90° CW , Rotate 90° CCW	Orientation fixes that operate on the pixels rather than on the element's transform.
Identify inner white areas , Split image along white areas	Finds enclosed white regions (for instance the holes in a letter, or the gaps in a logo) and can split the picture into separate pieces for them.
Split image into subimages	Divides the picture into its connected non-white regions — one element per object.
Divide into <i>n</i> images	Splits by grey level into a fixed number of sub-images.
Save original image to output.png / Save processed image to output.png	Writes the bitmap out so you can inspect what the laser will actually see. Invaluable when a job burns something you did not expect.
From Original / From Modified	Passthrough choice: burn the original pixels or your processed version.
Unlock manipulations / Lock manipulations	Prevents accidental changes to a picture you have finished tuning.

Right-clicking the image in the tree also offers **Image properties** , **Remove effect** and the Raster-Wizard scripts introduced in Chapter 6, plus **Convert to Path** with the choices *Horizontal*, *Vertical*, *Bidirectional* and *Optimize travel* — which converts the bitmap into path elements following the scan lines of the image, a technique for producing double-line lettering and outlines from artwork.

14.3 The Image Splitter: pictures bigger than the bed

Tools ► Editing ► Image Splitting, or the **Image ops** button, opens *Create split images* with two tabs.

Walkthrough 14.1 — Engrave a picture wider than the machine

1. Make the artwork the full size you want in reality — for example 600 mm wide on a 300 mm bed. Do not shrink it to fit.
2. Open *Create split images* and go to **Render + Split** .
3. Set *X-Axis* and *Y-Axis* to the number of tiles: two columns for a 600 mm picture on a 300 mm bed. Set *Order to process* to *Selection*, *First Selected* or *Last Selected* as appropriate.
4. Set *Image resolution: DPI* to your burning resolution — 500 is the default, 250 is often enough.
5. Press **Create split images** . The dialog reports how many elements are selected and then replaces them with the tiles.
6. Assign the tiles to your **Image** operation. Because each tile is its own element, you can now run them one at a time, repositioning the material between runs, or use the machine's own repeat feature.

The second tab, **Keyhole operation** , builds a *mask*: choose *Keyhole Object* (first or last selected), the *DPI*, *Invert* and *Trace*, then press **Create keyhole image** . The result is artwork where one

selected object is engraved through another as a keyhole — a picture inside a frame outline, a logo inside a control panel cut-out.

Both tools are also console commands, which is what to use in scripts: `render_split <columns> <rows> <dpi>` and `render_keyhole <dpi>`.

14.4 The image command tree

For repeatable work, the **image** command does everything the properties panels do, and a good deal more. Every transformation in this list applies to the selected image elements:

Command	Effect
<code>threshold <tmax> <tmin></code>	Cut the tonal range to pure black and white between two limits.
<code>dither -m <method></code>	Dither to one bit with a named algorithm.
<code>remove <color> -d <distance></code>	Remove a colour and everything within a distance of it — the standard way to erase a white background.
<code>crop <left> <upper> <right> <lower></code>	Crop by pixel.
<code>contrast <factor>, brightness <factor></code>	Tone adjustments.
<code>sharpness <factor>, blur, sharpen, edge_enhance, find_edges, emboss, smooth, contour, detail</code>	The usual image filters.
<code>quantize <colors></code>	Reduce to a limited palette.
<code>solarize <threshold>, invert, equalize, autocontrast <cutoff></code>	Tonal transformations.
<code>grayscale -m <method></code>	Convert to grayscale by red, green, blue or alpha.
<code>linecut <x1> <y1> <x2> <y2></code>	Cut the image along a line.
<code>slice <x>, slash <y></code>	Cut the picture into two images vertically or horizontally.
<code>pop <left> <upper> <right> <lower></code>	Remove pixels to make rastering more efficient.
<code>halftone</code>	Convert to a halftone dot screen.
<code>innerwhite, identify_contour</code>	Find enclosed white areas, and find contours for vectorisation.
<code>linefill</code>	Create a line-fill representation of the image — a path-based engraving instead of a raster.
<code>background</code>	Use the image as the bed background, for camera-style overlay work.
<code>save <filename></code>	Write the image to disk (add <code>--processed</code> for the processed version).

14.5 Tracing: from bitmap to cuttable vector

Turning a logo that only exists as a photograph or a PNG into paths you can cut is the oldest trick in laser work, and MeerK40t offers three engines for it.

Vectorisation with potrace

The standard engine is potrace. Open an image's properties and use the **Vectorisation** panel. If the engine is not installed, the panel says so — it needs the **potrace** provider, available through

`pip install pypotrace` on Linux with the `libpotrace-dev` and `libagg-dev` packages.

Control	Default	Meaning
<i>Turnpolicy</i>	Minority	How ambiguous pixels are resolved. Change it when thin features are dropped or filled in.
<i>Despeckle</i>	2	Suppresses speckles up to this size in pixels. Raise it to remove scanner noise and JPEG rubbish; raise it too far and small real features disappear.
<i>Corners</i>	9	Threshold for corner detection; lower values round off corners, higher values keep them sharp.
<i>Simplify</i>	on	Reduces the number of bezier segments.
<i>Tolerance</i>	20	How much error simplification may introduce.
<i>Black-Level</i>	50	Where “black” begins. Raise it to ignore a grey background, lower it to keep faint features.

Press **Preview** to see the result before committing, then **Vectorize**. The output is a path element with a normal stroke, ready to be assigned to a cut or engrave operation. The console equivalent is `vectorize` with options `-z` (turnpolicy), `-t` (despeckle), `-a` (corners), `-n`, `-O` (tolerance), `-k` (black level), `-d` (dpi), `-C` (colour) and `-i` (invert) — the whole panel is one command, which makes batch vectorisation easy.

VTracer and contour detection

- **Trace bitmap via vtracer** uses the VTracer engine instead, a colour-capable tracer. It requires the `vtracer` package (`pip install vtracer`); when it is missing, the console says so plainly.
- *Contour recognition* (the **Contour recognition** panel) uses OpenCV instead of a tracer. It offers its own *Contrast* controls, an *Invert* option, a choice between the original and the dithered image, *Minimal* and *Maximal* size limits in percent, a *Contour simplification* choice (*None*, *Visvalingam*, *Douglas-Peucker*) and a *Detection Method* choice of *Polygons* or *Bounding rectangles*. Press **Automatic update**, **Update** or **Generate contours**, and it lists what it found with areas, reporting “No contours found.” or how many it made and how long it took.
- **Generate placements** turns detected contours into *placements* — positioning references for each detected object, with a choice of *Edge (prefer short side)*, *Edge (prefer long side)*, *Center (unrotated)* or *Center (rotated)*. This is the mechanical way to find the centre of forty widgets on a jig.
- The tree’s **Vectorization** submenu wraps the same tools with useful presets: *Contour detection — shapes* and *— bounding*, *Ignore inner areas*, and size classes *Big objects*, *Normal objects*, *Small objects*.

Tracing is not magic, and it is not cutting

A traced photograph always contains far more paths than you want, with nodes everywhere and edges that double back on themselves. A traced logo has ragged edges that the original never had. Always inspect the result, delete junk with **Break Subpaths** and **Remove transparent objects**, and remember that a traced outline follows the *darkness*, not the shape you perceive — you often need the *Black-Level* and *Despeckle* controls before the result is usable. For anything that already exists as a vector file, get the vector file instead.

Two meanings of “trace”

The console command `trace <method> <resolution>` does *not* vectorise a bitmap. It creates a hull or cut outline around selected elements, with methods `quick`, `hull`, `segment` and `circle`. Bitmap vectorisation is `vectorize`.

Try it 14.1 — From photograph to cuttable silhouette

1. Take a clear, high-contrast photograph of a simple object on a plain background and import it.
2. In the image's properties, use `Contrast` to push the background to white and the object to black. `remove white -d 30` in the console is the quick version.
3. Open `Vectorisation`, set *Black-Level* around 50 and *Despeckle* around 3, and press `Preview`.
4. Press `Vectorize`, then inspect the result in the tree. Delete any stray paths; use `Break Subpaths` if a single path contains several objects.
5. Assign the paths to a **Cut** and physically cut the silhouette from card. Compare it with the object.
6. Iterate on *Despeckle* and *Black-Level* until the silhouette matches. Two or three iterations is normal.

Part IV

The machine

Chapter 15

Where the job runs

A job is not sent to “the laser” in the abstract; it is cut at a physical position on a physical sheet, and everything before that moment is guesswork. This chapter is about controlling where the burn lands: the coordinate system, the alignment aids MeerK40t gives you, and the camera.

15.1 The scene is the bed

The most important idea in this chapter is also the simplest: **the scene is the machine’s bed at full size, in millimetres**. If your artwork sits 20 mm from the left edge of the bed rectangle on screen and 30 mm from the top, that is where it will burn, relative to the machine’s zero point. There is no separate “send to bed” step and no hidden offset.

The bed rectangle and its internal origin come from three device settings, all covered in Chapter 4:

- **Width** and **Height** — the bed.
- **Flip X** , **Flip Y** , **Swap X and Y** — the direction of each axis as the machine sees it.
- **Force Declared Home** — which corner the machine calls zero.

Two things called “origin”

Machine origin is where the controller’s native zero sits, and it is decided by the driver and the machine. *User origin* is the corner of the bed rectangle as drawn on your scene, which you can move by changing the home-corner setting. When you read **devinfo** in the console you get both numbers: the logical position in scene coordinates and the native position in machine counts. Confusing the two is the root cause of most “the job is offset by exactly the width of the bed” complaints.

15.2 Positioning the workpiece instead of the artwork

Beginners move the drawing to match the material. Professionals move the machine to match a known datum. Both are supported; the second is more reliable.

Walkthrough 15.1 — Place a job against a physical corner

1. Put the material in the machine and align it by eye against a bed edge or a jig.
2. In the **Jog** pane, jog the head to the material’s corner and remember the coordinates (the **Position** pane or the status bar shows them).
3. Right-click the scene and choose **Move laser head here** to send the head to a chosen point on the drawing — useful in the other direction, when you want to confirm where a feature will land.

4. Set the artwork's X and Y in the **Position** pane so that its corner sits at the coordinates you measured.
5. Press **Outline** on the Laser tab. If your machine has a pilot laser or a low-power mode, it frames the job on the material; if not, verify the position by jogging the head to two corners of the drawing's bounding box.
6. Only then press **Start**.

The **Move Laser to:** panel does the same job numerically: type coordinates, press the go button, and the head goes there. The nine buttons beside it are stored positions — left-click to travel to one, right-click to store the current position. Storing “front-left of jig”, “hole centre” and “pen holder” once saves minutes on every subsequent job.

15.3 Placements: job start positions

A *placement* is a marker in the design that tells the job where to begin. You can add one from the ribbon (**Job Start** / **tool placement**), and the operations branch can hold **Append absolute placement** and **Append relative placement** commands. Placements are how you say “start burning from this point” rather than “start burning wherever the file begins”, which matters when you are positioning by eye and want the pilot route to be predictable.

15.4 Registration marks

A registration mark is a shape that represents something already on the material. The workflow has three parts:

1. **Model the feature.** Draw the corner, hole or printed box that physically exists, and move it to the Regmarks branch with **Move to regmarks**.
2. **Align the artwork to it.** Position the design relative to the regmark, ideally with the **Alignment** window and a reference object (Chapter 11).
3. **Find the mark on the machine.** Either burn the regmark lightly onto a test piece, or use the machine's pilot laser and **Outline** to frame it — with the material in place, nudge the material until the frame lands where the mark should be.

Regmarks can carry *magnet lines* around their border or centre (**Toggle magnet-lines**), which makes snapping artwork to a modelled physical feature easy. They are saved as a hidden group inside the project file, so an imported project's alignment aids come back with it. Remember that **Toggle visibility of regmarks** exists — half of all “where did my regmark go” questions are answered by that toggle.

15.5 Reading positions off the machine: Place Template

Tools ► Laser-Tools ► Place Template builds geometry from where the head actually is. It has three tabs, and each has **Use position** buttons that capture the laser's current position:

Find center

Three points *A*, *B*, *C*. Jog the head to three points on the circumference of a physical hole, capture each with **Use position**, then **Mark Center** to create a mark at the circle's centre, **Move to center** to send the head there and **Create circle** to draw the hole in the design. This is how you engrave concentric with an existing feature.

Place frame

Corner 1 and *Corner 2* with **Use position** and **Create frame** — model a rectangular region of the material (a panel, a wallet, a jig cut-out) by touching its corners.

Place square

Side A 1, *Side A 2* and *Side B*, plus a *Dimension* field (5 cm by default), *Mark Corner* and **Create square** — build a square from one measured physical edge and a given size.

Like the jog positions, all of these are references rather than movements: they create elements and remarks you can then align to, and they are just as useful for establishing a datum on a machine with no camera at all.

15.6 The camera

If your machine has a camera looking at the bed, MeerK40t can use it as a background layer: a live picture of the material, aligned with the bed coordinates, so that you can position artwork onto what you can see. Camera support needs OpenCV installed (`opencv-python-headless`).

Opening and configuring a camera

Each camera is its own pane, **Camera *n*** (Panes ► Camera). Camera 0 is the usual one. The pane's buttons are:

Control	What it does
Update Image	Take a fresh frame.
Export Snapshot	Save the current image to disk.
Reconnect Camera	Reopen the device — the first thing to try when the picture freezes.
Detect Distortions / Calibration	The guided calibration path (below).
FPS slider	Frame rate; a value of 0 means a frame every five seconds, which is plenty for alignment work and much lighter on the machine.
Correct Fisheye	Applies a fisheye correction, once trained with a checkerboard.
Correct Perspective	Applies a four-corner perspective correction so the picture matches the bed's rectangle.

Right-clicking the camera scene offers the operational menu: background refresh rates (*2x/1sec*, *1x/1sec*, *1x/2sec*, *1x/5sec*, *1x/10sec*, *Disable*), the device link (*Device independent* plus each device label), *Update Background*, *Export Snapshot*, *Reconnect Camera*, *Stop Camera*, *Set camera resolution...*, *Aspect* and *Preserve* submenus, the flips (*Flip horizontal*, *Flip vertical*, *Flip 180°*), the fisheye and perspective corrections with their resets, *Manage URIs* (**URI Manager**), and *Change Label*.

Alignment

Walkthrough 15.2 — Align the camera to the bed

The guided wizard (**Camera Calib**) walks through exactly this list; the version here adds the reasoning.

1. **Home the machine** so the head and bed are in a known state.
2. **Open the camera window** and watch the live picture. If the bed's top-left appears in the wrong corner of the image, use *Flip camera 180°*.

3. **Reset the perspective**, then — with **Correct Perspective** switched *off* — drag the large coloured corner markers (TL, TR, BR, BL) to the four corners of the bed in the picture.
4. **Turn **Correct Perspective** on.** After this, dragging the corners is no longer possible; you have told the program what the bed looks like from here.
5. **Link the camera to your device** (right-click the camera scene and choose the device). Without the link, the picture cannot be placed on the bed's coordinates.
6. **Update the background.** The bed photograph now appears on the main scene, correctly scaled, under your artwork.
7. **Check with corner test marks:** **Add corner test marks** places four red 12 mm squares inset 15 mm from the bed corners. Cut them at low power and see whether the burn lands inside the marks.
8. **Fine-tune with the offset controls** in the *Fine alignment (mm)* group — *Offset X*, *Offset Y* and the *Overlay* arrow buttons, then **Apply offset** . This nudges the picture without changing anything about the machine's own calibration.

Cameras over the network, and their limits

The camera **Connect** box accepts a USB index (0, 1, ...), an RTSP URL (of the form `rtsp://user:pass@host:8554/stream1`), a `user:pass@IP` form, or a bare IP address, which is expanded to `rtsp://IP:8554/profile0`. The **URI Manager** window stores a list of addresses with **Add URI** , **Connect** and a right-click menu (*Remove, Duplicate, Edit, Clear All*).

Three limitations are worth knowing before you invest in camera alignment:

- **Fisheye correction needs a 9×6 checkerboard** printed from the OpenCV pattern and several good captures. If the maths is ill-conditioned the program says so and asks you to keep trying. A cheap lens that cannot be corrected is a lens to avoid.
- **Perspective correction uses exactly four markers** and is frozen once enabled. *Flip 180°* resets the corners, and you must drag them again.
- **The overlay needs the device link.** A camera that is not linked to a device can show you a picture, but it cannot put that picture on the bed.

At the machine 15.1 — A camera is a convenience, not a datum

Camera alignment is typically good to a fraction of a millimetre and drifts with focus, lens height and how the lid sits. Use it for “is my artwork on the wallet?” — not for “will this hole be 0.2 mm from the edge?”. For precision on existing objects, **Place Template** and a jig still win.

Try it 15.1 — Engrave onto an object you did not make

This is the classic camera-and-regmark job: personalising something already manufactured.

1. Measure the object: overall width and height, and the position of one obvious feature relative to its corner.
2. Draw that rectangle in MeerK40t, put it in the Regmarks branch, and treat it as the object.
3. Place your artwork relative to the rectangle — centred, or offset as the product demands.
4. Put the object in the machine. Either link the camera and position it against the bed

image, or jog the head to the object's corner and align the regmark's coordinates to the jogged position numerically.

5. Frame the job with **Outline** on a piece of scrap cardboard in the object's place, and check the landing position.
6. Burn the real object at low power first to check placement, then at full power once you are sure.

Chapter 16

Order, optimisation and simulation

A laser job is a list of moves, and the order of those moves decides how long the job takes, how cleanly it cuts, and whether small parts stay where you put them. This chapter is about the planner — the part of MeerK40t that turns your operations into instructions — and about the two windows that let you inspect the result before it reaches the machine.

16.1 The plan

Between pressing **Start** and the machine moving, MeerK40t runs a fixed sequence of stages. You have already met them in the console output; here is what each one does:

Stage	What happens
<i>copy</i>	The operations are copied into a plan; references to elements are replaced by real copies, so the plan is immune to later edits.
<i>preprocess</i>	Scene coordinates are converted to device coordinates, and validation operations are inserted.
<i>validate</i>	The validation operations run — checking that what follows can actually be executed.
<i>blob</i>	Operations and elements become <i>cutcode</i> : lines, arcs, raster strokes and moves. This is where an operation's settings finally become machine instructions.
<i>preopt</i>	Optimisation commands are injected — inner-first ordering, travel reduction, sequencing.
<i>optimize</i>	Those commands run, reordering the cutcode.
<i>spool</i>	The finished job is wrapped up and handed to the spooler, which feeds the driver.

You can drive the same stages by hand. The **Execute Job** window (Tools ► Execute Job) is a staged wizard whose button changes label as you advance: **Copy**, **Preprocess Operations**, **Validate**, **Convert data**, **Preprocess Optimizations**, **Optimize**, **Spool**. The **Plan** tab of the **Laser-Control** pane shows the plan itself — columns *#*, *Plan*, *Status* and *Content* — with **Update**, **Export** and **Spool**, and a **Hold** checkbox which preserves the plan between runs so you can inspect and rerun it. Right-clicking a plan offers **Clear**, **Update**, **Update (background)**, **Export**, **Spool** and **Details**.

Why you would ever do it by hand

Two reasons. First, diagnosing: if a job behaves oddly, stopping after *blob* shows you the cutcode the plan produced, before optimisation had a chance to hide the problem. Second, iterating: with **Hold** on, a plan can be optimised repeatedly and compared in the simulation window without rebuilding from scratch each time.

16.2 The optimisation settings

Edit ► Settings ► Preferences, or the **Optimize** tab of the laser panel, exposes the planner's settings:

Label	Default	Effect
Reduce Travel Time	on	Reorders burns to shorten the distance travelled with the laser off. The single biggest time saver on most jobs.
Burn Inner First	on	Cuts inner contours before outer ones, so a part is still held by the surrounding material when its own holes are being cut.
Group Inner Burns	off	Completes one part — inner and outer — before starting the next, instead of doing all inner cuts first. Slower, but safer on sheets where parts could shift.
Merge Passes	off	Optimises the passes of an operation together rather than pass by pass.
Merge Operations	off	Optimises across operations instead of within each.
Burn Complete Subpaths	on	Finishes each open subpath in one go.
Combine path segments	off	Stitches nearly-touching path ends so they burn as one continuous move. With <i>Tolerance</i> (default 0).
Cluster raster objects	on	Groups rasters that do not overlap into separate optimisation groups, with a <i>Margin</i> (default 1mm).
Keep effect lines together	on	Treats hatch lines as one shape, so they are burned together rather than visited one by one.
Optimize internally	off	Optimises the lines inside a hatch individually.
Reduce polyline details	off	Thins redundant points, with a <i>Level</i> (<i>Minimal</i> ... <i>Coarse</i>). A fix for paths with absurd numbers of nodes.

Two further settings have no user interface but exist and can be set from the console: `opt_nearest_neighbor` (on — the travel algorithm used by *Reduce Travel Time*), `opt_2opt` (off — a slower two-opt refinement), `opt_start_from_position` (off — begin from the head's current position) and `opt_closed_distance` (15 — the distance at which path ends are considered closed). Which strategy runs depends on the combination chosen: with *Burn Inner First* on, the inner-first optimiser handles everything; otherwise travel reduction runs if it is enabled, and a basic sequencing pass is the fallback.

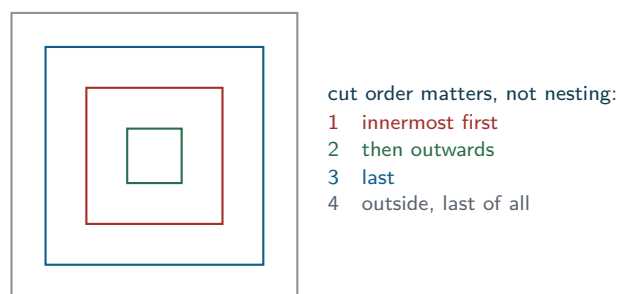


Figure 16.1: Why inner-first is the default. If the outer contour is cut first, the part is held by nothing but friction while its interior is being cut — and on thin material, the vibration is enough to move it. Burning from the inside out keeps the surrounding material as a clamp until the very last cut.

A sensible default set for most sheet work

Reduce Travel Time on, Burn Inner First on, Group Inner Burns off, Burn Complete Subpaths on, Keep effect lines together on. Build the plan, simulate it, then change one setting and compare. Optimisation is one of the few places where measurement beats intuition, because the numbers are right there in the simulation window.

16.3 The Simulation window

Simulate on the Laser tab (right-clicking it simulates without optimisation) opens the **Simulation** window: a replay of the job, drawn on the scene.

Control	What it does
Play button	Start or stop the replay.
<i>Playback Speed</i>	Replay speed, with a percentage readout.
<i>Mode</i>	Whether the replay advances in <i>Steps</i> , <i>Time (sec.)</i> or <i>Time (min)</i> — so a four-hour job is watchable.
Send to Laser	Spools the currently displayed plan. This is the shortcut from “looks right” to “running”.
Recalculate / Stop calculating	Rebuild the plan, and stop a long rebuild. During the rebuild the button reads <i>Calculating</i> and the window says the plan is still being processed.
Optimize	Whether the rebuild includes the optimisation stages.

The estimates are the most useful numbers the program produces:

- *Laser Time* and *Laser Distance* — beam on.
- *Travel Time* and *Travel Distance* — beam off.
- *Total Time* and *Total Distance* — the sum.
- *Extra Time* — allowances for acceleration, delays and the machine’s own overheads, and usually the reason reality is longer than the simulation.

Right-click on the simulation scene for the display options: *Delete cuts before*, *Delete cuts after* (trim the plan to a point in the replay), *Show Background*, the three grid toggles, *Show physical dimensions*, *Show travel path* (draw the jump moves — the fastest way to see whether optimisation did anything sensible), *Raster as Image*, *Simulate laser width*, and the zoom commands. The embedded operations list in the window shows the plan’s operations; the *D* checkbox *decompiles* the cutplan — making the operations visible and editable again — and the text field beside it lets you modify an operation parameter and press **Enter** to see the effect on the simulation.

Watch the travel path, not the burn

The laser path is what you expect to see; the travel path is where the time goes. Turn on *Show travel path* and look for long diagonal jumps between things that are physically next to each other. Each one is a missing optimisation — or artwork that needs stitching (stitch 0.1mm).

16.4 The spooler and running many jobs

Spooler (Tools ► Spooler or the ribbon’s **Spooler** button; the console form is **window toggle JobSpooler**, or **pane toggle spooler** to work on the pane itself) is a docked pane, not a window

of its own, and it is the queue between planning and the machine.

- The top row chooses the device and offers **Pause / Resume** , **Abort** and a *Silent* checkbox.
- The queue lists each job with its #, *Device*, *Name*, *Items*, *Status*, *Type*, *Steps*, *Passes*, *Priority*, *Runtime* and *Estimate*. Right-click a job for **Stop** (while running) or **Remove** , **Disable / Enable** , *Finish after this loop* and *Clear All*.
- Below the queue, *Completed jobs*: is the history, with columns for device, name, start, end, runtime, estimate, steps, passes, status and job information, and a **Clear History** button. Right-clicking the history offers *Delete this entry*, *Delete...*, *All entries*, *Older than n week*, *All incomplete jobs*, *Clear history on startup*, *Ignore helper jobs* and *Write CSV* — which writes `history.csv` into the program's working directory, and is how you keep a machine log without typing.

The **Launch Spooler on Job Start** preference (on by default) brings the spooler up automatically whenever a job starts, which is a good default: the queue is where you see what is about to happen.

At the machine 16.1 — Judge the order by the sound, then by the result

The simulation tells you what the planner thinks; the machine tells you the truth. Listen for the head crossing the sheet repeatedly, and look at the first finished part: if small pieces have shifted, you cut outer contours too early, and **Group Inner Burns** or a reordered operation list is the fix. If the edges are charred on the last cut only, the part moved because it was no longer supported — a tab (**Tab Edit**) solves that more reliably than any optimiser.

Try it 16.1 — Optimise a real job, with numbers

1. Load a job with a mixture of cutting and engraving, or build one from a duplicated part.
2. Simulate it with optimisation *off*. Write down *Total Time*, *Travel Time* and *Total Distance*.
3. Turn optimisation on, press **Recalculate** , and write the numbers down again. The difference is what the optimiser earned you.
4. Turn on *Show travel path* and look for the remaining long jumps.
5. In the console, run `stitch 0.1mm` on the artwork and recalculate. Note the change in total distance.
6. Try **Group Inner Burns** on and compare both the numbers and a physical test cut. Decide for your material which one holds the parts better.
7. Keep the winning combination as your default in the preferences, and record the numbers with your material notes.

Chapter 17

Rotary, coolant and remote control

Three accessories change how a laser is used rather than what it can burn: a rotary chuck turns cylinders into engravable objects, coolant and air assist keep the cut alive, and a network connection lets a job be started from somewhere else. This chapter covers all three, plus the remote-control servers that make MeerK40t scriptable from a browser or another machine.

17.1 Rotary: engraving on cylinders

A rotary attachment replaces one axis with a chuck that rotates the workpiece. The coordination problem is the same on every machine: the circumference of the object must be mapped onto the axis that used to be the bed's height.

Walkthrough 17.1 — Rotary, step by step

1. Open the rotary settings: Tools ► Rotary (device settings), or the **Rotary** button on the device's ribbon.
2. Tick **Rotary-Mode active**, then — before anything else — set **Outside diameter (mm)** and **Usable length (mm)**. The fields below them depend on these two values.
3. Leave **Auto Y wrap from diameter** on for your first run. The Y axis is then scaled so that the bed height corresponds to the object's circumference ($\pi \times \text{diameter}$).
4. Select your artwork on the scene and click **Fit selection to rotary**. It scales the selection uniformly to fit usable length $\times$ circumference.
5. Burn a test strip on a piece of scrap tube before committing to the real object. Check the join line where the engraving wraps around: if the pattern does not meet cleanly, the circumference is wrong — measure the object rather than trusting a stated size.
6. With rotary mode active, the Home command is suppressed by default (**suppress_home**), so the machine does not drive the head to a home position that may be occupied by the chuck.

Setting	Default	Meaning
rotary_active	off	Master switch for rotary mode.
rotary_diameter_mm	80.0	Outside diameter; πD becomes the wrap height.
rotary_length_mm	200.0	Usable length along the axis.
rotary_auto_wrap_scale	on	Derive the Y scale from the diameter rather than using a manual factor.
rotary_auto_length_scale	off	Derive the X scale from the usable length.
rotary_scale_x, rotary_scale_y	1.0	Manual scales, used when the automatic ones are off.
rotary_flat_y_steps	159.6	The flat-bed steps/mm to return to afterwards (GRBL).
rotary_y_steps	80.0	Steps/mm of the rotary axis (GRBL).
rotary_steps_compensate	on	Compensate in software, leaving the controller's own settings untouched (GRBL).
rotary_firmware_steps	off	Write the rotary steps into the controller at job start and restore them afterwards (GRBL).
rotary_cal_test_mm	100.0	Length of the calibration test burn.
suppress_home	on	Skip the home command while rotary mode is active.
rotary_home_x_only	on	Home only X, never Y, with a chuck fitted (GRBL).
rotary_flip_x, rotary_flip_y	off	Mirror the axes.

The console side is short and worth knowing, because it is what a repeatable rotary process ends up being driven by:

```
rotary          # show status: active, diameter, circumference, scales
rotaryfit       # scale emphasised elements to length x circumference
rotaryscale     # scale by the effective scales
rotarysuggest 80 200 8 1 # estimate steps/mm: diameter, motor steps, microsteps, ratio
rotarycal 314.2  # recalibrate from a measured test burn
```

Cylinder correction (galvo machines)

Galvo markers with a JCZ controller can also engrave curved surfaces without rotating them, by projecting the design onto the cylinder. This is a *different* feature, and it applies to Balor devices only:

```
cylinder axis X 50mm # the cylinder's diameter, and which axis runs around it
cylinder distance 120mm # distance from the cylinder to the mirror
cylinder on
```

The matching settings are `cylinder_active`, `cylinder_mirror_distance` (100 mm by default), `cylinder_x_axis` / `cylinder_y_axis` and the diameters `cylinder_x_diameter` / `cylinder_y_diameter` (50 mm). Curved-surface correction keeps straight lines straight when the surface falls away from the lens — which, for a small tube entirely inside the galvo field, is often better than rotating it.

17.2 Coolant, air assist and sleep

Coolant and air assist

Each device has a **Coolant** setting that names how the machine’s coolant or air assist is switched. The built-in methods are:

- *Warnmessage* — shows a dialog asking you to switch the air assist on or off at the right moment;
- *GCode M7/M9* and *GCode M8/M9* — for GRBL controllers with a controllable output.

Where you use it:

- Per operation, with the *Coolant* field on the operation’s properties: *No changes*, *Turn on* or *Turn off*. Put “turn on” on your first operation and “turn off” on your last.
- In the operations tree, with **Append Coolant On** and **Append Coolant Off** as explicit steps in the job.
- From the ribbon with the **Coolant** button, which is only visible when the device has a coolant configured.

The console commands are `coolants` (list the available methods), `coolant\_on/coolant\_off`, and `coolant\_on\_by\_id <id>/coolant\_off\_by\_id <id>` for choosing a method explicitly.

Preventing the computer from sleeping

A long job that outlives the screen saver is a job that can be interrupted. `system\_hibernate prevent` stops the machine from sleeping; `system\_hibernate allow` releases it. On Linux this masks the sleep, suspend, hibernate and hybrid-sleep targets through `systemctl` — which is why the program mentions that administrative privileges may be required — and on Windows it uses the standard execution-state API. The device configuration’s *Default Actions* can bind *Hibernation off* and *Hibernation on* to the start and end of a job.

17.3 Remote control over the network

Network starts and stops MeerK40t’s own servers. Each one speaks a different language to a different kind of client:

Server	Default port	What it is for
Webserver	8080 (menu), 2080 (console command default)	A small web console: device status, the job queue with pause/stop, a command box and the last console output as HTML. It listens on localhost only.
Telnet (<i>consoleserver</i>)	23	The full MeerK40t console over telnet. Every command you can type locally, you can type from another machine.
GRBL-Server (<i>grblcontrol</i>)	23	Emulates a GRBL controller, so software that expects to talk to GRBL (or a sender like a g-code streaming tool) can drive your machine through MeerK40t. <code>grblmock</code> is a trivial mock version for testing clients.
Ruida-Server (<i>ruidacontrol</i>)	UDP 50200, jog 50207	Emulates a Ruida controller — the bridge that lets Ruida-speaking software drive a K40. It can also work as a man in the middle (40200/40207) or as a LightBurn-to-Ruida bridge on TCP 5005.

Start them from the menu, or from the console, which is how you script them:

```
webserver -p 8080      # start the web console on a chosen port
```

```
consoleserver -p 2323      # a second telnet console, no privilege needed
grblcontrol -p 23         # GRBL emulator, spooling to the active device
ruidacontrol              # Ruida emulation (UDP 50200/50207)
```

The web console accepts commands as `?cmd=<command>` in a GET or POST request, and job control as `job\_cmd=<index>:<operation>` where the operations are `stop\_job`, `remove\_job`, `pause\_device` and `resume\_device`. That is enough to build a simple shop-floor dashboard: pause every machine, see what is running, and start the next job, from a browser.

Remote control is real control

A telnet console or an emulated controller accepts commands that move the head, fire the laser and run jobs. The web server binds to localhost by default for exactly this reason — do not expose these ports to a network you do not control, and never leave a server running on a machine that is also reachable from the internet.

A networked K40

Lihuiyu devices can also be driven over the network: in the device's **Interface** tab, choose **Networked** and set the *Address* and *Port* of a machine running MeerK40t as a server (the default ports are 1025 on Linux/macOS and 1022 on Windows). The **Network-Controller** window then replaces the USB controller window. This is the standard way to put the noisy machine in one room and the computer in another.

GRBL over Wi-Fi: ESP3D

Diode machines with an ESP32-based Wi-Fi module — MKS DLC32 boards are the common ones — can be driven without a cable. Enable it in the GRBL configuration's **ESP3D Upload** tab: **Enable ESP3D Upload**, then **ESP3D Host** (default 192.168.1.100), **ESP3D Port** (80), **ESP3D SD Path** (/) and optional credentials, with **Test Connection** and **List Files** showing the SD card's usage. The ribbon then gains **ESP3D Upload+Run** (right-click to upload without starting), **ESP3D Pause**, **ESP3D Resume**, **ESP3D Stop** and **ESP3D Files**. **Prepare Plan exports for SD card** adjusts exported files for the MKS DLC32 (LF line endings, M3 laser mode, no G28), and **Cleanup Local Files** deletes the local copy after upload. The console equivalents are `esp3d\_config`, `test|set <param> <value>` and `esp3d\_list`.

17.4 Connecting to other software

Two bridges are worth knowing about even if you never use them:

- **GRBL-Server** lets a phone app or a g-code sender think it is talking to a normal GRBL machine while MeerK40t does the work — useful when you want someone else's jogging interface or a CAM program's streamer.
- **Ruida-Server** does the same trick in the other direction: software that only knows Ruida can drive a K40, because MeerK40t emulates the controller the software expects.

Try it 17.1 — Drive a job from a browser

1. Start the web server: **Network ► Start Webserver**, or **webserver** in the console.
2. Open a browser at `http://127.0.0.1:8080`. You should see the web console with the active device, the job list and a command box.
3. Queue a small job from the desktop interface and watch it appear in the web page's job

- list.
4. Use the web page to pause and resume the device. The desktop interface should follow along — they are two views of one program.
 5. Stop the server from the **Network** menu, and confirm in the console that the port is closed.
 6. If you use a shop network, repeat with `consoleserver` and a real second machine — then decide, deliberately, whether that convenience is worth the risk.

Chapter 18

The console, and scripting

Every button in MeerK40t is a front end for a console command. That fact is what makes the program scriptable, reproducible and — once you are comfortable — much faster to use. This chapter teaches the console as a tool rather than as a reference; the complete command tables are in Appendix B.

18.1 Opening the console and reading it

- Open the **Console** pane with the ribbon's **Console** button, or from **Panes ► Console**. (Many people bind **F12** to window open Console in the Keymap window; it is not bound by default.)
- Type a command and press **Enter**. Output appears above the input line, coloured, with clickable links when a message contains a URL.
- Up and Down arrows walk your command history, which is kept between sessions — the last fifty commands are stored in `cmdhistory.log` in the program's working directory.
- **Tab** completes command names: type `sca` then **Tab** and the console offers the candidates.
- `cls` or `clear` empties the output; `console\_font` changes the typeface, and `console\_font -r` resets it.

The console is also the quiet witness of everything the program does. When a job is planned, the stages go past in the console; when a device disconnects, the reason is very often printed here first. If you report a problem, the console text is the first thing anyone will ask for.

Record a session

`channel save console` writes everything printed to the console channel into a timestamped file — the polite way to hand a support question to someone else. `channel list` shows which channels exist, and `channel open <name>` watches an individual one (device traffic, for example).

18.2 Asking the console for help

```
help                # every command, grouped by the kind of data it acts on
help rect           # one command, with its arguments and options
?                  # same as help
find rotary         # every command and option containing "rotary"
?? rotary          # same as find
```

`help` with no argument is the real index: commands are grouped by category, and the groupings tell you how the console is structured — some commands create data, some act on selected

elements, some act on a plan.

18.3 The syntax rules that matter

Rule	Example and effect
Units are written with the number	<code>rect 2cm 2cm 1cm 1cm</code> makes a 1 cm square 2 cm in from the origin. <code>mm</code> , <code>cm</code> , <code>inch</code> , <code>mil</code> , <code>deg</code> and percentages all work.
One command per line	A newline ends a command. Inside <code>-e "..."</code> on the command line, a newline separates commands too.
separates commands on one line	<code>rect 0 0 1cm 1cm cut</code> runs both. A inside quotes is not a separator.
A leading <code>.</code> silences the echo	<code>.timer 1 0 spool jog</code> runs without printing the command itself — useful in scripts that would otherwise flood the console.
Pipelines pass data along	A command that produces elements hands them to the next command that accepts elements. <code>rect 0 0 1cm 1cm engrave -s 15</code> creates a rectangle and immediately groups it into an engrave operation.
<code>;</code> belongs to aliases and stored batches	It is not a command separator in ordinary use.

18.4 The job pipeline in one line

The most useful sentence in MehrK40t's console is this:

```
planz clear copy preprocess validate blob preopt optimize spool
```

It builds a plan (`planz`), clears it, copies the enabled operations in, preprocesses and validates them, converts them to cutcode (`blob`), prepares and runs the optimisers, and sends the result to the spooler. It is exactly what the **Start** button does. There are two built-in aliases for close relatives of it:

```
burn      # the chain above
simulate  # the same without spooling, then opens the Simulation window
```

Substitute `save_job <file>` for `spool` and the same chain writes a device job file instead of running the machine:

```
plan0 clear copy preprocess validate blob preopt optimize save_job /tmp/tag.egv
```

`plan` with no name means the default plan, `plan0`; `planz` addresses the plan called `z`. Naming plans is how you keep several jobs in flight at once — the Laser panel's **Plan** tab is a view of the same objects.

18.5 A complete example, typed

```
rect 20mm 20mm 60mm 30mm      # the tag outline
circle 26mm 33mm 2mm          # a hanging hole (radius form)
select 0                      # nothing selected: element 0 is the rect
cut -s 8 -p 70                # cut it: 8 mm/s, 70% power
select all
engrave -s 30 -p 25            # engrave everything else
planz clear copy preprocess validate blob preopt optimize spool
```

Note the shape of it: create, select, group into an operation with its settings, then run the standard pipeline. Once you can read that, you can write it.

18.6 Settings from the console

`set` is the single entry point for scalar settings. With no arguments it lists them; with a key and value it assigns:

```
set                # list the variables in the current context
set bedwidth 400mm # set a device value
set -p balor rotary_active True # set it in the device's own context
set -p / rotary_diameter_mm 80 # explicit path form
flush              # write settings to disk now
setting_export /home/me/backup # copy the whole configuration out
setting_import /home/me/backup/meerk40t_2026-09-17_10_00_00.cfg
```

There is no general-purpose `get`: to read a value, list the context with `set` and read the line you want. Device settings are covered in Appendix C; the important habit is to use `set -p <device>` rather than changing values globally.

18.7 Selecting and acting from the console

Element commands act on the current selection, and the console gives you several ways to build one:

```
element            # the currently selected elements
element*           # all elements
element~           # everything that is not selected
element0,3,4       # specific elements by index
element* select    # select everything (select+ adds, select- removes, select^ toggles)
range 2 8          # indices 2 to 8
filter speed>=10   # operations matching an expression
tree selected delete
```

This is worth playing with rather than reading: create a few shapes and watch the different selectors change what a following `translate 5mm 0` affects.

18.8 Aliases: your own commands

An alias is a stored command line, and the console ships with some useful ones:

```
alias              # list the aliases
burn               # alias: the full plan-and-spool chain
simulate           # alias: plan and open the simulation
alias myburn cut -s 6 -p 80 ; planz clear copy preprocess validate blob preopt optimize spool
```

Note that `;` separates the individual commands *inside* an alias body — that is what the semicolon is for. Default aliases can be reset if you break them (`Help` or the `bind/alias` commands), and keyboard bindings ultimately call these same console commands, which is why the Keymap window can bind a key to anything you can type.

18.9 Scripts: files, batches and the CLI

There are three ways to run a list of commands without typing them, and they are appropriate in different places.

A command file

`execute /home/me/scripts/tag.txt` runs each line of the file as a console command. Lines beginning with `\#` are comments. This is the general-purpose scripting route.

The command line

`-e "command"` may be repeated, and `-b <file>` runs a file of commands at startup:

```
meerk40t -z -e "load /home/me/tag.svg" \
-e "planz clear copy preprocess validate blob preopt optimize spool" -a -q
```

That invocation loads a design, builds the job, starts the laser and quits when the spooler finishes — headless, from a script or a scheduler.

The Auto-Exec window

Tools ► Auto-Exec (titled *File startup commands*) stores commands inside the project file itself, and runs them when that file is loaded, if **Execute on load** is ticked. The **Execute commands** button runs them immediately. Its small ***** menu offers ready-made entries such as *Start simulation*, *Burn content on current device* and **device activate** for each of your devices. This is the right place for the three lines a particular project always needs.

Useful meta-commands for scripts: `wait <seconds>` pauses, `loop <from> <to> <command>` repeats, `threaded <command>` runs something in the background, and `timer <times> <duration> <command>` repeats on a schedule.

18.10 Batch production from a spreadsheet

The **CSV Batch Run** window (Edit ► Settings ► CSV Batch Run) is a production tool: it reads a CSV file, exposes the columns as variables, and can burn a job once per row.

Walkthrough 18.1 — Numbered tags for a list of names

1. Prepare a CSV with a header row, for example **name,serial**, and one row per item.
2. Design the job with text elements that use those variables — `{name}` and `{serial}` — exactly as wordlists do (Chapter 13).
3. Open the window and choose the *File*. Press **Import**.
4. Set *First row* to *Auto*, *Data* or *Headers* depending on your file, and walk the rows with **Prev**, **Next** and **Go to**; the header shows *Row x of y*.
5. Check the two preview panes: *Preview (text with variables)* shows what the design will say, and *Current row values* shows what the variables currently hold.
6. Press **Apply row to design** to see one row in place without burning.
7. Press **Run current row** to burn just that row, or **Run all rows** to work through the file — with *After each job completes, run the next row automatically* controlling whether it waits for you between jobs.
8. **Stop batch** stops the run; it does not unload the file, so you can resume where you left off.

The window builds each job with the same plan chain discussed above (with **threaded** in front when appropriate), which is why it behaves exactly like pressing **Start** — including honouring your optimisation settings.

18.11 Driving the interface from the console

Two more command families are worth knowing because they turn repetitive setup into one line:

```

window toggle Console          # open or close a window by name
window open Simulation 0 1 1   # open Simulation for plan 0 with flags
window reset *                 # forget all stored window positions
pane show console              # show a pane
pane hide tree ; pane float jog # ... and the rest of the pane controls
page modify                    # switch the ribbon page
arm ; startjob                 # arm and start (same as the Ctrl+Alt+Shift+A binding)

```

Try it 18.1 — Script a repeatable production run

1. Load the project you want to repeat and confirm it runs normally by pressing **Start**.
2. In the console, type `planz clear copy preprocess validate blob preopt optimize spool` and watch the stages go past. Nothing should differ from the button.
3. Write those commands into a file called `run-tag.txt`, one per line, with a comment line at the top explaining what it is for.
4. Test it with `execute /full/path/run-tag.txt`.
5. Put the same commands into the Auto-Exec window and tick **Execute on load**. Now loading the project prepares the job for you.
6. Finally, try the headless form from a terminal: `meerk40t -z -e "load /full/path/tag.svg" -e "burn" -a -q`. This is the same job with no windows at all.
7. Save the command file next to the project. In six months, it will be the only part of this exercise you still need.

Chapter 19

Preferences, housekeeping and troubleshooting

This chapter covers the settings that shape the program itself — as opposed to the job — and then the practical business of fixing things when they go wrong: where your data lives, what to keep, and what to do when something misbehaves.

19.1 The Preferences window

Open it with `Ctrl+ ,`, `Edit ▶ Settings ▶ Preferences`, or the `Preferences` button. It is a notebook, and its tabs each collect settings from a different part of the program:

Tab	What is in it
General	Units and language, plus the settings for <i>Start</i> , <i>Options</i> , <i>Laser</i> , <i>Optimisations</i> and <i>Laser Settings</i> . At the bottom, a saving panel with <code>Save</code> , <code>Export</code> and <code>Import</code> .
Input/Output	How files are read and written: pixels per inch, <i>Images</i> , <i>SVG-Features</i> , <i>DXF</i> and the I/O page.
Classification	Colour-based assignment of elements to operations, with presets <code>Automatic</code> and <code>Manual</code> .
Operations	Defaults applied when operations are created.
GUI	Behaviour of the interface itself — the table below.
Scene	Element display, selection behaviour and snapping.
Colors	Every colour used on the scene, with <code>Reset Colors to Default</code> and <code>Reset Colors to brighter defaults</code> (for dark themes), plus the object-label colour.
Ribbon	The ribbon editor: pages, panels and buttons.
Coordinate Space	Visible only with <code>developer_mode</code> on; shows the coordinate transforms.

The settings you are most likely to want, with their defaults:

Label	Default	Effect
UI-Colours	System	Force <i>Dark</i> or <i>Light</i> regardless of the operating system's setting.
Single Instance	on	Allow only one MeerK40t at a time. Turn it off when you genuinely want two devices driven from two windows.
Soundfile	(system)	The sound played by beep — the end-of-job alarm for machines you cannot see.
Autosave workspace	on	Periodic save to <code>_autosave.svg</code> . See the warning below.
Frequency	5 minutes	How often the autosave runs (30 s to 10 min).
Optimize element display	on	Do not draw invisible elements.
Simplify Effects	on	Limit effect complexity on screen; CapsLock temporarily disables it.
Remember main window position	on	Reopen where you left it.
Save Window Positions	on	Reopen windows where you left them.
Launch Spooler on Job Start	on	Bring up the spooler when a job starts.
MouseWheel Pan	off	Wheel pans instead of zooming; Ctrl + wheel does the other.
Invert MouseWheel Pan / Invert MouseWheel Zoom	off	Reverse the directions if they feel wrong.
ToolTip delay	100 ms	How quickly tooltips appear.
Ribbon: hover before description	2000 ms	How long to hover a ribbon button before its long description appears.
Long UI descriptions	on	Show extended help in ribbons and context menus.
Level	0	Warning severity: 1 all warnings, 2 normal + critical, 3 critical only, 4 ignore all.
Enable Laser Arm	off	Require the laser to be armed (Arm) before it will start a job.
Track changes and allow undo	on	Undo system on or off (needs a restart).
Levels of Undo-States	20	How many undo steps to keep (3–250).

Autosave is not a recovery system

MeerK40t writes `\_autosave.svg` every few minutes, but it does not offer to reopen it after a crash or a bad exit. If something goes wrong, the file is there — you have to open it yourself. Do not treat it as a substitute for **Ctrl+S** .

19.2 Appearance, language and updates

- **Themes:** **UI-Colours** switches between the system setting, dark and light. The scene has its own palette in **Colors** ; if a dark theme makes the bed or the grid hard to see, the single-click fix is **Reset Colors to brighter defaults** .
- **Language:** **Languages** switches the interface language. Changing it requires a restart, and the dialog says so. The bundled set includes English, German, Spanish, French, Hungarian, Italian, Japanese, Dutch, Polish, both Portuguese variants, Russian, Turkish and Chinese.
- **Updates:** the updater's settings live in the *Options* page: *Action* (*No, thank you, Look for major releases, Look for major+beta releases*) and *Frequency* (*At every startup, Once per day, Once per week*). The same check can be run on demand from **Help ► Check for Updates**, or with `check\_for\_updates` in the console; the result appears in a window titled

Update-Info. If you use beta releases, expect beta problems.

- **About** (Help ► About MeerK40t) has four tabs: *About*, *In Memory of David Olsen*, *System-Information* and *MeerK40t-Components*. The last two list the versions of Python, wxPython, numpy, Pillow, potrace, vtracer, ezdxf, pyusb, pyserial, OpenCV, clipper and numba that are actually loaded — exactly what to copy into a bug report.
- **Tips** (Help ► Tips && Tricks) shows a tip with optional image, [Previous tip](#), [Next tip](#) and [Try it out](#) (which loads an example, so be careful: it can change your design). [Show tips at startup](#) and [Automatically Update](#) control whether you see them and whether they are refreshed from the project wiki.

19.3 Where your data actually lives

Knowing this list turns several classes of problem into a two-minute fix.

File	Where and why
MeerK40t.cfg	The configuration, in the per-user configuration directory (<code>~/.config/MeerK40t/</code> on Linux, <code>~\Library\Application Support/MeerK40t/</code> on macOS, <code>~\LOCALAPPDATA\MeerK40t\</code> on Windows). Settings, devices, window layout, recent files.
MeerK40t.bak, .ba1...ba5	The five previous versions of that file, newest first.
_autosave.svg	The periodic workspace save, in the program's working directory.
cmdhistory.log	Your console command history (last fifty commands).
MeerK40t-\emph{timestamp}.txt	Crash logs: written to the current working directory, or to the configuration directory if that is not writable.
history.csv	Job history exported from the spooler with <i>Write CSV</i> .
tips.txt	The local cache of tips shown by the Tips window.

The console also writes useful things on demand: `channel save console` records the console output to a timestamped file, `setting\_export <dir>` copies your whole configuration out (and `setting\_import <file>` reads it back), and the [Export](#) button in the preferences does the same thing from the GUI.

19.4 Troubleshooting by symptom

The window is broken, a pane is missing, or a dialog is off-screen

1. Panes ► Reset Panes restores the default layout.
2. Tools ► Reset Windows forgets stored window positions; the console version is `window reset *`.
3. Still wrong? Close the program, move `MeerK40t.cfg` aside, and start again. You lose the layout, not your projects.

Nothing happens when I press Start

The program is usually refusing on purpose, and it says why:

- “*Nothing to burn. Operations need assigned shapes...*” — there is no artwork attached to any enabled operation.
- “*Some shapes are not assigned to any operation... Queue anyway?*” — some artwork will be skipped. Fix it with Edit ► Select all non-assigned and the operation squares.

- “*Connect your laser device first*” — the device is not talking to the machine; Chapter 4 has the checklist.
- **Enable Laser Arm** is on and the laser is not armed — press **Arm**, or turn the preference off.
- Nothing at all, not even a dialog: check the **Console** pane. The refusal is printed there.

A job stops in the middle, or the head goes somewhere unexpected

1. Look at the **Console** for the last device message; the driver’s **Controller** window carries the raw traffic and any alarm.
2. An alarm (GRBL especially) usually means the machine hit a soft limit: check the bed size, the flips and the declared home corner (Chapter 4) before assuming a mechanical fault.
3. If the job is still queued, the **Spooler** shows its status; a job can be paused, resumed, disabled or removed there.
4. If the machine stopped but the program thinks it is running, use **Abort** in the spooler and re-home before continuing.

The burn is wrong

Almost always one of four things, in this order of likelihood: focus is off; the material is not what you think it is; the speed/power pair has not been verified on this machine; or the raster settings (DPI, overscan, direction) are wrong for the machine. Chapter 8 is the systematic answer, and Chapter 6 covers raster-specific faults.

The program is slow

- Huge imported files: **Remove transparent objects**, **Break Subpaths**, and **Reduce polyline details** in the optimisation settings. A drawing with 40 000 nodes will never feel quick.
- Display: turn off *Instant Snap Display*, animation and image caching toggles in **View ► Scene Appearance**, and *Simplify Effects* (or hold **CapsLock**).
- Raster jobs: lower the DPI. Time scales linearly with it.
- Still slow: run once with **-f profile.txt** and send the result with a bug report — it is a Python profile of the session, and it points at the actual bottleneck.

Something crashed

MeerK40t installs a crash handler. On a crash it:

1. writes the traceback — including a summary of the local variables at each frame — to **MeerK40t-`{timestamp}.txt`** in the working directory (falling back to the configuration directory if that is not writable), and to the terminal’s standard error;
2. shows a dialog titled *Crash Detected! Send Log?* offering to send those details to the developers. The dialog is explicit that only the crash details are sent — no project data, no personal information — and **Cancel** aborts the program rather than continuing.

The crash log is the most useful thing you can attach to a report, and it survives the crash: find it, read the first traceback, and note the version from the title bar or **meerK40t --version** (the suffix, **git**, **src** or **pkg**, tells the developers how your copy was built). More on reporting in Appendix D.

Permissions, sudo and USB

If MeerK40t only sees the laser when run as root or with **sudo**, install the udev rules from Chapter 4 and add your user to the **dialout** group. Running the program with elevated privileges

is not recommended, and the program warns about it — the warning can be silenced, but the better fix is the rules.

Two copies of the program

With **Single Instance** on, launching a second copy brings the first one forward. If you really do want two, or if the check is getting in your way on a shared machine, turn it off in the **General** tab.

Try it 19.1 — Make your setup survivable

Twenty minutes now, an afternoon saved later.

1. In the console, run `setting\_export /home/me/meerk40t-backup` and confirm the timestamped `.cfg` file is there. Do it again after any significant device or material work.
2. Note your configuration file's location from the table above, and confirm the `.bak` files beside it.
3. Set *Levels of Undo-States* to something larger than the default if you do exploratory editing, and consider whether *Autosave* interval of five minutes suits your machine.
4. Turn on *Enable Laser Arm* if anyone else ever uses this machine; leave it off if you are the only operator and find it annoying.
5. Deliberately break the layout — hide panes, float windows, move everything — then fix it with **Panes ► Reset Panes** and **Tools ► Reset Windows**.
6. Open a project, make a change, and confirm **Save Window Positions** and *Remember main window position* bring back your workspace on the next start.

Chapter A

Keyboard shortcuts and mouse controls

Read this appendix when you already know what you want to do and only need the keystroke for it, or when a key that is described somewhere else does not appear to work. Chapter 3 describes the interface these keys drive, and the commands behind them are documented in Appendix B.

Every keyboard shortcut in MeerK40t is a console command (Chapter 18). The keymap is data, not code: the “nudge the selection 1 mm right” instruction that **Right** sends is the same `translate 1mm 0` you could type by hand. That is why shortcuts can be moved to other keys, and why a key with no shortcut can be given one.

Shortcuts are edited in the **Keymap** window (Edit ► Settings ► Key-Bindings, or window open **Keymap** in the console). It lists one row per binding, a **Key** and the **Command** it runs, and applies every change at once. A command beginning with `+` runs while the key is held down and is cancelled when it is released; all other commands run once per press.

Remapping and restoring shortcuts

1. Open Edit ► Settings ► Key-Bindings.
2. Click into the left-hand field and press the key combination you want to use. What you press is what is stored.
3. Type the console command into the right-hand field, for example `outline 1mm`.
4. Press **Add Hotkey**. A binding that already exists for that key is replaced.
5. To remove bindings, select one or more rows, right-click and choose **Remove ...** (or **Remove N entries**).
6. To start again: **Standard** ► **Reset Keymap to defaults** in the same window restores every default; the console equivalents are `bind default` for the keys and `alias default` for the held-key timers.

Bindings are stored in your settings, so they survive a restart. The **Keymap** window lists key names in the spelling the keymap stores them in, for example `ctrl+shift+right`; on macOS it shows **Cmd** where the map stores *ctrl*.

A.1 Movement and nudging

Two different things move here: the arrow keys move *selected elements*, while the held keys **A**, **D**, **W** and **S** jog the *laser head*. Nothing selected (or nothing connected), nothing moves.

Key	Command	What happens
Right , Left , Up , Down	<code>translate 1mm 0</code> and the <code>-1mm 0</code> , <code>0 1mm</code> , <code>0 -1mm</code> companions	The selected elements move 1 mm in that direction, once per press.
Shift+Right and the other arrows	<code>translate 10mm 0</code> and companions	The same move in 10 mm steps.
Ctrl+Shift+Right and the other arrows	<code>translate 0.1mm 0</code> and companions	Fine positioning: 0.1 mm per press.
Alt+Right and the other arrows	<code>translate 1spx 0</code> and companions	One scene pixel (<i>spx</i>): the smallest step, independent of the unit in use.
Alt+Shift+Right and the other arrows	<code>translate 10spx 0</code> and companions	Ten scene pixels.
A , D , W , S	<code>+left</code> , <code>+right</code> , <code>+up</code> , <code>+down</code>	Hold to jog the laser head 1 mm every 0.1 s in that direction; releasing the key stops it. Device required.
Numpad Left , Numpad Right , Numpad Up , Numpad Down	<code>+translate_left</code> , <code>+translate_right</code> , <code>+translate_up</code> , <code>+translate_down</code>	Hold to move the selection 1 mm every 0.1 s.
Numpad * , Numpad /	<code>+scale_up</code> / <code>+scale_down</code>	Hold to grow or shrink the selection by 2 % every 0.1 s, about its own centre.
Numpad + , Numpad -	<code>+rotate_cw</code> / <code>+rotate_ccw</code>	Hold to rotate the selection 2° clockwise or anticlockwise every 0.1 s.

The held keys are aliases rather than simple commands: each one starts a repeating timer, and the matching release (`-left`, `-translate_left`, and so on) cancels it. `alias` in the console lists them. The **Keymap** window spells the keypad keys `numpad_left`, `numpad_right`, `numpad_up`, `numpad_down`, `numpad_multiply`, `numpad_divide`, `numpad_add` and `numpad_subtract`.

A.2 Selection, editing and document keys

Key	Command	What happens
Ctrl+D	element copy	Duplicates the selected elements, in place on top of the originals.
Ctrl+G	planz clear copy preprocess validate blob preopt optimize spool	Builds the job from the current operations and sends it to the spooler — the same pipeline the ribbon Start button runs.
Ctrl+1 ... Ctrl+5	bind 1 move \$x \$y, bind 2 move \$x \$y, and so on	Stores the laser head's current position in the number keys: after pressing Ctrl+3 , the key 3 moves the head back to that point. Device required.
Ctrl+Shift+C	align bed group xy center center	Centres the selection, treated as one group, on the bed.
Delete , Numpad Del (numpad_delete)	tree selected delete	Deletes the selected nodes. A selection made in the scene is mirrored in the Tree pane, so this deletes the selected elements.
Escape	—	Not a keymap entry: it is handled by the active tool, which returns you to the selection tool. The old pause binding on this key has been removed; use the Pause key instead.

Selection and clipboard keys you may expect here — **Ctrl+A** (select all), **Ctrl+I** (invert), **Ctrl+C**, **Ctrl+V**, **Ctrl+X**, **Ctrl+Z** and **Ctrl+Shift+Z** (undo, redo), **Ctrl+O** (open project) — belong to the menus rather than the keymap (§A.6). They work, but they do not appear in the **Keymap** window and cannot be reassigned there.

A.3 Machine, windows and panels

Key	Command	What happens
Home	home	Homes the laser (sends it to the endstops). Device required.
Pause	pause	Real-time pause and resume of the running job. Device required.
L , U	lock / unlock	Locks or unlocks the rail, so the gantry cannot be pushed around by hand. Needs a driver that supports a rail lock.
F5	refresh	Refreshes the main window: caches are invalidated and the scene is redrawn.
Ctrl+F6	page home	Switches the ribbon to its Project page.
Ctrl+F7	page design	Switches the ribbon to a page named <i>design</i> . The stock ribbon has only Project , Modify and Config pages, so on a default profile this command finds nothing.
Ctrl+F8	page modify	Switches the ribbon to its Modify page.
Ctrl+F9	page config	Switches the ribbon to its Config page.
F1	(Help menu)	Opens the wiki page of the control under the pointer, falling back to the general GUI page. Menu accelerator, not remappable.
F11	(Help menu)	The same handler as F1 , from the Help menu. Not remappable.

The F-keys that older instructions associate with windows — **F4** , **F6** , **F7** , **F9** and **F12** — have no default binding in this version; see §A.5.

A.4 Test, rotary, wizard and reset bindings

Key	Command	What happens
Ctrl+E	circle 0.5in 0.5in 0.5in stroke red classify	Draws a test circle — a radius of 0.5in about a centre 0.5in from the origin on both axes, stroked red — and hands it to classification, which assigns it to a matching operation.
Ctrl+R	rect 0 0 1in 1in stroke red classify	Draws a 1in × 1in test square with its top left corner on the origin, stroked red, and classifies it the same way.
Ctrl+Shift+O	outline 1mm	Outlines the selection with a 1 mm offset.
Ctrl+Shift+H	scale -1 1	Mirrors the selection horizontally, about its own centre.
Ctrl+Shift+V	scale 1 -1	Mirrors the selection vertically.
Ctrl+Alt+D	image wizard Gold	Runs the <i>Gold</i> image wizard on the selected image.
Ctrl+Alt+E	image wizard Simple	<i>Simple</i> wizard.
Ctrl+Alt+N	image wizard Newsy	<i>Newsy</i> wizard.
Ctrl+Alt+O	image wizard Stipo	<i>Stipo</i> wizard.
Ctrl+Alt+X	image wizard Xin	<i>Xin</i> wizard.
Ctrl+Alt+Y	image wizard Gravy	<i>Gravy</i> wizard.
Ctrl+Alt+Shift+A	arm;startjob	Arms the laser and starts the job in one press: the two steps of Arm followed by Start .
Ctrl+Alt+Shift+Home	bind default;alias default	Restores every default binding and alias.

A wizard works on an image, so select the image first; a locked image is skipped and reported in the console.

A.5 Keys with no default binding

Several keys that older MeerK40t releases, and some third-party instructions, bind are deliberately unbound in this version. The defaults live in `meerK40t/core/bindalias.py` as a list of values per key, newest first; a leading empty value means “no binding”, and it also removes that key from a profile that still carries an older default. The table lists the command each key used to run, so that you can put it back with **Add Hotkey** if you want it.

Key	Command it used to run	How to get the same thing now
F4	window open CameraInterface	Tools ► CameraInterface; requires a camera.
F6	window open JobSpooler	Tools ► Spooler, or the Spooler pane.
F7	window open Controller	Tools ► Controller. The window is provided by the driver of the loaded device, so a device must be connected — with no device there is no Controller window to open.
F8	dialog_path	The path entry dialog, from the tree's right-click menu.
F9	dialog_transform	The transform dialog, from the same menu.
F12	window open Console	Tools ► Console, the Console ribbon button, or the Console pane.
Escape	pause	Pause . Escape still cancels the active tool.
Ctrl+A , Ctrl+C , Ctrl+V , Ctrl+X , Ctrl+I , Ctrl+O , Ctrl+S , Ctrl+Z , Ctrl+Shift+Z Ctrl+F , Ctrl+S	element* select, clipboard copy, clipboard paste, clipboard cut, element* select^, outline, dialog_stroke, undo, redo dialog_fill / dialog_stroke	The menus now own these keys — see §A.6. The commands themselves still exist.
Alt+C , Alt+E , Alt+R	cut / engrave / raster	The fill and stroke entry dialogs, from the tree's right-click menu.
Alt+F , Alt+S , Alt+H , Alt+P , Alt+T	dialog_fill, dialog_stroke, dialog_path, dialog_flip, dialog_transform	Groups the selection into that kind of operation; use the Operations pane or the right-click menu.
Alt+F3 , Ctrl+F3	rotaryscale / rotaryview	The same entry and flip dialogs.
Ctrl+Alt+G , Ctrl+Alt+S	image wizard Gold / image wizard Stipo	Both commands still exist: rotaryscale scales the selected elements for rotary work, rotaryview toggles the stretched scene preview. Bind them, or type them in the console.
Ctrl+Alt+Shift+Escape	reset_bind_alias	Duplicates of Ctrl+Alt+D and Ctrl+Alt+O above.
Ctrl+Shift+L	signal lock_helper	Reset the shortcuts in the Keymap window, or with bind default;alias default .
		Removed upstream, with the comment that it does not work. Rail lock and unlock are on L and U .

A.6 Menu accelerators that are not in the keymap

These keys are wired directly into the menus. They work exactly as the menu entry does, they are printed next to the menu label, and they cannot be changed in the **Keymap** window — rebinding one would mean editing the menu.

Key	Menu item	What happens
Ctrl+N	File ► New	Clears operations, elements and notes.
Ctrl+O	File ► Open Project	Opens a project; hold Shift to be asked where to place it.
Ctrl+S	File ► Save	Saves the project as an SVG file, overwriting the existing file.
Ctrl+Shift+S	File ► Save As	Saves into a new SVG file.
Ctrl+Z	Edit ► Undo	Undoes the last action.
Ctrl+Shift+Z	Edit ► Redo	Reverts the last undo.
Ctrl+X , Ctrl+C , Ctrl+V	Edit ► Cut, Edit ► Copy, Edit ► Paste	Cut, copy and paste the selected elements.
Ctrl+Shift+V	Edit ► Paste from system	Pastes data copied in another application.
Ctrl+A	Edit ► Select all	Selects every element on the scene.
Ctrl+I	Edit ► Invert selection	Inverts which elements are selected.
Ctrl+- , Ctrl++	View ► Zoom Out, View ► Zoom In	Zooms the scene out or in.
Ctrl+B	View ► Zoom to Bed	Fits the machine bed in the window.
Ctrl+Shift+B	View ► Zoom to Selected	Fits the current selection in the window.
Ctrl+U	View ► GUI Appearance ► Show/Hide UI-Panels	Hides or shows every pane except the scene.
Ctrl+,	Edit ► Settings ► Preferences	Opens the Preferences window.
F1	Help ► Help	Help for the control under the pointer (not on macOS).
F11	Help ► Online Reference	The same help handler.

A.7 Mouse and trackpad

The left button is for working on the design; the right and middle buttons — with a modifier where needed — move the camera you look at it with. Chapter 3 introduces these in context; this is the complete list.

Input	What happens
Wheel up / down	Zooms in or out around the pointer.
Wheel with Ctrl	The other of zoom and pan; by default that means Ctrl +wheel pans vertically.
Wheel left / right, or Shift +wheel	Pans horizontally.
Right-drag with Alt	Drag-zoom: dragging down or right zooms in, up or left zooms out.
Middle-drag with Ctrl	Drag-zoom, identical to the above.
Right-drag with Ctrl	Drag-pan: the scene follows the pointer.
Middle-drag	Nothing; the event is consumed so that it does not disturb the scene.
Trackpad pinch, or gesture zoom	Zooms in and out about the pointer.
Trackpad two-finger pan	Pans the scene.
Left-drag on empty space	Rubber-band selection. Dragging <i>right</i> (enclosing the elements) selects everything the rectangle contains; dragging <i>left</i> (touching them) selects everything the rectangle touches. The status bar names the method in use, and the rectangle is drawn in the matching colour. Shift adds to the existing selection, Ctrl inverts the selection state of what it catches.
Left click	Selects the element under the pointer.
Left-drag on the selection	Moves the selected elements. Press inside the selection, hold for about half a second, then drag: the pointer changes and the elements follow. Start dragging straight away and you get a rubber band instead. Grid snapping and magnet lines apply while you drag.
Right click	The scene menu: background and grid switches, <i>Move laser head here</i> , <i>Zoom Out</i> , <i>Zoom In</i> , <i>Zoom to Bed</i> and <i>Zoom to Selected</i> .

Three preferences change this behaviour: `mouse_wheel_pan` makes the wheel pan instead of zoom, and `mouse_pan_invert` and `mouse_zoom_invert` reverse the pan and zoom directions. They live in Edit ► Settings ► Preferences, **GUI**.

A.8 Where these defaults come from

The defaults are built from tables in `meerk40t/core/bindalias.py`: `DEFAULT_KEYMAP` for the keys, `DEFAULT_ALIAS` for the held-key timers. Each entry keeps its current value first and its older values after it, so a default that has changed since you last saved your settings can be migrated to the new value — unless you had already bound that key yourself, in which case your version wins.

- **Nothing happens at all.** The key has no binding in this version. Compare it with §A.5 — **F12** for the console, **F6** for the spooler and **F7** for the controller window are the frequent cases — and add it with **Add Hotkey** if you want it.
- **Nothing happens, and the key is bound.** Most likely nothing was selected, or the command needs a machine. `translate`, `scale`, `rotate`, `outline`, `element copy` and the alignment commands all work on the selection. `home`, `pause`, `lock`, `unlock`, the **A / D / W / S** jogs and the **Ctrl+1** to **Ctrl+5** position keys need a connected device.
- **A window will not open.** Window commands such as `window open Controller` are registered by the driver of the loaded device. With no device loaded there is no window to show, so connect or add the device first (Chapter 4).
- **The key types instead.** Keyboard events go to whatever has focus. Inside the console, a

text field or a numeric field, a key is text — click in the scene, or leave the field, first. While a creation or node-editing tool is active it consumes keys as well; press **Escape** to leave the tool.

Reading the current keymap

bind with no arguments prints every binding as a numbered list; **bind <key> <command>** sets one, and **alias** does the same for the held-key timers. The **Keymap** window is a front end to exactly those two commands, which is why anything you can type in the console can be bound to a key. Bindings are stored in your settings profile, and **bind default;alias default** discards your changes and rebuilds the defaults listed in this appendix.

Chapter B

Console command reference

The console is MeerK40t's command line, and it is not a side door: every button, menu item and pane action ends up as a console command, and many things the interface does not offer at all are console commands. This appendix is the lookup table to keep open while you type. Chapter 18 is the tutorial that teaches the console; read that first if you have never used a command line.

Open the **Console** window with the ribbon's **Console** button, or toggle the pane from **Panes ► Console** (panes are described in Section 3.5). **F12** is a popular key to bind to **window open Console** in the Keymap window, but it has no binding by default (Appendix A). In the console window, **Tab** completes command names, **Up** and **Down** walk the history, and **Enter** runs the line. Three commands answer almost every question you will have while typing:

- **help** lists every command, grouped by the context it belongs to; **help <command>** prints that command's arguments and options.
- **find <text>** searches command and argument names, e.g. **find dpi**.
- **load_types** and **save_types** list every file format this installation can read and write.

The tables below follow the grouping of **help**. Every row is one complete command line that you can type as printed once you have replaced the placeholders with your own values.

How to read the syntax column

- **command** — literal text: type it exactly like this.
- **<value>** — a required value that you supply.
- **[<value>]** — the same, but optional.
- **[--long/-s <value>]** — one option with two spellings; use either **--long** or **-s**, and put a space before its value. Options may appear before or after the plain arguments.
- **a|b** — choose one of the alternatives.
- **...** — the preceding pattern repeats (used for point lists and for multi-part commands).

The brackets, angle brackets, vertical bars and dots are notation and are never typed. Curly braces mean something else entirely: in a text element they mark a wordlist reference, as in **text "Job {serial}"**.

Session, help and scripting

Command	Syntax	What it does
help ?	help [<command>] [--input <context>] [--output <context>]	List all commands, or the arguments and options of one command.
find ??	find <text>	List every command and argument name containing <text>.
echo	echo <text>	Print <text> in the console. Mainly useful in scripts, which otherwise run silently.
timer	timer[<name>] <count> <seconds> <command>	Run <command> <count> times, <seconds> apart. A named timer can be cancelled with timer<name> off. Bare timer lists the running timers. Options: --off/-o, --gui/-g, --quiet/-q.
loop	loop <first> <last> [--variable/-v <name>] <command>	Repeat <command> once for every integer from <first> to <last>. The loop number is written into the command as a brace reference (default name loop). On Python 3.12 and newer the substitution fails silently and the literal braces are passed through, so check one loop before you rely on it.
threaded	threaded <command>	Run the rest of the line in the background so the console is usable again immediately. This is the prefix the batch and simulation buttons use.
wait	wait <seconds>	Wait before running the next command. This blocks the console, so it belongs in scripts, not in things you type interactively.
quit shutdown	quit	Shut the program down. Both spellings exist.
restart	restart	Shut down, exit and start MeerK40t again.
context	context	List the context names. A context is a settings scope with its own variables.
service	service [<domain>]	List the services and providers, or enter a service domain.
service start	service <domain> start <name> [--label/-l <label>] [--path/-p <path>] [--init/-i]	Create and start a service from a registered provider, for example service device start grbl.
service activate	service <domain> activate <index>	Make the service at <index> the active one of its domain.
service destroy	service <domain> destroy <index>	Destroy the service at <index>.
module	module [(open close) <name>] [--path/-p <path>]	List the open modules, or open and close one.
channel	channel open <name> [--force/-f]	Watch a kernel channel in the console. There is no channel called console, and unknown names are refused unless you force them.

Command	Syntax	What it does
<code>channel close</code>	<code>channel close <name></code>	Stop watching a channel.
<code>channel list</code>	<code>channel list</code>	List the channels and mark the ones being watched.
<code>channel print</code>	<code>channel print <name> [--close/-c]</code>	Print a channel to standard output (starting a terminal run).
<code>channel save</code>	<code>channel save <name> [--filename/-f <file>]</code>	Record a channel to disk, by default a timestamped file in the working directory.
<code>execute</code>	<code>execute <file></code>	Run every line of <code><file></code> as a console command. Blank lines are skipped and lines starting with <code>#</code> are comments.
<code>batch</code>	<code>batch</code>	List the stored batch lines with their indices.
<code>batch add</code>	<code>batch add <line></code>	Append one command to the stored batch list.
<code>batch remove</code>	<code>batch remove <index></code>	Delete the batch line at <code><index></code> (indices are the ones <code>batch</code> prints).
<code>batch run</code>	<code>batch run <index></code>	Run one batch line.
<code>batch enable</code>	<code>batch disable enable <index></code>	Disable or re-enable a batch line without deleting it.
<code>alias</code>	<code>alias</code>	List the defined aliases and the keystroke bound to each.
<code>alias set</code>	<code>alias <name> <commands></code>	Define an alias. Several commands are separated by <code>;</code> , and the alias is then a single word you can type (or bind).
<code>alias unset</code>	<code>alias <name></code>	Delete that alias.
<code>alias default</code>	<code>alias default</code>	Restore the built-in aliases. Note that this discards your own.
<code>bind</code>	<code>bind</code>	List the key bindings and the commands they run.
<code>bind set</code>	<code>bind <key> <command></code>	Bind a keystroke to a command. An empty command unbinds the key, and <code>bind default</code> restores the built-in keymap.

Scripts run without feedback

`execute` and the batch list run line by line and never wait for you. Anything that needs a pause (a move, a laser pulse, a human) works better as a plan stage: add the matching utility operation to the tree rather than relying on `wait`. A file run with `execute` is also the easiest way to keep a job recipe under version control.

Settings and configuration

Command	Syntax	What it does
<code>set</code>	<code>set [--path/-p <context>] [<key> <value>]</code>	With no key, list the scalar variables of a context (the kernel root by default). With a key and a value, change that variable.
<code>flush</code>	<code>flush</code>	Write the persistent settings to disk immediately, instead of waiting for the program to exit.
<code>setting_export</code>	<code>setting_export <directory></code>	Write a timestamped copy of the whole configuration into <code><directory></code> .
<code>setting_import</code>	<code>setting_import <file></code>	Read a configuration file written by <code>setting_export</code> back in.

There is no `get` command

There is no top-level `get` or `setting` command: to read a value, run `set` with nothing after it and look through the listing. `set` also only writes keys that already exist on that context, and it fails silently when the key name is wrong, so list first:

```
set -p elements
set -p elements classify_new False
set -p elements
flush
```

`get` and `set` do exist as wordlist subcommands; see the materials and wordlist table below. Changes made with `set` are permanent: they are stored in the configuration and reloaded on the next start.

Files and documents

Command	Syntax	What it does
<code>load</code>	<code>load <file></code>	Load a project or drawing. The reader is chosen from the file, so the format usually does not have to be named.
<code>load_types</code>	<code>load_types</code>	List every format this installation can load.
<code>save</code>	<code>save <file> [--version/-v <version>]</code>	Save the whole scene to <code><file></code> . The version must be one of the versions <code>save_types</code> prints, and it has to match the file extension: the plain and default versions write <code>.svg</code> , the compressed version writes <code>.svgz</code> .
<code>save_types</code>	<code>save_types</code>	List the formats and versions that can be saved.
<code>file_startup</code>	<code>file_startup</code>	Run the stored file-startup commands, the list you edit in the AutoExec pane.
<code>xload</code>	<code>xload <file> <x> <y> <width> <height></code>	Load a file and place it into the given rectangle on the bed.

Command	Syntax	What it does
---------	--------	--------------

Creating elements

Command	Syntax	What it does
rect	rect <x> <y> <width> <height> [--rx/-x <r>] [--ry/-y <r>]	Rectangle whose top-left corner is at <x> <y>, optionally with rounded corners.
circle	circle <x> <y> <r>	Circle centred on <x> <y> with radius <r>.
ellipse	ellipse <cx> <cy> <rx> <ry>	Ellipse centred on <cx> <cy>.
arc	arc <cx> <cy> <rx> <ry> <start> <end> [--rotation/-r <angle>]	Arc of an ellipse from <start> to <end>. Write the angles as angles with a unit, e.g. 90deg.
line	line <x0> <y0> <x1> <y1>	Straight line between two points.
polyline	polyline <x1> <y1> <x2> <y2> ...	Open polyline through the listed points.
polygon	polygon <x1> <y1> <x2> <y2> ...	Closed polygon through the listed points.
path	path "<svg path data>"	Path element built from SVG path syntax. Quote the whole path string.
text	text "<string>" [--size/-s <pt>]	Bitmap text element.
linetext	linetext <x> <y> "<text>" [--font/-f <file>] [--font_size/-s <size>] [--font_spacing/-g <factor>]	Single-line vector text drawn with a Hershey font, so it can be cut rather than engraved.

Elements appear selected

Every command in this table puts the new element into the selection, so the next command can act on it, and it is classified into a matching operation if you have **classify new** enabled. There is no console command that creates a raster image element: images arrive through **load** or the interface's import, which is why the image commands below take an existing image as their input.

Selecting, transforming and editing elements

Commands whose first column contains a bare **element** or **operation** prefix take their input from the console's current data rather than from the selection. Commands without such a prefix, such as **scale**, start a new chain and fall back to the current selection.

Command	Syntax	What it does
element	element	The currently selected elements, ready to be passed to the next command.
element*	element*	Every element in the scene.
element~	element~	Every element that is not selected.

Command	Syntax	What it does
<code>element0,3,4</code>	<code>element<n>,<n></code>	The elements with those indices, as printed by <code>element* list</code> .
<code>select</code>	<code><data> select</code>	Make the data the selection, e.g. <code>element* select</code> to select everything.
<code>select+</code>	<code><data> select+</code>	Add the data to the selection.
<code>select-</code>	<code><data> select-</code>	Remove the data from the selection.
<code>select^</code>	<code><data> select^</code>	Toggle the data in the selection.
<code>range</code>	<code><data> range <first> <last> [--step/-s <n>]</code>	Reduce the data to a slice by index before passing it on.
<code>id</code>	<code><data> id [<id>]</code>	Print, or set, the ids of the data.
<code>scale</code>	<code>scale <sx> [<sy>] [--px/-x <x>] [--py/-y <y>] [--absolute/-a]</code>	Scale the selection, optionally differently in y and from a chosen origin instead of the centre of the selection.
<code>rotate</code>	<code>rotate <angle> [--cx/-x <x>] [--cy/-y <y>] [--absolute/-a]</code>	Rotate the selection around its centre, or around <code><cx> <cy></code> .
<code>translate</code>	<code>translate <tx> <ty> [--absolute/-a]</code>	Move the selection by an offset.
<code>position</code>	<code>position <x> <y></code>	Move the selection so that it starts at <code><x> <y></code> .
<code>move_to_laser</code>	<code>move_to_laser</code>	Move the selection to the current laser position.
<code>matrix</code>	<code>matrix <sx> <kx> <ky> <sy> <tx> <ty></code>	Set the selection's transformation matrix directly.
<code>resize</code>	<code>resize <x> <y> <width> <height></code>	Move and scale the selection so that it fills the given rectangle.
<code>reset</code>	<code>reset</code>	Reset the affine transforms of the selection back to identity.
<code>area</code>	<code>area [--new_area/-n <area>] [--density/-d <n>]</code>	Report, or change, the area a raster element covers.
<code>copy</code>	<code>copy [--dx/-x <dx>] [--dy/-y <dy>] [--copies/-c <n>]</code>	Duplicate the selection, optionally offset and repeated.
<code>delete</code>	<code><data> delete</code>	Delete the data, e.g. <code>element delete</code> . <code>delete</code> on its own is rejected, because it does not accept the base context.
<code>group</code>	<code>group <label> [--reparent/-r]</code>	Group the selection under a new label.
<code>ungroup</code>	<code>ungroup</code>	Dissolve the selected groups.

Note the asymmetry: `scale`, `rotate`, `translate`, `position`, `resize`, `copy` and `area` can be typed on their own and then work on the selection, while `delete`, `select`, `select+`, `select-`, `select^`, `range` and `id` must be given their data, for example `element* select` or `element delete`. `move` is not an element command at all: it moves the laser head, and lives in the device table below.

Operations and the plan pipeline

Command	Syntax	What it does
cut engrave raster imageop dots hatch	<type> [--speed/-s <mm/s>] [--power/-p <ppi>] [--dpi/-d <n>] [--overscan/-o <length>] [--passes/-x <n>] [--color/-c <colour>] [--default/-D] [--parallel/-P] [--stroke/-K] [--fill/-F]	Create an operation of that type and attach the elements it is given to it. With --parallel every element gets its own operation instead.
operation operation* operation~ operation* list	operation, operation<n>,<n> operation* list	The selected, all, unselected or listed operations, ready to be passed on. List the operations with their children.
operation* empty	operation* empty	Remove every child from the selected operations, leaving the operations.
operation* enable	operation* disable, enable	Disable or enable operations, so they are left out of the plan.
plan	plan[<name>] <command>	Work on the plan called <name>. Bare plan uses the plan named 0; typing planz renames the default plan to z for this session, and plan2 switches to the plan named 2.
list-plans	list-plans	List the plans that exist and the stage each one has reached.
plan clear	plan clear	Empty the plan.
plan copy	plan copy	Copy the artwork of every operation into the plan.
plan copy-selected	plan copy-selected	Copy the artwork of the selected operations only.
plan preprocess	plan preprocess	Prepare the plan for execution: convert the scene into device coordinates.
plan validate	plan validate	Run the plan's validation.
plan blob	plan blob	Convert the plan into the device's own instruction format.
plan preopt	plan preopt	Prepare the plan for optimisation.
plan optimize	plan optimize	Optimise the plan, for example to reduce travel between cuts.
plan finish	plan finish	Mark the plan finished, which is what the simulation expects.
plan return	plan return	Push the plan's operations back into the tree.
plan sublist	plan sublist [<first>] [<last>]	Keep only that slice of the plan, where the last index -1 means the end.
plan console	plan console [--index/-i <n>] <command>	Insert a console command into the plan at that position.
spool	spool	Send the plan it is given to the device spooler. Typed on its own it prints the spooler queue instead.

Command	Syntax	What it does
save_job	plan <stages> save_job <file>	Export the plan in the active driver's own file format instead of burning it. Only drivers that have such a format offer this.

The order matters.

`copy`, `preprocess`, `validate`, `blob`, `preopt`, `optimize` must run in that order; `spool` refuses a plan that has not been through `blob`, because there is nothing in the device's language yet. Each stage records itself, which is what `list-plans` shows you.

copy or copy-selected?

`copy` takes the artwork of every operation that produces output. `copy-selected` takes only the selected ones, which is how the interface burns a single operation.

Device, motion and the spooler

Command	Syntax	What it does
device	device	List the device providers that can be added, with the family each one belongs to.
device add	device add <provider> [--label/-l <label>]	Create and start a new device from a provider, e.g. <code>device add grbl-generic</code> .
device activate	device activate [<name> <index>]	With no argument, list the defined devices with their labels, types and status. With a label or an index, make that one the active device.
device duplicate	device duplicate <name> <index> [<label>]	Copy an existing device entry, which is how you keep two configurations of the same machine.
device delete	device delete <name> <index>	Delete a device entry. The active device is protected.
devinfo	devinfo	Print the current position in the form <code>x,y;nx,ny;</code> .
home	home	Home the laser, according to the device's own definition of home.
physical_home	physical_home	Home against the physical end-stops.
lock unlock	lock, unlock	Lock or unlock the rail, so the head cannot be pushed around by accident.
move move_absolute	move <x> <y> [--force/-f]	Move the head to an absolute position. Without <code>--force</code> the move is refused while the spooler is busy (Busy Error).
move_relative	move_relative <dx> <dy> [--force/-f]	Move the head by an offset. Same busy rule.

Command	Syntax	What it does
left right up down	left <amount>	Queue a relative jog, 1 mm when no amount is given. The move is sent a moment later by <code>jog</code> , which is how the arrow keys produce a continuous nudge while held.
jog	jog [--force/-f]	Send the queued jog move. <code>--force</code> sends it even when the device would otherwise confine it.
+laser -laser	+laser, -laser	Turn the laser on or off without moving the head.
spool list	spool list	List the jobs in the spooler queue.
spooler clear	spooler clear	Empty the spooler queue.
spooler send	spooler send <command>	Send a single plan command (<code>home</code> , <code>unlock</code> , <code>origin</code> , ...) to the spooler as a small job.
pause resume	pause, resume	Pause or resume the running job. Only registered by drivers that can hold a job.
estop abort	estop, abort	Abort the running job. Driver dependent.
goto	goto <x> <y>	Send a goto to a galvo controller. Driver dependent.

Motion needs a device

The motion and job commands come from the active device's driver, not from the kernel: a machine without endstops has no `physical_home` that does anything useful, and `pause` does not exist on every driver. If a command in this table is rejected as "not a registered command", check which device is active with `device activate` and see Appendix C for what that driver supports.

Materials and wordlists

Command	Syntax	What it does
material	material	Enter the materials context, which holds the sections of your material library.
material save	material save <name> [--author/-a <author>] [--description/-d <text>]	Save the current materials into the settings under <name>.
material load	material load <name>	Load a saved material library.
material list	material list [<name>]	Show the material sections, or one named library.
material delete	material delete <name>	Delete a saved material library.
wordlist	wordlist	Enter the wordlist context.
wordlist add	wordlist add <key> [<value>]	Add a key with a value (empty when none is given).
wordlist addcounter	wordlist addcounter <key> [<start>]	Add a numeric counter that starts at <start>, or 1.

Command	Syntax	What it does
<code>wordlist get</code>	<code>wordlist get <key> [<index>]</code>	Print the value the key has at that index.
<code>wordlist set</code>	<code>wordlist set <key> <value> [<index>]</code>	Write a value at an index.
<code>wordlist index</code>	<code>wordlist index <key @ALL> <index +n></code>	Move an index, where +2 means “two on” rather than “set to 2”.
<code>wordlist list</code>	<code>wordlist list [<key>]</code>	List the keys, or the values stored under one key.
<code>wordlist advance</code>	<code>wordlist advance</code>	Move every index on, so the next job gets the next values. Spooling a plan runs this for you.
<code>wordlist restore</code>	<code>wordlist restore [<file>]</code>	Load a saved wordlist, by default the standard one.
<code>wordlist backup</code>	<code>wordlist backup [<file>]</code>	Save the current wordlist.
<code>wordlist load</code>	<code>wordlist load <csv-file></code>	Attach a CSV file to the wordlist, which is how you import a list of names or serial numbers.

How the wordlist reaches the machine

A text element can contain brace references such as `{name}` or `{serial}`. They are left alone on screen and replaced by the wordlist values while the plan is built, which is why the preview still shows the braces. The job that is built is the one that carries the substituted text, so build a plan and spool it before judging the result.

Images, rendering and tracing

Command	Syntax	What it does
<code>image</code>	<code>image</code>	Enter the image context for the selected images, which is how the raster commands below are reached.
<code>image dewhite</code>	<code>image dewhite</code>	Remove the white background from the image.
<code>image rgba</code>	<code>image rgba</code>	Convert the image to RGBA.
<code>image quantize</code>	<code>image quantize [<colours>]</code>	Reduce the image to that many colours, 10 by default.
<code>image solarize</code>	<code>image solarize [<threshold>]</code>	Solarise the tones, with a threshold of 250 by default.
<code>image save</code>	<code>image save <file> [--processed/-p]</code>	Write the image to disk, optionally the processed result rather than the original.
<code>render</code>	<code>render [--dpi/-d <dpi>]</code>	Rasterise the selected elements into an image element, 500 dpi by default.
<code>render_split</code>	<code>render_split <columns> <rows> <dpi> [--order/-o <mode>]</code>	Rasterise and cut the result into a grid of image parts, for tiling a large engraving.

Command	Syntax	What it does
<code>render_keyhole</code>	<code>render_keyhole <dpi> [--order/-o <mode>] [--invert/-i] [--outline/-b]</code>	Rasterise the selection as a keyhole mask, with optional inverted mask and outline.
<code>vectorize</code>	<code>vectorize [--dpi/-d <dpi>] [--turnpolicy/-z <policy>] [--turdsiz/-t <n>] [--alphamax/-a <f>] [--optcurve/-n] [--opttolerance/-0 <f>] [--color/-C <colour>] [--invert/-i] [--blacklevel/-k <f>]</code>	Convert the selected elements into paths by rasterising them and tracing the result.
<code>potrace</code>	<code>image potrace [<options>]</code>	Trace the selected image into paths with Potrace. It takes the image options above.
<code>vectrace</code>	<code>image vectrace</code>	Trace the selected image into paths by following the boundaries between black and white.
<code>vtracer</code>	<code>vtracer</code>	Trace the selection into paths with VTracer, which also handles colour images. It needs the optional <code>vtracer</code> package installed.
<code>trace</code>	<code>trace <method> <resolution> [--start/-s <0 1 2>] [--force/-f]</code>	Work out the outline of the selection and send that outline to the device as a job, so you can see where the job would run. Methods: <code>quick</code> , <code>hull</code> , <code>segment</code> , <code>circle</code> .
<code>tracegen</code>	<code>tracegen <method> <resolution></code>	The same outline, but created as a new element instead of being burned.

`trace` and `tracegen` default to the selection and can be redirected with a chain. Elements that are not in an enabled operation are left out of the outline unless you pass `--force`, and the console tells you how many were skipped.

Windows, panes and dialogs

These commands need the graphical interface; in a console-only run (`--no-gui`) they are not registered at all.

Command	Syntax	What it does
<code>window</code>	<code>window [--path/-p <context>]</code>	List the open and registered windows in that context.
<code>window list</code>	<code>window list</code>	List the window names that can be opened.
<code>window displays</code>	<code>window displays</code>	Show the displays and where each open window currently sits.

Command	Syntax	What it does
window open window toggle window open <window> [--multi/-m <n>]		Open a window, or close it again if it is open. --multi/-m with an index opens another copy. Typical names: Console, AutoExec, Preferences, JobSpooler, Simulation, DeviceManager.
window close window reset	window close <window> window reset <window>	Close the window. Forget the stored position and size of one window, or of all of them with *.
page	page [<page>]	Switch the ribbon bar to a page, or report which page is active.
pane	pane	Enter the pane context.
pane show pane hide pane toggle	pane show <pane>, pane hide <pane>, pane toggle <pane>	Show, hide, or show-or-hide a docked pane, e.g. <code>pane toggle spooler</code> .
pane float pane dock pane save pane load	pane float [--always/-a] <pane>, pane dock <pane> pane save <name>, pane load <name>	Float a pane as a free window, or put it back in the frame. Store the current pane layout under a name, or restore a stored one.
pane reset pane lock pane unlock pane toggleui	pane reset pane lock, pane unlock pane toggleui	Restore the default layout. Lock the panes so they cannot be moved, or unlock them. Hide every pane except the scene, or bring them all back.
dialog_path	dialog_path	Open a dialog to type SVG path data and add the path to the scene.
dialog_transform	dialog_transform	Open a dialog to type an SVG transform such as <code>scale(1.5)</code> and apply it to the elements.
dialog_fill dialog_stroke	dialog_fill, dialog_stroke	Open the colour picker for the fill or the stroke of the selection.

The four dialog commands are hidden from `help`, because they exist only to put a keyboard shortcut on an interface dialog. They are listed here so that a macro of yours can use them.

Remote control and server commands

These start servers that let other software drive this copy of MeerK40t, or let another computer forward data to the laser. `--quit/-q` stops the server again, and `--verbose/-v` mirrors the server's own channel into the console.

Command	Syntax	What it does
grblcontrol	grblcontrol [--port/-p <port>] [--verbose/-v] [--quit/-q]	Listen on a TCP port (23 by default) and answer as a GRBL controller, so GRBL software can send jobs through MeerK40t to this machine.

Command	Syntax	What it does
<code>ruidacontrol</code>	<code>ruidacontrol [--verbose/-v] [--quit/-q] [--jogless/-j] [--man_in_the_middle/-m <host>] [--bridge/-b]</code>	The same idea for Ruida software. <code>--man_in_the_middle</code> forwards the traffic to a real laser instead of emulating it, and <code>--bridge</code> uses the bridge protocol.
<code>lhyserver</code>	<code>lhyserver [--port/-p <port>] [--verbose/-v] [--watch/-w] [--quit/-q]</code>	Serve the lihuiyu (K40) controller over TCP, so a small computer next to the laser can forward its USB traffic. <code>--watch</code> also logs the data.
<code>webserver</code>	<code>webserver [--port/-p <port>] [--silent/-s] [--quit/-q]</code>	Start the built-in web server, port 2080 by default.

Worked command chains

The console runs one command per line, but a line may hold several commands: a `|` between them splits the line, unless the bar sits inside a quoted string. Commands also chain without any separator at all, because each command passes its result to the next one. A command that cannot create its own input has to follow a command that produces it; when a command returns nothing, chaining stops and the console says so.

```
rect 0 0 20mm 10mm engrave -s 15 plan copy-selected preprocess validate blob preopt optimize
spool
```

Read from the left: `rect` creates the element and selects it; `engrave -s 15` creates an engrave operation at 15 mm/s and attaches the rectangle to it; `plan` hands that operation to the plan named 0; `copy-selected` copies only the selected operations into the plan; `preprocess` converts the scene into device coordinates; `validate`, `blob`, `preopt` and `optimize` prepare the device format and reduce travel; and `spool` queues the result on the device. Nothing moves until you press the machine's start button or run `startjob`.

The same job written one stage per line is easier to read and easier to paste into the **AutoExec** pane, because each line restates the plan:

```
rect 0 0 20mm 10mm | engrave -s 15
plan copy-selected
plan preprocess
plan validate
plan blob
plan preopt
plan optimize
plan spool
```

To write the job to a file instead of burning it, replace the last stage with `save_job <file>`:

```
rect 0 0 20mm 10mm engrave plan copy preprocess validate blob preopt optimize save_job job.egv
```

`save_job` writes whatever the active driver sends to the machine, so the extension depends on the driver. Use `plan copy` here rather than `copy-selected` only when you want the whole project, not one operation.

The two house aliases do the whole pipeline for you, and `alias` prints them along with everything else you have defined:

```
burn
simulate
alias
```

`burn` is `planz clear copy preprocess validate blob preopt optimize spool`, and `simulate` is the same chain with `finish` and the simulation window instead of `spool`. Both switch the default plan to `z` on the way, so a `plan` command you type afterwards refers to that plan.

Loading, changing and saving a document is three lines, and the save format is chosen by the extension:

```
load drawing.svg
element* select
scale 0.5
save smaller.svg
```

A saved copy of a scene can be compressed by asking for the matching version, `save smaller.svgz -v compressed`; the version and the extension must agree, which is what `save_types` lists.

Adding a machine is the same three commands every time: list the providers, add one, and make it active. `device activate` on its own lists the devices you already have, with the index the other forms accept:

```
device
device add grbl-generic
device activate
device activate Grbl
```

Personalising text is a wordlist plus a brace reference. The substitution happens while the plan is built, so build and spool the job to see it:

```
wordlist add name Alice
wordlist addcounter serial 100
wordlist list
text "Hello {name}"
```

Between jobs, `wordlist advance` steps every index on, and spooling does that for you. To keep several jobs apart instead, put the whole recipe in a file and run it with `execute`, or store it in the batch list with `batch add <line>`.

Gotchas

- **Always write a unit.** A bare number is not a millimetre and not a pixel: for lengths it is a raw internal unit of one 65535th of an inch, so `rect 0 0 10 10` produces a rectangle you cannot see, and for angles it is radians, so `rotate 45` turns the element seven times over rather than 45 degrees. Write `rect 0 0 10mm 10mm` and `rotate 45deg`. `cm`, `mm`, `in`, `mil`, `px`, `pt` and `%` are all accepted; a percentage is a fraction of the bed, the width for `x` and `width` and the height for `y` and `height`. In the console of the graphical interface, `unit 2cm` translates one value into every other unit.
- **Elements and operations need selecting.** Transform and element commands work on the selection when they start a chain, so `scale 0.5` with nothing selected reports “No selected elements” and does nothing. The pipeline is no different: `plan copy-selected` copies the

selected operations, and if none is selected the plan is quietly empty. Run the selection commands first (`element* select`, `operation* select`).

- **Some commands cannot start a line.** `delete`, `select`, `range`, `id` and the `operation* list` family only accept elements, operations or a plan as input. Typed first they are refused (“not a registered command in this context: Base”); put the input in front of them: `element* select`, `element delete`, `operation* list`.
- **Quote anything with spaces.** Only double quotes group a value; single quotes are not special, and there are no escape sequences. A comma also separates arguments, so quote text that contains one: `text "Hello world"`.
- **Write options with a space.** `-s 15` is the speed option with value 15. `-s15` is read as three separate options and will not do what you meant.
- **Case.** Command and option names are matched in lower case, so `RECT` works, but the values you pass keep their case, which matters for text and file names.
- **set is scoped and silent.** `set -p <context> <key> <value>` changes a variable that must already exist on that context; a wrong key name does nothing at all and prints nothing. List with `set -p <context>` first, and use `flush` to write settings to disk. The change is permanent until you change it back.
- **Plan names stick.** `planz` is not a typo: the letters after `plan` become the name of the default plan for the rest of the session, and bare `plan` then means “the plan named z”. The plan that a fresh session uses is named 0.
- **The pipeline is ordered.** `spool` refuses a plan that has not been through `blob`, and `optimize` does nothing useful before `blob` has run. When a chain half-works, `list-plans` shows how far the plan got.
- **Drivers differ.** Anything that moves the head or changes the state of the machine belongs to the active device’s driver. `pause`, `estop`, `status` and `goto` exist only where a driver implements them, so a command that works on one machine may be rejected on another.

Chapter C

Machine settings by driver

A *driver* is the part of MeerK40t that speaks one family of laser controller. Six families ship with the program: Lihuiyu (the K40 nano boards), GRBL, Ruida, Moshi, Newly and Balor (the JCZ galvo controllers). Each driver defines its own settings, and a device shows only the settings of its own driver — two machines with similar-looking Configuration windows can hold quite different contents. This appendix is the lookup table for those settings. Every row gives the label that appears in the device's **Config** window, the internal *key*, the value the program ships with, and what the setting changes.

Open the window from the **Devices** pane with **Config** for the active device, or through **Device-Settings ► Device Manager**. The key column is the name of the underlying variable, which is what the console **set** command reads and writes (Section 18, Appendix B).

Bed size and origin belong to the machine

Width, Height and *Force Declared Home* describe the hardware, not the artwork. If you change them after placing a design, the job is re-mapped onto the new bed and the same drawing may end up somewhere else on the material. Set the bed up before you position anything, and check placement again afterwards (Chapter 15).

Common device settings

Most drivers offer the same handful of settings, usually under the same keys. Those are listed once below. The driver sections that follow give the values that differ, and the settings that only one family has. A driver is expected to show only the rows relevant to it.

Setting	Key	Default	What it changes
Label	<code>label</code>	shipped name	Device name in the Devices list.
Width, Height	<code>bedwidth</code> , <code>bedheight</code>	see each driver	Bed size. Sets the scene, the bed limits and jog confinement.
Laserspot	<code>laserspot</code>	0.3 mm	Beam spot size used for display and planning. All six drivers.
X-Axis, Y-Axis	<code>scale_x</code> , <code>scale_y</code>	1.000	Scale from board units to millimetres. Lihuiyu calls these <code>user_scale_x</code> and <code>user_scale_y</code> .

continued overleaf

Setting	Key	Default	What it changes
Flip X, Flip Y	<code>flip_x, flip_y</code>	off	Mirror an axis. On for <code>flip_y</code> on GRBL and Balor.
Swap X and Y	<code>swap_xy</code>	off	Exchanges the axes. Applied before the flips. On by default for Balor.
Force Declared Home	<code>home_corner</code>	auto	Which corner counts as home. All drivers except Balor.
Curve Interpolation	<code>interp</code> ; Ruida: <code>interpolate</code>	5 mils; Ruida 500µm	Distance at which curves are flattened into straight segments.
Device Position	<code>signal_updates</code>	on	Draws the live position indicator.
Coolant	<code>device_coolant</code>	nothing	Coolant or air-assist method claimed for the device. All except Newly.
Max vector speed, Max raster speed	<code>max_vector_speed, max_raster_speed</code>	140, 750	Highest reliable cutting and raster speeds. Lihuiyu and GRBL.
Travel speed	<code>rapid_speed</code> (Balor: <code>default_rapid_speed</code>)	see drivers	Speed used for moves that do not cut.
Operation Defaults, speed	<code>default_speed_op_cut, default_speed_op_engrave</code> and the matching keys for raster and image	Cut 5, Engrave 25, Raster 250	Speed given to a newly created operation. Balor uses 150, 250 and 500.
Operation Defaults, power	<code>default_power_op_cut</code> and the same key pattern for the other three	1000	Power given to a newly created operation, in whatever unit the row below selects.
Show power as %	<code>use_percent_for_power_display</code>	on for GRBL, Newly, Balor	On: full power reads as 100%. Off: full power reads as 1000.
Show speed in mm/min	<code>use_mm_min_for_speed_display</code>	off	On: speeds are shown in mm/min. Off: mm/s. The Lihuiyu, Ruida and Moshi sources keep the same preference under the parallel key <code>use_minute_for_speed_display</code> .
Limit the controller buffer size, Controller Buffer	<code>limit_buffer, max_buffer</code>	on, 200 lines	GRBL: stop sending once the controller holds the stated number of lines.
Write Buffer	<code>buffer_max, buffer_limit</code>	900, on	Lihuiyu: cap on the USB write queue. The Interface panel exposes it as <i>Limit Write Buffer</i> with its own maximum.
X-Margin, Y-Margin	<code>user_margin_x, user_margin_y</code>	0	Present in the source but marked as not working, so no driver shows it in the window.

Two things that look like driver settings but are not. Homing on start is a job step, not a device setting: add *Append Physical Home* or *Append Return to Origin* in the Operations tree (Chapter 16). And the coolant list only expands after a coolant method has been registered, so the combo may hold nothing but *Nothing* until you use one.

Lihuiyu (K40)

The Lihuiyu driver runs the M2 and M3 nano boards found in K40-style CO₂ cutters and their clones. The board is a USB device reached through the CH341 chip: choose *USB* in the Interface panel for the normal case, *Networked* to send the same packets to another MeerK40t instance over TCP, or *Mock* for a dry run with no machine attached. These boards have no endstops, so *Home* moves to the declared origin rather than seeking a switch. The Configuration window carries the tabs Configuration, Interface, Setup, Effects, Operation Defaults, Warning, Default Actions and Display Options.

Setting	Key	Default	What it changes
Device Name	<code>label</code>	M2-Nano, M3-Nano	Internal name used for the device; <code>lihuiyu-device</code> when a device is added without a model entry.
Width, Height	<code>bedwidth, bedheight</code>	310 mm, 210 mm	Bed size of the usual K40.
X Scale Factor, Y Scale Factor	<code>user_scale_x, user_scale_y</code>	1.000	Board units to millimetres.
Board	<code>board</code>	M2	Which nano board is fitted: M2, M3, B2, M, M1, A, B or B1. This changes the speed codes sent to the board.
Hardware-PWM	<code>supports_pwm</code>	off	Power control by hardware PWM. Only M3 boards with firmware 2024.01.18g or newer; earlier M3 revisions behave as M2+. Shown only for the M3 board.
Maximum Laser strength	<code>power_scale</code>	30%	Ceiling that every operation's power is a fraction of; adjustable from 1 to 50, or to 100 with developer mode. Setting it too high can damage the laser tube.
Strict	<code>strict</code>	off	Enter and leave speed mode travelling in the same direction (an M2-V4 fix).
Twitch Vectors	<code>twitches</code>	off	Re-enables the old twitch behaviour when moving the vectors.
Use legacy raster method	<code>legacy_raster</code>	on	Uses the legacy raster engine: better at high speed, but with artifacts.
Rapid Moves Between Objects	<code>opt_rapid_between</code>	on	Travel at rapid speed between separate objects.
Minimum Jog Distance	<code>opt_jog_minimum</code>	256	Step below which a move is treated as a jog.
Jog Method	<code>opt_jog_mode</code>	0	<i>Default</i> , <i>Reset</i> or <i>Finish</i> .
Override Rapid Movements	<code>rapid_override</code>	off	Replaces the travel speed with the two values below.
X Travel Speed, Y Travel Speed	<code>rapid_override_speed_x, rapid_override_speed_y</code>	50.0	Per-axis travel speed used while the override is on.

continued overleaf

Setting	Key	Default	What it changes
Max vector speed, Max raster speed	<code>max_vector_speed</code> , <code>max_raster_speed</code>	140, 750	Highest reliable speeds.
Automatically lock rail	<code>autolock</code>	on	Locks the rail when a job finishes.
Phase Type, Phase Value	<code>plot_phase_type</code> , <code>plot_phase_value</code>	0, 0	Raster phase grouping: <i>Sequential</i> , <i>Random</i> , <i>Progressive</i> or <i>Static</i> , with a value out of 1000.
Pulse Grouping	<code>plot_shift</code>	off	Groups pulses in raster output.
Fix rated to actual speed	<code>fix_speeds</code>	off	These boards run at about 92% of the rated speed; enable this to correct the displayed speed.
Networked address, port	<code>address</code> , <code>port</code>	localhost, 1025	Host and port used when the interface is set to <i>Networked</i> . Windows builds default to port 1022.
Write Buffer	<code>buffer_max</code> , <code>buffer_limit</code>	900, on	Cap on the USB write queue, with the Interface panel's <i>Limit Write Buffer</i> as the visible control.

Outside the configuration window, the acceleration chart holds the vector and raster acceleration breakpoints: the groups *Vector* and *Vertical Raster* default to 25.4, 60, 127 and infinity, *Horizontal Raster* to 25.4, 127, 320 and infinity, each group with its own enable checkbox.

GRBL

The GRBL driver covers a large group of diode and CO₂ machines: Ortur and Longer diode engravers, generic GRBL boards, FluidNC controllers and K40 cutters fitted with a GRBL controller such as the MKS DLC32. It reaches the board over a serial port (a USB-serial adapter appears as a port), over TCP or WebSocket for boards with a network socket, or over a mock. The shipped device entries already fill in the bed size and homing details for common machines, so start from the entry that matches your machine. The Configuration window has the tabs Device, Interface, Protocol, Advanced, Effects, Operation Defaults, ESP3D Upload, Warning, Default Actions and Display Options.

Setting	Key	Default	What it changes
Label	<code>label</code>	<code>grbl</code>	Device name. The shipped entries use names such as <code>GRBL-DLC32-400</code> or <code>GRBL-K40-CO2</code> .
Width, Height	<code>bedwidth</code> , <code>bedheight</code>	235×235 mm	Bed size. The entries carry their own: 405×285 mm for the MKS DLC32, 400×430 mm for the Ortur, 450×450 mm for the Longer Ray5.
X-Axis, Y-Axis	<code>scale_x</code> , <code>scale_y</code>	1.000	Board units to millimetres.

continued overleaf

Setting	Key	Default	What it changes
Flip X	<code>flip_x</code>	off	Mirrors X. +X is standard for GRBL; leave it off unless your machine differs.
Flip Y	<code>flip_y</code>	on	On this machine Y goes negative into the bed. Leave it on with a top-left home corner, or Y jogs can trip the soft limit (ALARM:2).
Swap XY	<code>swap_xy</code>	off	Exchanges the axes before the flips.
Force Declared Home	<code>home_corner</code>	auto	Set the corner here if the machine homes somewhere other than the default.
Supports Z-axis	<code>supports_z_axis</code>	off	Enables Z commands for a machine with a Z axis.
Z-Homing	<code>z_home_command</code>	<code>\$HZ</code>	Which command starts the Z homing sequence: <code>\$HZ</code> or <code>G28 Z</code> .
Interface Type	<code>interface</code>	serial	<i>Serial</i> , <i>TCP-Network</i> , <i>WebSocket-Network</i> or <i>Mock</i> .
(serial port)	<code>serial_port</code>	UNCONFIGURED	Combo of the ports detected on the computer.
Baud Rate	<code>baud_rate</code>	115200	Serial speed.
Address, Port	<code>address, port</code>	localhost, 23	Host and port for the TCP interface.
Address, Port (WebSocket)	<code>address, port</code>	localhost, 81	Host and port for the WebSocket interface.
Use M3	<code>use_m3</code>	off	Starts the laser with M3 instead of M4.
Use G1 before Rapid Moves	<code>use_g1_for_power</code>	off	Sends <code>G1 S0</code> instead of <code>G0 X Y S0</code> , for firmwares that mishandle rapid moves carrying power.
Reset on 'Clear Alarm'	<code>extended_alarm_clear</code>	off	Clearing an alarm also soft-resets the controller.
Device has endstops	<code>has_endstops</code>	off	Enables physical homing with <code>\$H</code> .
Sequential homing (<code>\$HY</code> then <code>\$HX</code>)	<code>sequential_homing</code>	off	For the MKS DLC32 and machines whose limit switches sit at the top left. Homing both axes at once can stop about 50 mm short with an ALARM:1.
Simulate redden	<code>use_red_dot</code>	off	Burns a low-power visible dot for framing. GRBL has no separate red-dot output pin; this uses the laser itself.
Reddot Laser strength	<code>red_dot_level</code>	30	Strength of that dot, out of 1000.
Active during Outline	<code>use_red_dot_for_outline</code>	on	Uses the red dot automatically when outlining the burn area.

continued overleaf

Setting	Key	Default	What it changes
Max vector speed	<code>max_vector_speed</code>	140	Highest reliable speed for cutting and engraving moves.
Max raster speed	<code>max_raster_speed</code>	750	Highest reliable speed for raster moves.
Travel speed	<code>rapid_speed</code>	600	Speed of moves that do not cut.
Limit the controller buffer size	<code>limit_buffer</code>	on	Enables the buffer limit below.
Controller Buffer	<code>max_buffer</code>	200	Lines the controller may hold before sending pauses.
Require Validator	<code>require_validator</code>	off	Waits for the expected welcome text before validating a job.
Welcome Validator	<code>welcome</code>	<code>Grbl</code>	The welcome text to wait for.
Reset on connect	<code>reset_on_connect</code>	off	Soft-resets the controller when connecting.
Check sequence on connect.	<code>boot_connect_sequence</code>	on	Runs the information query when connecting.
Sending Protocol	<code>buffer_mode</code>	<code>buffered</code>	<i>buffered</i> or <i>sync</i> .
Planning Buffer Size	<code>planning_buffer_size</code>	128	Size of the planning queue.
Line Ending	<code>line_end</code>	<code>LF</code>	<i>CR</i> , <i>LF</i> or <i>CRLF</i> . <i>LF</i> is required for the SD-card files of the MKS DLC32.
Post Connection Delay	<code>connect_delay</code>	0	Milliseconds to wait after connecting before sending.
Startup commands	<code>startup_commands</code>	empty	Multiline G-code sent on connect.
Enable ESP3D Upload	<code>esp3d_enabled</code>	off	Shows the ESP3D buttons on the ribbon and unlocks the page below.
ESP3D Host, Port, SD Path	<code>esp3d_host</code> , <code>esp3d_port</code> , <code>esp3d_path</code>	192.168.1.100, 80, /	Where the ESP3D web interface lives and where its SD card is rooted.
ESP3D Username, Password	<code>esp3d_username</code> , <code>esp3d_password</code>	empty, empty	Optional login for that interface.
Cleanup Local Files	<code>esp3d_cleanup</code>	on	Deletes the local G-code once it has been uploaded.
Prepare Plan exports for SD card	<code>sd_export_prepare</code>	on	Patches a Plan export for a MKS DLC32 SD card: <i>LF</i> line endings, M3 laser mode, no G28. Home the machine yourself before running such a file.

Ruida

The Ruida driver targets CO₂ machines with a Ruida controller, the K50/K60 class. It connects over USB serial — the Serial Interface field takes a Linux device path or a Windows COM port — or over the network, where the driver talks UDP to the controller. It also ships the `ruidacontrol` emulator, which lets MeerK40t answer as a Ruida controller for LightBurn on UDP 50200 (jog on 50207), with optional man-in-the-middle (40200/40207) and LB2RD bridge (TCP 5005) modes. The Configuration window has the tabs Ruida, Interface, Configuration, Effects, Operation

Defaults, Warning, Default Actions and Display Options.

Under revision

The Ruida module is marked incomplete in the project. It has been tested on Linux only; the network emulation is described as untested and direct hardware control as not implemented. The project's own notes also require raster layers to use overscan 0 and, on the controller that was tested, Flip X to be enabled. Treat this driver as experimental.

Setting	Key	Default	What it changes
Label	<code>label</code>	<code>ruida</code>	Device name.
Width, Height	<code>bedwidth, bedheight</code>	24 in, 16 in	Bed size.
X-Axis, Y-Axis	<code>scale_x, scale_y</code>	1.000	Board units to millimetres.
Flip X	<code>flip_x</code>	off	Mirrors X. The project's notes require this to be on for the tested machine.
Flip Y	<code>flip_y</code>	off	Mirrors Y.
Swap X and Y	<code>swap_xy</code>	off	Exchanges the axes.
Force Declared Home	<code>home_corner</code>	<code>auto</code>	Which corner counts as home.
Curve Interpolation	<code>interpolate</code>	500µm	Distance at which curves are flattened.
Laser Power	<code>default_power</code>	20.0	Global default power, overridden by each operation.
Cut Speed	<code>default_speed</code>	40.0	Global default speed, overridden by each operation.
Swizzle Magic Number	<code>magic</code>	0x88	Value used to encode the job file sent to the laser.
Baud Rate	<code>baud_rate</code>	115200	Serial speed.
Serial Interface	<code>serial</code>	empty	Serial port the controller is on, for example a Linux device path or COM3.
Interface	<code>interface</code>	<code>usb</code>	<i>USB</i> or <i>Networked</i> (UDP). The address field defaults to localhost and is only used when networked.
Coolant	<code>device_coolant</code>	empty	Coolant or air-assist method.

Ruida jobs are exported as Ruida job files with the `.rd` extension. The Plan export uses `save_job <file>` and applies the swizzle magic number unless another value is passed for it. The ribbon adds `Focus Z`, which sends the controller's FocusZ command.

Moshi

The Moshi driver covers CO₂ cutters with a Moshiboard, a clone family that presents itself to the computer much like a Lihuiyu board. It connects over USB through the same CH341 chip and offers no network interface. There are no endstops, so *Home* is simply a move to position 0,0; where the job sits on the material is decided entirely by the bed size and the home corner. The Configuration window has the tabs Configuration, Effects, Operation Defaults, Warning, Default Actions and Display Options.

Setting	Key	Default	What it changes
Label	<code>label</code>	Moshiboard	Device name.
Width, Height	<code>bedwidth, bedheight</code>	330 mm, 210 mm	Bed size.
X-Axis, Y-Axis	<code>scale_x, scale_y</code>	1.000	Board units to millimetres.
Flip X, Flip Y	<code>flip_x, flip_y</code>	off, off	Mirror an axis.
Swap XY	<code>swap_xy</code>	off	Exchanges the axes before the flips.
Force Declared Home	<code>home_corner</code>	auto	Which corner counts as home.
Curve Interpolation	<code>interp</code>	5	Distance at which curves are flattened, in mils.
Use legacy raster method	<code>legacy_raster</code>	on	Legacy raster engine: better at high speed, but with artifacts.
Run mock-usb backend	<code>mock</code>	off	Connects to a fake laser instead of the real board, for debugging.
Device Position	<code>signal_updates</code>	on	Draws the live position indicator.
Coolant	<code>device_coolant</code>	empty	Coolant or air-assist method.
Travel speed	<code>rapid_speed</code>	40	Speed of moves that do not cut. Present in the source but not shown in the window.

Newly

The Newly driver covers CO₂ machines whose controller presents itself as a Newly or U-SET board: the RayLaser class and about twenty clones, including the Beijing SZTaiming, Gama, HPC, Rabbit, Lion and Jinan families. It connects over USB and has no network option. Its distinguishing idea is the file slot: a job is sent to one of ten files on the machine, File 0 runs immediately, and the others can be sent and started later from the ribbon (`File 0` to `File 9`, `Send Only` or `Send & Start`, `Draw Frame`, `Move Frame`, `Replay`). The Configuration window has the tabs Newly, Device, Global, Raster Chart, Effects, Operation Defaults, Warning, Default Actions and Display Options.

Setting	Key	Default	What it changes
Label	<code>label</code>	newly-device	Device name.
Width, Height	<code>bedwidth, bedheight</code>	310 mm, 210 mm	Bed size.
Output File	<code>file_index</code>	0	Which machine file the job is sent to, 0 to 9. File 0 executes immediately; the rest must be sent and then started.
Automatically Start	<code>autoplay</code>	on	Starts the job as soon as the output file has been sent. Uncheck to send without running.
Machine index to select	<code>machine_index</code>	0	Which attached board to use; leave at 0 with a single machine.

continued overleaf

Setting	Key	Default	What it changes
Max Power	<code>max_power</code>	20.0%	Ceiling that operation power is scaled against.
Max Pulse Power	<code>max_pulse_power</code>	65.0%	Power level at which pulses are fired.
PWM Power	<code>pwm_enabled</code>	off	Enables pulse-width modulation for power control.
PWM Frequency	<code>pwm_frequency</code>	2 kHz	1, 2, 3, 4, 5, 10, 20, 50, 75, 100, 200 or 255 kilohertz. Shown only when PWM Power is on.
Horizontal DPI, Vertical DPI	<code>h_dpi</code> , <code>v_dpi</code>	1000, 1000	Dots per inch along each axis.
Horizontal Backlash, Vertical Backlash	<code>h_backlash</code> , <code>v_backlash</code>	0 mm, 0 mm	Backlash compensation per axis.
Cut Speed, Cut Power	<code>default_cut_speed</code> , <code>default_cut_power</code>	15.0, 1000.0	Global defaults for cutting; operations override them.
Raster Speed	<code>default_raster_speed</code>	200.0	Global default raster speed.
On Delay, Off Delay	<code>default_on_delay</code> , <code>default_off_delay</code>	0 ms, 0 ms	Delay before the laser turns on and after it turns off.
Cut DC, Move DC	<code>cut_dc</code> , <code>move_dc</code>	100, 100	Current setting for cutting moves and for regular moves.
Maximum Raster Jog	<code>max_raster_jog</code>	15	Longest distance allowed in a single raster step or jog.
Speed Chart	<code>speedchart</code>	five rows	Raster chart of speed against acceleration length, backlash and corner speed: 100/8/0/20, 200/10/0/20, 300/14/0/20, 400/16/0/20 and 500/18/0/20. Rows can be added, changed and removed.
Run mock-usb backend	<code>mock</code>	off	Connects to a fake laser instead of the real board.
Curve Interpolation	<code>interp</code>	5	Distance at which curves are flattened, in mils.
Force Declared Home	<code>home_corner</code>	auto	Which corner counts as home.
Flip X, Flip Y	<code>flip_x</code> , <code>flip_y</code>	off, off	Mirror an axis.
Swap X and Y	<code>swap_xy</code>	off	Exchanges the axes.
Device Position	<code>signal_updates</code>	on	Draws the live position indicator.

Balor (galvo)

The Balor driver runs galvo marking heads built on a JCZ controller: fibre, CO₂ and UV sources, including MOPA fibre lasers. It connects over USB only — there is no host or port setting, and the device location reports either `usb` or `mock` — with a machine index and an optional serial-number check to pick the right head when several are attached. The bed is the lens field, so the Width setting is labelled *Lens Size* and defaults to 110 mm; a correction file can replace it with the measured field. The Configuration window has the tabs Balor, Redlight, Global, Timings, Extras, Effects, Operation Defaults, Warning, Default Actions and Display Options.

Setting	Key	Default	What it changes
Label	<code>label</code>	<code>balor-device</code>	Device name.
Laser Source	<code>source</code>	<code>fiber</code>	<i>fiber</i> , <i>co2</i> or <i>uv</i> .
Enable (correction file)	<code>corfile_enabled</code>	off	Uses a lens correction file.
File	<code>corfile</code>	none	The <code>.cor</code> correction file. Loading one updates Lens Size from its scale.
Width (Lens Size)	<code>lens_size</code>	110 mm	Size of the galvo field; the bed is this square.
Invert Power	<code>power_invert</code>	off	Inverts the power scale. Shown for the UV source.
Flip X, Flip Y	<code>flip_x</code> , <code>flip_y</code>	off, on	Mirror an axis.
Swap XY	<code>swap_xy</code>	on	Exchanges the axes.
Rotate View	<code>rotate</code>	0	Rotates the working view: 0, 90, 180 or 270 degrees.
Curve Interpolation	<code>interp</code>	5	Distance at which curves are flattened.
Machine index to select	<code>machine_index</code>	0	Which attached controller to use.
Check serial no	<code>serial_enable</code>	off	Requires a specific head, matched by serial number.
serial	<code>serial</code>	empty	The serial number to match. Shown when the check is enabled.
Footpedal	<code>footpedal_pin</code>	15	GPIO pin the foot pedal is wired to.
Redlight laser	<code>light_pin</code>	8	GPIO pin the red pilot laser is wired to.
Pedal action	<code>pedal_mode</code>	<code>ignore</code>	What a pedal press does: <code>ignore</code> , pause and resume the job, pause while pressed, stop the job, or arm and start the job.
Active on low signal	<code>pedal_active_low</code>	on	Treats the pedal as pressed when the signal is low.
Pedal poll interval	<code>pedal_poll_interval</code>	0.25	Seconds between checks of the pedal.
Redlight travel speed	<code>redlight_speed</code>	3000	Speed of the galvo while using the red pilot laser.
Dark, Light	<code>redlight_delay_dark</code> , <code>redlight_delay_light</code>	1, 1	Microseconds of delay for dark and for light redlight moves.
X-Offset, Y-Offset, Angle Offset	<code>redlight_offset_x</code> , <code>redlight_offset_y</code>	0 mm, 0 mm	Shifts the redlight positions in x and in y. The angle offset, <code>redlight_angle</code> , curves them around the centre and also defaults to 0°.
Prefer redlight on	<code>redlight_preferred</code>	off	Turns a toggleable redlight back on when a job completes.
Restart light jobs	<code>restart_light_jobs</code>	off	Restarts light jobs after they finish.

continued overleaf

Setting	Key	Default	What it changes
Power	<code>default_power</code>	500.0	Global default power; operations override it.
Speed	<code>default_speed</code>	100.0	Global default marking speed.
Q Switch Frequency	<code>default_frequency</code>	30.0	Q-switch frequency in kilohertz.
First Pulse Killer	<code>default_fpk</code>	10.0%	Percentage of first-pulse suppression, for the CO ₂ and UV sources.
Travel Speed	<code>default_rapid_speed</code>	2000.0	Speed used when not marking.
Enable (Pulse Width)	<code>pulse_width_enabled</code>	off	MOPA heads: use pulse width as well as power.
Set Pulse Width (ns)	<code>default_pulse_width</code>	4	Pulse width in nanoseconds, chosen from a fixed list: 1, 2, 4, 6, 9, 13, 20, 30, 45, 55, 60, 80, 100, 150, 200 or 250. Shown when pulse width is enabled.
Laser On, Laser Off	<code>delay_laser_on</code> , <code>delay_laser_off</code>	100, 100	Microseconds of delay at the start and at the end of each marking command.
Polygon Delay	<code>delay_polygon</code>	100	Microseconds of delay between points along a path.
End Delay	<code>delay_end</code>	300	Microseconds of delay at the end of the job.
Long jump delay, Short jump delay	<code>delay_jump_long</code> , <code>delay_jump_short</code>	200, 8	Microseconds of delay for long and for short jumps.
Run mock-usb backend	<code>mock</code>	off	Connects to a fake controller instead of the real head.
Device Position	<code>signal_updates</code>	on	Draws the live position indicator.

Where to check the current value

The Configuration window always shows the live values, under the same labels as the tables above: open the **Devices** pane, select the device and press **Config**. Three console commands help when a window is not enough.

Command	What it shows
<code>device</code>	The device entries that could be added, with their friendly names and providers.
<code>device activate <name></code>	Makes another device the active one, so its settings are the ones in force.
<code>devinfo</code>	The current logical and native position, printed in the form <code>x,y;nx,ny;</code> . Compare the two to see how the driver's transform is placing the scene on the bed.

Settings can also be read and written from the console. The `set` command works on variables in a context: `set -p <context> <key> <value>` writes one value, and `set -p <context>` with no value lists what that context currently holds — the reliable way to confirm the exact key names and their present values on your installation.

For diagnosis, the driver's own controller window shows the packet flow and the counters, the **Buffer** window shows the command buffer being fed to the device, and the *UsbConnect* window shows the USB pipe and accepts typed input. What those windows report, and what to do about it, is covered in Chapter 4.

Chapter D

Glossary, help and reporting problems

The tutorial chapters use their words without ceremony and expect you to keep moving. This appendix is the one to keep beside the machine: what the vocabulary means, where the program sends you when you ask for help, how to report a fault so that it can actually be fixed, which hardware is currently known to work, and a checklist to run through before the beam comes on.

Glossary

Terms are listed as an operator meets them, in alphabetical order. Where a term has a specific meaning inside MeerK40t — rather than the general laser-shop meaning — that meaning is the one given.

Term	Meaning
Bed	The flat working area of the machine, and the white rectangle MeerK40t draws for it in the scene. Its size comes from the device's bedwidth and bedheight settings, so a bed with the wrong proportions is a device-configuration problem, not a drawing problem (Chapter 4).
Blob	The device-specific data that a plan is converted into before it is streamed to the controller — the blob stage of planning. A blob shows up as a <i>blob</i> node in the Tree , and can be executed, converted back to elements, or deleted.
Burn	Running a job with the beam on: the machine is marking or cutting material. Inside MeerK40t “burn” also names the work itself — the Burn-Operation tab of the Tree lists what will burn and in what order, and the status bar reports burn progress.
Classification	The rule that decides which operation an element belongs to, by comparing the element's stroke and fill colours with the colours of your operations. Whether matching is exact or approximate, whether unmatched elements go to a “default” operation, and whether MeerK40t may create a new operation for them are all set on the Classification tab of the Preferences window (Chapter 7).
Cutcode	MeerK40t's internal list of laser primitives — lines, arcs, curves, raster sweeps, dwells and moves — each carrying the settings it must be burned with. Operations are turned into cutcode, and the drivers consume cutcode; you never edit it directly.
DPI	Dots per inch: how many raster scan lines fit into an inch of travel. It is the reciprocal of the raster step, so doubling the DPI halves the step, roughly doubles the burn time, and only pays off while the step is still larger than your laser's spot (Chapter 8).
Dwell	A deliberate pause at one point with the beam on, used to mark a dot or to pierce material. MeerK40t's Dots operation carries a dwell time and produces one dwell per point element attached to it.

Term	Meaning
Effect	A node that rewrites geometry instead of burning something itself. <i>Hatch</i> fills a closed shape with parallel lines, <i>Wobble</i> follows the outline with a wavy contour, and <i>Warp</i> distorts geometry into a grid. Effects can be attached to elements or appended to a Cut or Engrave operation.
Element	One piece of artwork in the Tree : a path, rectangle, ellipse, line, polyline, point, text, image or blob. Elements live in the <i>Elements</i> branch; operations do not contain them, they hold references to them.
Emphasis (selected)	The emphasised nodes are the current selection — the ones the toolbar, the property panes and the console act on. Emphasising an element also highlights the operations that reference it, and emphasising an operation highlights its sub-operations and the elements it uses.
Engrave	A vector operation that traces a path with the beam on but does not cut through: thin, dark lines, good for text and detail. The same geometry burned with a Cut operation and heavier settings is a cut (Chapter 7).
Frame / outline	Running the machine around the outside of a job so you can see where it will land before burning it. In MeerK40t the outline <offset> command — also Outline element(s)... on an element — creates an offset outline path around the emphasised elements, and some drivers add machine-side frame buttons (Newly: Draw Frame , Move Frame).
Galvo	A marking head that steers the beam with mirrors across a fixed field instead of moving a gantry, as on fibre and UV marking machines. MeerK40t drives Ezcad2-compatible JCZ galvo controllers through the Balor driver.
G-code	The motion language used by GRBL-family controllers. MeerK40t generates G-code for GRBL devices and can export it as .gcode , .nc or .gc ; K40, Ruida, Moshi and Newly machines speak their own protocols instead.
Hatch	A fill made of parallel lines inside a closed shape, with a hatch distance and angle (algorithms: scanline, Eulerian, spiral). Hatching is how an area is engraved in sweeps instead of outline by outline.
Home	A fixed position the machine can find for itself, usually with limit switches; the home command and the controller's own Home button send the head there. Home is the machine's reference point and is not necessarily the origin a job is placed from (Chapter 4).
Image operation	The Image operation, which engraves an existing image pixel by pixel with the operation's own settings. It is not the same as Raster , which first renders everything attached to it into one intermediate image and then burns that.
Job	One compiled unit of work: the operations built into cutcode, wrapped up and handed to the spooler to stream to the machine. Jobs appear in the Spooler pane with their status, passes, runtime and estimate.
Job origin (job start, user origin)	The point a job is placed from, so that the same artwork can be burned at several positions on the bed. In MeerK40t it is a <i>placement</i> node under the operations branch (Define Job Start Position), and you may define several.
Kerf	The width of material that the beam destroys. A part drawn 20 mm wide comes out narrower unless you compensate, which is why operations expose a kerf value and MeerK40t can offset paths by half of it.
Laser path	The preview of where the beam will actually travel, drawn over the artwork with the direction of each cut. Toggle it under View ► Scene Appearance .
Lid / interlock	The safety switch that stops the beam when the enclosure is opened. MeerK40t cannot see it and cannot override it; a machine whose lid switch has been defeated is a machine that will injure someone.

Term	Meaning
Lihuiyu	The driver family for Lihuiyu M2-Nano and M3-Nano boards — the original K40 controller. USB only, vector and raster both supported. See C for its settings.
Machine origin	The zero point the machine itself works in, fixed by homing or by the controller. Where it physically sits — front-left, centre, or somewhere else — is decided by your device profile and controller, so establish it with a light test on scrap before you cut anything valuable.
Moshi	The driver for Moshiboard controllers, found in older laser machines. USB, with the board's own command set.
Newly	The driver for NewlyDraw System 8.1 machines. USB, with its own file slots on the controller and Draw Frame / Move Frame controls for checking placement.
Op	A single operation node in the Tree , holding one recipe: its type, colour, speed, power, passes and effects. An op is what you select, edit and drag elements onto.
Operation	The recipe for burning a group of artwork: cut, engrave, raster, image or dots, plus its settings. Operations are listed in the <i>Operations</i> branch and are planned in order, top to bottom (Chapter 7).
Overscan	Extra travel added at each end of a raster line so that the head is already at full speed when it reaches the image. Without it, the ends of each sweep linger and burn darker.
Passes	How many times an operation repeats the same path or sweep. Two passes at moderate power usually cut cleaner than one at maximum, especially in plywood and acrylic.
PPI	Pulses per inch: how many laser pulses are emitted per inch of travel, shown as <i>frequency</i> in kilohertz on machines that expose it. It changes how an edge looks more than whether the material cuts at all.
Plan	The working copy of the selected operations that the planner builds, stage by stage, and then hands to the spooler. Plans are numbered, so you can hold several at once; the console's plan commands act on them (Chapter 16).
Preopt / optimize	The two optimisation stages of planning. preopt prepares the plan for reordering and adds the optimiser passes; optimize runs them, reducing travel and cutting inner contours before outer ones. With optimisation switched off, a plan skips both.
Profile	A saved machine configuration — a device entry with its driver, bed size, origin and settings. Profiles are chosen and edited in the Devices pane; the device chooser at first start offers generic profiles grouped under each driver.
Raster	Pixel artwork, and the operation that burns it line by line. A Raster operation renders everything attached to it into one image and sweeps that image; a raster engrave is much slower than a vector one over the same area (Chapter 6).
Reference	A node under an operation that points at an element in the elements branch, which is how one element can belong to several operations without being copied. (The alignment “reference object” — an element other elements are aligned to — is a different thing with the same name.)
Regmark	A registration mark: artwork used only to align the work — with a camera, a jig, or the machine's own pointer. Regmarks live in their own <i>Regmarks</i> branch so they are never burned, and invisible objects in imported files can be loaded there.
Rotary	A rotating attachment that turns a cylindrical workpiece so the laser can engrave around it. MeerK40t configures rotary work per device and also has support for cylindrical material on flat beds.

Term	Meaning
Ruida	The driver for Ruida controllers, the black-and-yellow CO ₂ machines with an RDC6442- or RDC6445-class panel. Network first, and able to emulate a Ruida controller for other software.
Spooler	The queue of jobs waiting for the machine, and the pane that shows it (Spooler , docked at the bottom of the main window on first use). It reports each job's items, steps, passes, runtime and estimate, and carries the Pause and Abort controls.
Step (raster step)	The distance between adjacent raster scan lines, in millimetres or inches. It is the number MeerK40t actually works in; DPI is the same quantity expressed as lines per inch, and around 500 dpi the step is about 0.05 mm.
SVG	The vector file format MeerK40t imports best, and also its native project format: File ► Save writes your project — artwork, operations and notes — out as SVG.
TTF / Hershey	The font kinds available to vector (line) text: TrueType (.ttf), Hershey (.jhf) and Autocad shape (.shx) fonts. All three are converted to paths that can be scaled and cut like any other vector shape (Chapter 6).
Vector	A shape described mathematically — a line, arc or curve — rather than as pixels. Vectors scale without degrading and are burned by tracing the path, which is why they cut and score.
Wobble	An effect that makes the beam follow a wavy contour around a path — circle, sine wave, sawtooth, jigsaw, gear and other shapes — with a radius, an interval and a speed. It produces decorative, stitch-like edges and wider, softer cuts.
Wordlist	A named list of text values (optionally CSV-backed) whose entries are substituted into text at burn time, advancing one entry per burn. Used for serial numbers, batch labels and the like.
Work area	The region the machine can actually reach, which is the bed minus whatever the head, the gantry or a rotary attachment takes away. Elements outside the work area are either unburnable or will be cut somewhere unexpected; the Tree warns about them.
Zero, 0,0	The coordinate origin the scene measures positions from. A <i>Goto</i> utility node placed at 0,0 is shown in the Tree as <i>Origin</i> , which is a convenient way to see where the zero point of a job sits.

Getting help

MeerK40t is developed by a community that mostly answers questions in public forums rather than by email, and the **Help** menu is the front door to those places. The entries are always the same; only the addresses they open change.

Menu entry	What it opens
Help ► Help (F1) and Help ► Online Reference (F11)	Context-sensitive online help. MeerK40t looks at the widget under the mouse pointer, walks up through its parents until it finds help text, and opens the wiki page for that section; if nothing under the pointer has help text, it opens the general interface section instead.
Help ► Beginners' Help	The wiki index for new users: https://github.com/meerk40t/meerk40t/wiki/Beginners:-0.-Index . Start here when the manual has not answered the question.
Help ► Github	The project page: https://github.com/meerk40t/meerk40t .
Help ► Releases	The release list, where installers, stable releases and weekly builds live: https://github.com/meerk40t/meerk40t/releases .
Help ► Facebook	The K40 MeerK40t group: https://www.facebook.com/groups/716000085655097 .
Help ► Discord	The project's Discord server, where developers and users talk: https://discord.gg/vkDD3HdQq6 .
Help ► Makers Forum	The MeerK40t section of the Makers Forum: https://forum.makerforums.info/c/k40/meerk40t/120 .
Help ► Feature request/feedback	The GitHub Discussions chooser for reporting requests and ideas: https://github.com/meerk40t/meerk40t/discussions/new/choose . Feature requests belong here rather than in the issue tracker.
Help ► Check for Updates	Runs the update check immediately and reports the result in an <i>Update-Info</i> dialog.
Help ► Tips && Tricks	The Tips window: previous and next tip, a Try it out button, and switches for showing tips at startup.
Help ► About MeerK40t	The About window, described below.

Other doors into the same project

On macOS the first two entries differ: **MeerK40t Help** opens a local help file shipped with the program, and **Online Help** opens the wiki root. The bug tracker — <https://github.com/meerk40t/meerk40t/issues> — is also reachable from the console with **webhelp issues**, although it is not on the Help menu. The project asks that the tracker be kept for actionable bug reports and that general questions go to the forums or Discussions.

The About window

Help ► About MeerK40t opens a window with four tabs. Two of them exist for bug reports, and you should treat them as part of the tool rather than as decoration:

About

The program version, the project's self-description and the credits — including the people, translators and icon set that made the program.

In Memory of David Olsen

A memorial to David Olsen (1982–2024), known as Tatarize, who wrote MeerK40t, with a short history of the project.

System-Information

The exact MeerK40t version, the folder holding your configuration file, and a full report of your operating system, release, machine and processor. It has **Copy to Clipboard** and **Check for Updates** buttons. This is the fastest way to state your environment in a bug report.

MeerK40t-Components

A table of the optional and required libraries MeerK40t found — Python, wxPython, numpy, pillow, the tracing engines, serial and USB support, OpenCV, the clipping library and the rest — with a *Version* and a *Status* column and a link to each project’s home. A component marked *Missing* or *Faulty* explains a whole class of “the feature does not work” reports, so include this table when you file one.

Updates

Whether MeerK40t looks for new versions is a preference, on the **Options** page of the Preferences window, under “Check for updates on startup”: *Action* chooses between checking not at all, for major releases, or for major and beta releases, and *Frequency* chooses between every startup, once a day or once a week. **Help ► Check for Updates** performs the same check on demand. The answers MeerK40t can give are that a newer release is available, that you already have the latest version for your system, or that no release information could be retrieved from GitHub. The same check is available in the console as `check_for_updates`, with `--beta` to include prereleases.

Reporting a problem so it can be fixed

A bug report that can be acted on contains five things: what you ran, what you expected, what happened, the text of the error, and the smallest example that shows it. Work down this list and you will have all five.

1. **Read the console.** Open the **Console** pane (the ribbon’s **Console** button, or **Panes ► Console**) and scroll back through it with the error visible. MeerK40t reports failures, warnings about unburnable or unassigned elements, and the planning stages here. Copy the text rather than paraphrasing it — a Python traceback’s last line and a driver’s error code are the parts that identify the fault.
2. **Note the exact version.** The title bar shows the program name, its version and the active device; the About window’s **About** and **System-Information** tabs show the same string; `meerk40t --version` prints it on its own. A build from a git checkout appends *git* and the branch, a source tree appends *src*, and an installed package appends *pkg* — that suffix matters, because it tells maintainers which artefact you are running.
3. **Say which machine and which profile.** Name the device entry from the **Devices** pane (the highlighted row is the active one), the driver behind it, and how it is connected — USB, serial, network or not connected at all. A fault that only happens on one driver is a different report from one that happens with no device at all.
4. **Include the diagnostics.** Use **Copy to Clipboard** on the **System-Information** tab, and again on the **MeerK40t-Components** tab, and paste both into the report. If the problem is about speed, `-f/--profiler` records a Python profile of a session to a file, which is the useful evidence there.
5. **Attach the smallest file that reproduces it.** Delete everything from a copy of the project that is not needed to trigger the fault, save it, and check that the reduced file still misbehaves. An imported original plus your saved project is often enough; a file with one rectangle in it is far more likely to be investigated than a 20 MB artwork.
6. **Describe the steps.** “Do this, then this, then that” — including the settings involved (`dpi`, `bedwidth`, the operation’s speed and power) and whether the problem repeats on a second attempt. State what you expected to happen as well as what did.
7. **Search before filing.** Look through the issue tracker and Discussions for the same symptom; adding your version and diagnostics to an existing report is more useful than opening a near-duplicate.

8. **Match your expectations to the project's state.** MeerK40t is in a low-maintenance phase: issues and pull requests may wait a long time for an answer, and silence usually means lack of time rather than lack of interest. Weekly builds are automated snapshots that will contain regressions — if you are chasing a bug in one, say so, and reproduce it on the latest stable release before reporting it as a stable-release bug.

Project status and driver support

MeerK40t is currently in a low-maintenance phase: the original maintainers have limited bandwidth for active development, so issues and pull requests may not receive immediate feedback. Weekly builds are automated experimental snapshots that will contain bugs and regressions — if you need a reliable machine, stay on the latest stable release — and the fastest route to a fix for something you find is usually a pull request or a clear, reproducible report.

Hardware support is tracked separately from program features. The table below reproduces the project's own status list; treat the notes column as the honest summary of what you are buying into.

Device type	Use status	Development status	Notes
Lihuiyu	Minor issues	Under revision	Users report raster Y-axis registration issues.
GRBL	Okay	Stable	No issues known.
Moshi	Okay	Stable	No issues known.
Newly	Okay	Stable	No issues known.
Ruida	Okay	Under revision	Fit for testing (rather than production).
Balor	Okay	Stable	No issues known.

Status as published by the project on 17/12/2025. In the project's own legend, *Okay* means generally reliable for production use, *Minor issues* means usable but liable to occasional problems, *Stable* means the core functionality is mature and well tested, and *Under revision* means active development with changes expected. The driver-specific settings that go with each of these families are collected in C.

Final safety checklist

Tick all of these before pressing **Start**. They take a minute and they cover the failures that ruin work, burn material or start fires — in roughly that order of likelihood.

Before the beam comes on

- ☐ Enclosure closed, lid shut, every interlock intact and working — no bypassed switches, no tape over sensors.
 - ☐ Air assist on, if your machine has it; if it is off, you have decided that deliberately.
 - ☐ Focus checked for this material and this thickness, with the machine's own gauge or jig.
 - ☐ Workpiece secured: clamped, taped or magnetised, and clear of the head's travel over the whole job.
 - ☐ Material identified with certainty. If you do not know what it is, do not cut it — PVC, polyurethane, vinyl, ABS and unknown composites release poisonous fumes.
 - ☐ Settings proven on scrap of the same material and thickness, not assumed from a forum post or a previous job.
 - ☐ Bed size and origin correct for this machine, and the job inside the work area — the job is not going to run off the bed or into a clamp.
 - ☐ Material thickness and kerf accounted for, so that parts will fit after the cut rather than before it.
 - ☐ Job estimated in Simulation or in the **Spooler** estimate column, and the time and material cost accepted.
 - ☐ Nothing flammable inside the enclosure that is not part of the job: no loose offcuts, no paper, no cleaning rags, no solvent.
 - ☐ Extinguisher within reach and suitable for the material, and you know where the pause and stop controls are — on the machine and in the **Laser-Control** pane.
 - ☐ You intend to stay with the machine for the whole job. It is not left unattended, ever.
-